

Spring 基础

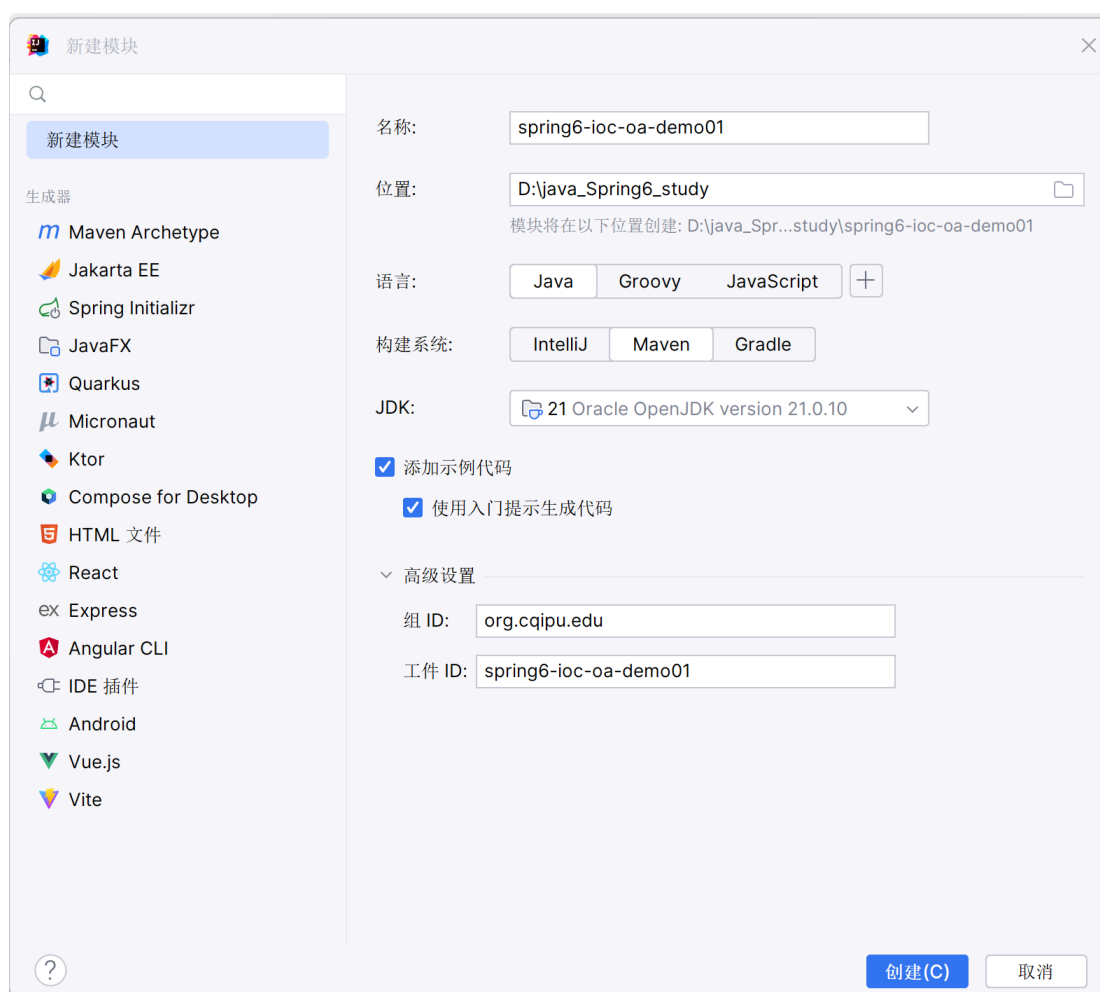
参考动力节点的Spring6笔记：<https://www.yuque.com/dujubin/ltckqu/kipzgd?#oJHkp>

哔哩哔哩教程：https://www.bilibili.com/video/BV1Ft4y1g7Fb/?spm_id_from=333.337.search-card.all.click&vd_source=c899d1ec12749d974be5871ce6410cd7

Spring 启示录

为什么需要 Spring?

在开始学习 Spring 框架之前，我们需要先理解**原生 Java 开发的痛点**——这正是 Spring 诞生的核心原因。本文将基于 **org.cqipu.edu** 包下的 OA 系统登录功能代码，一步步揭示 Spring 框架的设计初衷与核心价值。



目录结构

严格遵循 Maven 标准目录结构

```
src/
├── main/
│   ├── java/
│   │   └── org/
│   │       └── cqipu/
```

```
├── edu/
│   ├── controller/ # 控制层：处理请求触发业务逻辑
│   │   └── UserController.java
│   ├── service/ # 业务层：封装业务逻辑
│   │   └── UserService.java
│   ├── service/impl/ # 业务层实现
│   │   └── UserServiceImpl.java
│   ├── dao/ # 数据访问层：定义数据库操作接口
│   │   └── UserDao.java
│   ├── dao/impl/ # 数据访问层实现：不同数据库适配
│   │   ├── UserDaoImplForMySQL.java
│   │   └── UserDaoImplForOracle.java
│   └── bean/ # 实体类：封装数据
│       └── User.java
├── resources/
│   └── beans.xml # Spring IoC配置文件
└── test/
    ├── java/
    │   └── org/
    │       └── cqipu/
    │           └── edu/
    │               └── TestOASpring.java # 测试类
```

核心代码编写

- 实体类：User.java
 - 封装用户数据的实体类，对应数据库中的用户表：

```
package org.cqipu.edu.bean;

public class User {
    private Long id;
    private String username;
    private String password;

    // Getter和Setter方法（必须提供，用于Spring数据注入）
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

- 数据访问层接口：UserDao.java
 - 定义数据库操作的规范，实现面向接口编程（这是解耦的基础）

```

package org.cqipu.edu.dao;

import org.cqipu.edu.bean.User;

public interface UserDao {
    /**
     * 根据用户名和密码查询用户
     * @param username 用户名
     * @param password 密码
     * @return 用户对象 (null表示未找到)
     */
    User selectByUsernameAndPassword(String username, String password);
}

```

- 业务层接口：UserServiceImpl.java
 - 定义业务逻辑的规范，封装登录业务的核心流程

```

package org.cqipu.edu.service;

public interface UserService {
    /**
     * 登录验证
     * @param username 用户名
     * @param password 密码
     * @return 验证结果 (true表示成功, false表示失败)
     */
    boolean login(String username, String password);
}

```

- 业务层实现：UserServiceImpl.java
 - 业务层的具体实现，依赖UserDao接口完成数据查询

```

package org.cqipu.edu.service.impl;

import org.cqipu.edu.bean.User;
import org.cqipu.edu.dao.UserDao;
import org.cqipu.edu.dao.impl.UserDaoImplForMySQL;
import org.cqipu.edu.service.UserService;

public class UserServiceImpl implements UserService {
    // 原生开发痛点：硬编码依赖具体实现类，而非接口
    private UserDao userDao = new UserDaoImplForMySQL();

    @Override
    public boolean login(String username, String password) {
        // 调用Dao层查询用户
        User user = userDao.selectByUsernameAndPassword(username, password);
        // 业务逻辑：用户存在则登录成功
        return user != null;
    }
}

```

- MySQL 数据访问实现：UserDaoImplForMySQL.java
 - 针对 MySQL 数据库的具体实现类，负责实际的数据库连接与查询：

```

package org.cqipu.edu.dao.impl;

```

```
import org.cqipu.edu.bean.User;
import org.cqipu.edu.dao.UserDao;

public class UserDaoImplForMySQL implements UserDao {
    @Override
    public User selectByUsernameAndPassword(String username, String password) {
        // 实际开发中会使用JDBC/MyBatis连接MySQL
        // 示例：模拟查询结果（此处仅为演示，实际需编写数据库操作代码）
        System.out.println("使用MySQL数据库查询用户: " + username);
        return null; // 实际场景需返回查询到的User对象
    }
}
```

可以看出，UserDaoImplForMySQL中主要是连接MySQL数据库进行操作。如果更换到Oracle数据库上，则需要再提供一个UserDaoImplForOracle，如下：

- Oracle 数据访问实现：UserDaoImplForOracle.java
 - 针对 Oracle 数据库的具体实现类，与 MySQL 实现类完全解耦

```
package org.cqipu.edu.dao.impl;

import org.cqipu.edu.bean.User;
import org.cqipu.edu.dao.UserDao;

public class UserDaoImplForOracle implements UserDao {
    @Override
    public User selectByUsernameAndPassword(String username, String password) {
        // 实际开发中会使用JDBC/MyBatis连接Oracle
        System.out.println("使用Oracle数据库查询用户: " + username);
        return null; // 实际场景需返回查询到的User对象
    }
}
```

可以看出，UserDaoImplForMySQL中主要是连接MySQL数据库进行操作。如果更换到Oracle数据库上，则需要再提供一个UserDaoImplForOracle，如下：

```
package org.cqipu.edu.service.impl;

import org.cqipu.edu.bean.User;
import org.cqipu.edu.dao.UserDao;
import org.cqipu.edu.dao.impl.UserDaoImplForMySQL;
import org.cqipu.edu.service.UserService;

public class UserServiceImpl implements UserService {
    // 原生开发痛点：硬编码依赖具体实现类，而非接口
    // private UserDao userDao = new UserDaoImplForMySQL();
    private UserDao userDao = new UserDaoImplForOracle();

    @Override
    public boolean login(String username, String password) {
        // 调用Dao层查询用户
        User user = userDao.selectByUsernameAndPassword(username, password);
        // 业务逻辑：用户存在则登录成功
        return user != null;
    }
}
```

- 控制层：UserController.java
 - 控制层，负责触发登录业务逻辑（模拟 Web 请求入口）



```
package org.cqipu.edu.controller;

import org.cqipu.edu.service.UserService;
import org.cqipu.edu.service.impl.UserServiceImpl;

public class UserController {
    // 原生开发痛点：硬编码依赖具体实现类，而非接口
    private UserService userService = new UserServiceImpl();

    /**
     * 模拟登录请求
     */
    public void login() {
        // 模拟前端传入的用户名和密码
        String username = "admin";
        String password = "123456";
        // 调用业务层验证
        boolean success = userService.login(username, password);
        // 处理结果（模拟返回响应）
        if (success) {
            System.out.println("登录成功！欢迎回来，" + username);
        } else {
            System.out.println("登录失败：用户名或密码错误");
        }
    }
}
```

原生开发的痛点分析

以上代码是典型的三层架构（Controller→Service→Dao），但在原生 Java 开发模式下，存在无法忽视的致命问题 —— 这正是 Spring 要解决的核心]痛点。

痛点 1：硬编码导致的强耦合

代码中所有对象的创建都通过new关键字 **硬编码实现**，导致 **上层代码直接依赖具体实现类，而非接口**：

- UserController直接依赖UserServiceImpl（而非UserService接口）；
- UserServiceImpl直接依赖UserDaoImplForMySQL（而非UserDao接口）。

这种设计违反了 **依赖倒置原则（DIP）**：高层模块不应依赖低层模块的具体实现，而应依赖其抽象（接口）。

痛点 2：扩展或切换实现需修改源码

若要将数据库从 MySQL 切换到 Oracle，必须修改UserServiceImpl的源码：



```
// 原代码（依赖MySQL）
private UserDao userDao = new UserDaoImplForMySQL();

// 修改后（依赖Oracle）
private UserDao userDao = new UserDaoImplForOracle();
```

这违反了 **开闭原则（OCP）**：软件实体应对扩展开放，对修改关闭。

实际项目中，若有 100 个类似的UserServiceImpl类，切换数据库需修改 100 处源码 —— 不仅效率极低，还可能引入新 Bug。

痛点 3：对象管理全靠手动

原生开发中，程序员需手动完成**所有对象的创建、依赖赋值与生命周期管理**：

- 若 UserDao 依赖数据库连接池，需手动创建连接池对象并注入 UserDao；
- 若 UserService 依赖多个 Dao，需手动创建所有 Dao 对象并赋值。

项目越大，对象数量越多，手动管理的复杂度呈**指数级上升**—— 程序员的精力被消耗在“创建对象”而非“编写业务逻辑”上。

Spring 的解决方案：IoC 容器（控制反转）

控制反转IoC

控制反转（Inversion of Control，缩写为 IoC）不是某一种具体的技术，而是面向对象编程中的一种设计思想，可以用来降低代码之间的耦合度，符合依赖倒置原则。

- 没有 IoC：程序员需要写核心业务逻辑，又要手动创建所有对象、管理对象依赖、处理对象销毁；
- 有了 IoC：程序员只需要专注业务，把创建对象、管理依赖、控制生命周期交给 Spring 容器，这就是控制权反转。对象创建或管理的控制权，从程序员手里反转到了 Spring 容器手里。

控制反转的核心是：**将对象的创建权交出去，将对象和对象之间关系的管理权交出去，由第三方容器来负责创建与维护。**

控制反转常见的实现方式：依赖注入（Dependency Injection，简称DI）

通常，依赖注入的实现又包括两种方式：

- set方法注入
- 构造方法注入

而Spring框架就是一个实现了IoC思想的框架。

IoC可以认为是一种**全新的设计模式**，但是理论和时间成熟相对较晚，并没有包含在GoF中。（GoF指的是23种设计模式）

IoC 的核心逻辑

IoC 并非技术，而是一种**设计思想**，其核心逻辑可总结为 3 句话：

1. **程序员不 new 对象**：所有对象的创建由 Spring 容器负责；
2. **配置文件定义对象**：通过 XML 或注解告诉 Spring “要创建哪些对象”；
3. **容器自动注入依赖**：Spring 自动处理对象间的依赖关系（谁依赖谁）。

基于 XML 的 IoC 改造

- 在pom.xml中添加 Spring 核心依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.cqipu.edu</groupId>
    <artifactId>spring6-ioc-oa-demo01</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- 核心配置：指定JDK21编译版本 -->
    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
    </properties>
</project>
```

```

        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <spring.version>6.0.10</spring.version>
    </properties>

    <dependencies>
        <!-- Spring6 核心上下文依赖（包含IoC容器） -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>6.0.10</version>
        </dependency>
    </dependencies>

</project>

```

给需要依赖注入的属性添加setter方法（这是 Spring 实现依赖注入的基础）。

- 改造 1: UserController添加setUserService方法

```

● ● ●
package org.cqipu.edu.controller;

import org.cqipu.edu.service.UserService;
import org.cqipu.edu.service.impl.UserServiceImpl;

public class UserController {
    // 移除硬编码new，保留私有属性
    private UserService userService;

    // 添加setter方法（Spring将通过此方法注入依赖）
    public void setUserService(UserService userService) {
        this.userService = userService;
    }

    // 原login方法完全不变
    public void login() {
        String username = "admin";
        String password = "123456";
        boolean success = userService.login(username, password);
        if (success) {
            System.out.println("登录成功！欢迎回来，" + username);
        } else {
            System.out.println("登录失败：用户名或密码错误");
        }
    }
}

```

- 改造 2: UserServiceImpl添加setUserDao方法

```

● ● ●
package org.cqipu.edu.service.impl;

import org.cqipu.edu.bean.User;
import org.cqipu.edu.dao.UserDao;
import org.cqipu.edu.service.UserService;

public class UserServiceImpl implements UserService {
    // 移除硬编码new，保留私有属性
    private UserDao userDao;

    // 添加setter方法（Spring将通过此方法注入依赖）
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }
}

```

```
// 原login方法完全不变
@Override
public boolean login(String username, String password) {
    User user = userDao.selectByUsernameAndPassword(username, password);
    return user != null;
}
}
```

- 编写 Spring 配置文件 (beans.xml)
 - 在src/main/resources目录下创建beans.xml，告诉 Spring：
 - 要创建哪些对象 (Bean)；
 - 对象的全限定类名 (class属性)；
 - 对象间的依赖关系 (property标签)。
 - 配置文件内容如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- 1. 配置Dao层Bean: MySQL实现 -->
    <bean id="userDao" class="org.cqipu.edu.dao.impl.UserDaoImplForMySQL"/>

    <!-- 2. 配置Service层Bean，并注入Dao依赖 -->
    <bean id="userService" class="org.cqipu.edu.service.impl.UserServiceImpl">
        <!-- property: 注入依赖 -->
        <!-- name: 对应UserServiceImpl中的属性名(userDao) -->
        <!-- ref: 对应要注入的Bean的id(即上面的userDao) -->
        <property name="userDao" ref="userDao"/>
    </bean>

    <!-- 3. 配置Controller层Bean，并注入Service依赖 -->
    <bean id="userController" class="org.cqipu.edu.controller.UserController">
        <property name="userService" ref="userService"/>
    </bean>

</beans>
```

- 编写测试类

在src/test/java/org/cqipu/edu目录下创建TestOASpring，从 Spring 容器中获取对象，而非手动new：

```
package org.cqipu.edu;

import org.cqipu.edu.controller.UserController;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class TestOASpring {
    public static void main(String[] args) {
        // 1. 创建Spring容器，加载类路径下的beans.xml配置文件
        // ClassPathXmlApplicationContext: 自动从classpath (src/main/resources) 加载配置
        ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");

        // 2. 从容器中获取UserController对象(无需手动new，依赖已自动注入)
        UserController controller = (UserController) context.getBean("userController");
    }
}
```

```
// 3. 调用登录方法（业务逻辑完全不变）
controller.login();
}
}
```

执行结果：

```
D:\Java21\bin\java.exe "-javaagent:D:\JAVAidea\IntelliJ IDEA 2023.2\lib\idea_rt.jar=4712:D:\JAVAidea\IntelliJ IDEA 2023.2\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath D:\java_Spring6_study\spring6-ioc-oa-demo01\target\test-classes;
D:\java_Spring6_study\spring6-ioc-oa-demo01\target\classes;C:\Users\jack\.m2\repository\org\springframework\spring-context\6.0.10\spring-context-6.0.10.jar;
C:\Users\jack\.m2\repository\org\springframework\spring-aop\6.0.10\spring-aop-6.0.10.jar;C:\Users\jack\.m2\repository\org\springframework\spring-beans\6.0
.10\spring-beans-6.0.10.jar;C:\Users\jack\.m2\repository\org\springframework\spring-core\6.0.10\spring-core-6.0.10.jar;C:\Users\jack\
.m2\repository\org\springframework\spring-jcl\6.0.10\spring-jcl-6.0.10.jar;C:\Users\jack\.m2\repository\org\springframework\spring-expression\6.0
.10\spring-expression-6.0.10.jar org.cqipu.edu.Test0ASpring
使用MySQL数据库查询用户: admin
登录失败: 用户名或密码错误
```

切换数据库到 Oracle

若要切换到 Oracle 数据库，仅需修改beans.xml中的 Dao Bean 配置，无需改动任何 Java 业务代码：

```
<!-- 将userDao的class属性改为Oracle实现类 -->
<bean id="userDao" class="org.cqipu.edu.dao.impl.UserDaoImplForOracle"/>
```

这就是 Spring IoC 的核心价值：配置化实现扩展，无需修改源码。

总结

通过上述改造，我们可以总结出 Spring 框架的 3 个核心启示 —— 这也是 Spring 能成为 Java 开发事实标准的根本原因。

- 启示 1：Spring 的核心价值是“解耦”
 - Spring 的所有设计都是为了解决代码耦合问题：
 - 上层代码（如UserController）仅依赖接口（UserService），不依赖具体实现；
 - 扩展功能（如切换数据库）仅需修改配置文件，无需改动原有业务代码；
 - 即使移除 Spring 框架，业务代码仍可独立运行（非侵入式设计）。
- 启示 2：IoC 是思想，DI 是实现手段
 - IoC（控制反转）：将对象的创建权从程序员转移给 Spring 容器（核心思想）；
 - DI（依赖注入）：Spring 容器通过setter/ 构造方法 / 注解，将依赖对象自动注入到目标对象中（IoC 的具体实现手段）。
- 启示 3：Spring 是非侵入式框架
 - 改造后的业务类（UserController、UserServiceImpl）：
 - 未继承 Spring 的任何类；
 - 未实现 Spring 的任何接口；
 - 仅添加了setter方法（甚至可通过构造方法注入省略setter）。

这意味着：Spring 不“绑架”你的代码 —— 即使不用 Spring，业务代码也能正常编译运行。

原生 Java 开发中，硬编码new对象会导致强耦合、扩展难、开发效率低的问题；
Spring 通过IoC 容器解决了这些问题：将对象创建权交给容器，通过配置文件管理对象与依赖；

两个核心设计原则

OCP 开闭原则（Open Closed Principle）

官方定义

一个软件实体（类、模块、方法）应该对扩展开放，对修改关闭。

核心：新增功能时，尽量通过“扩展”实现，而非“修改原有代码”。

- 还是用之前的 UserService/UserDao 例子：

```
// 传统写法：Service直接依赖Dao实现类
public class UserService {
    // 硬编码依赖UserDaoImpl（MySQL实现）
    private UserDao userDao = new UserDaoImpl();

    public void addUser() {
        userDao.insert();
    }
}
```

如果要换数据库（比如换成 Oracle，写UserDaoOracleImpl），必须修改 UserService 的源码：

```
// 被迫修改原有代码 → 违反OCP
private UserDao userDao = new UserDaoOracleImpl();
```

修改原有代码的风险：可能引入新 bug，还要重新测试、编译、部署。

符合 OCP 的正例

```
// 1. 定义Dao接口（稳定不变）
public interface UserDao {
    void insert();
}
```

```
// 2. MySQL实现（扩展1）
public class UserDaoMysqlImpl implements UserDao {
    @Override
    public void insert() {System.out.println("MySQL插入");}
}
```

```
// 3. Oracle实现（扩展2，新增功能）
public class UserDaoOracleImpl implements UserDao {
    @Override
    public void insert() {System.out.println("Oracle插入");}
}
```

```
// 4. Service依赖接口，而非实现
public class UserService {
```

```
// 依赖接口（稳定）
private UserDao userDao;

// 通过setter扩展（不用改Service源码）
public void setUserDao(UserDao userDao) {
    this.userDao = userDao;
}

public void addUser() {
    userDao.insert();
}
}
```

想换 Oracle，不用改 UserService 一行代码，只需要“扩展”新的实现类，然后通过 setter 传入即可：

```
// 扩展新功能，无需修改Service源码 → 符合OCP
UserService service = new UserService();
service.setUserDao(new UserDaoOracleImpl());
service.addUser();
```

DIP 依赖倒置原则（Dependency Inversion Principle）

解耦的核心原则

官方定义

- 高层模块（比如 Service）不应该依赖低层模块（比如 Dao 实现类），二者都应该依赖抽象（接口 / 抽象类）；
- 抽象（接口）不应该依赖细节（具体实现类），细节应该依赖抽象。

依赖抽象，而非具体实现

先看传统开发的依赖方向--正置

```
高层模块（Service） → 依赖 → 低层模块（Dao实现类）
```

比如：UserService → 直接new UserDaoImpl(), Service 依赖具体的 Dao 实现，这是“正置”——高层依赖低层的细节，完全被低层绑定。

再看DIP 的依赖方向--倒置

```
高层模块（Service） → 依赖 → 抽象（UserDao接口）
低层模块（Dao实现类） → 依赖 → 抽象（UserDao接口）
```

DIP 反例

```
// 低层模块：Dao实现类（细节）
public class UserDaoMysqlImpl {
    public void insert() {
        System.out.println("MySQL 插入用户");
    }
}

// 高层模块：Service（直接依赖Dao实现类）
public class UserService {
    // 核心问题：依赖具体实现类（细节），而非抽象
```

```
private UserDaoMysqlImpl userDao = new UserDaoMysqlImpl();

public void addUser() {
    userDao.insert();
}
}
```

- 违反 DIP 的 3 大痛点

1. 耦合死: UserService 和 UserDaoMysqlImpl 绑死, 换 Oracle 必须改 Service 源码;
2. 违反 OCP: 新增功能 (比如 Oracle Dao) 必须修改原有代码, 无法“对扩展开放”;
3. 测试难: 单元测试时, 无法替换 Dao 为 Mock 对象 (比如模拟数据库操作), 只能连真实数据库。

DIP 正例

● ● ●

```
// 抽象: UserDao接口 (高低层都依赖它)
public interface UserDao {
    void insert();
}
```

● ● ●

```
// 低层模块1: MySQL Dao实现 (细节依赖抽象)
public class UserDaoMysqlImpl implements UserDao {
    @Override
    public void insert() {
        System.out.println("MySQL 插入用户");
    }
}
```

● ● ●

```
// 低层模块2: Oracle Dao实现 (新增扩展, 同样依赖抽象)
public class UserDaoOracleImpl implements UserDao {
    @Override
    public void insert() {
        System.out.println("Oracle 插入用户");
    }
}
```

● ● ●

```
// 高层模块 (UserService) — 仅依赖抽象 (UserDao接口)
public class UserService {
    // 核心: 依赖抽象 (接口), 而非任何具体实现类
    private UserDao userDao;

    // 提供setter方法: 让外部 (比如Spring容器) 注入具体的Dao
}
```

分层架构

- 分层就像公司分工
 - 前台 (Controller) 只负责接客户请求, 不处理业务;
 - 销售部 (Service) 处理核心业务, 不直接碰数据库;
 - 库管 (DAO) 只负责存取货 (数据库 CRUD), 不管业务。
 - 换个库管 (MySQL→Oracle), 销售部不用改任何东西, 这就是分层的意义。

Impl 不是独立的“层”, 而是接口的具体实现类 (比如UserDaoImpl是UserDao接口的实现, UserServiceImpl是UserService接口的实

现);

分层架构不是 Spring 规定的，是软件工程“解耦、落地 OCP/DIP 原则”的最佳实践；

所有层的代码设计，最终都是为了：高层依赖抽象（接口），不依赖细节（Impl）。

DAO 层：数据访问层（持久层）

- 核心职责

“专门做数据库 CRUD（增删改查）的层，纯数据操作，不掺任何业务逻辑。”

- 为什么需要 DAO 层

如果没有 DAO 层，业务代码（Service）里混着 JDBC/MyBatis 代码，改数据库（MySQL→Oracle）要改所有业务代码，违反 OCP 原则（对修改开放）。

- 代码怎么写（必分“接口 + Impl”，落地 DIP）

```
// 让高层（Service）只依赖这个接口，而非具体实现
public interface UserDao {
    // 只定义“做什么”（保存用户），不管“怎么做”（用MySQL还是Oracle）
    void save(User user);
    // 根据ID查用户
    User getById(Long id);
}
```

```
// 与DAO实现类（Impl，细节，可变）—— 封装具体数据库操作
// 核心：扩展新数据库只需加新Impl，不用改接口/Service
public class UserDaoMysqlImpl implements UserDao {
    @Override
    public void save(User user) {
        // 这里只写JDBC/MyBatis代码，纯数据库操作，无任何业务
        System.out.println("MySQL保存用户: " + user.getName());
    }

    @Override
    public User getById(Long id) {
        // 模拟数据库查询，返回用户数据
        return new User(id, "张三", 20);
    }
}
```

```
// 扩展：新增Oracle实现（不用改任何原有代码，符合OCP）
public class UserDaoOracleImpl implements UserDao {
    @Override
    public void save(User user) {
        System.out.println("Oracle保存用户: " + user.getName());
    }

    @Override
    public User getById(Long id) {
        return new User(id, "李四", 21);
    }
}
```

- 为什么这么写？

- 接口 (UserDao)：定义稳定的 CRUD 规则，高层 (Service) 只依赖接口，落地DIP 原则；
- Impl (UserDaoMysqlImpl)：封装具体数据库操作，换数据库只需换 Impl，不用改 Service，落地OCP 原则；
- DAO 层只做数据操作，避免“业务逻辑混在数据库代码里”，符合“单一职责原则”。

Service 层：业务逻辑层（核心层）

- 核心职责

“程序的大脑”，处理所有核心业务逻辑（比如‘注册用户’要先查用户是否存在→再加密密码→再保存），不直接碰数据库，只调用 DAO 层。”

- 痛点（为什么需要 Service 层）

如果没有 Service 层，业务逻辑混在 Controller/DAO 里，代码混乱且无法复用（比如“注册用户”逻辑在多个地方用，要写多遍）。

- 代码怎么写（必分“接口 + Impl”，依赖 DAO 接口）



```
// 先定义User实体（数据载体，各层传递数据用）
// 必须写无参构造、getter/setter、toString
public class User {
    private Long id;
    private String name;
    private Integer age;

    public User() {} // 无参构造（Spring创建对象需要）
    public User(Long id, String name, Integer age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }
    // getter/setter省略
    @Override
    public String toString() {
        return "User{" + "id=" + id + ", name='" + name + '\'' + ", age=" + age + '}';
    }
}
```



```
// 定义Service接口（抽象，稳定）—— 定义业务规则
public interface UserService {
    // 注册用户（核心业务逻辑）
    void registerUser(User user);
    // 根据ID查用户（复用DAO方法）
    User getUserById(Long id);
}
```



```
// 写Service实现类（Impl，细节）—— 依赖DAO接口，封装业务逻辑
public class UserServiceImpl implements UserService {
    // 核心：依赖DAO接口（抽象），而非具体Impl（DIP原则）
    private UserDao userDao;

    // 提供setter：让外部（比如Spring）注入具体的DAO Impl
    // 关键：换DAO Impl（MySQL→Oracle），不用改Service任何代码
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    @Override
    public void registerUser(User user) {
        // 业务逻辑1：先查用户是否存在（调用DAO）
    }
}
```

```

    User existUser = userDao.getById(user.getId());
    if (existUser == null) {
        // 业务逻辑2: 保存用户 (调用DAO)
        userDao.save(user);
        System.out.println("用户注册成功: " + user.getName());
    } else {
        System.out.println("用户已存在: " + user.getName());
    }
}

@Override
public User getUserById(Long id) {
    // 直接复用DAO方法, 无额外业务逻辑
    return userDao.getById(id);
}
}

```

- 为什么这么写？
 - 接口 (UserService): 定义业务规则, 高层 (Controller) 只依赖接口, 落地 DIP;
 - Impl (UserServiceImpl): 依赖 DAO 接口而非 Impl, 换 DAO Impl (MySQL→Oracle) 只需改注入的 Impl, 不用改业务逻辑, 落地 OCP;
 - 业务逻辑集中在 Service, 复用性高 (多个 Controller 可以调用同一个 Service 方法)。

Controller 层：控制层（表现层）

- 核心职责

“前台接待员, 只做 3 件事: 接收前端请求参数→调用 Service 处理业务→返回结果给前端, 不碰任何业务逻辑 / 数据库操作。”

- 痛点 (为什么需要 Controller 层)

如果没有 Controller 层, Service 直接对接前端, 前端参数格式变了 (比如手机号从 String→Long) 要改 Service, 违反 OCP 原则。

- 代码怎么写 (依赖 Service 接口, 不碰业务)

```

public class UserController {
    // 核心: 依赖Service接口 (抽象), 而非具体Impl (DIP原则)
    private UserService userService;

    // 提供setter: 注入具体的Service Impl
    public void setUserService(UserService userService) {
        this.userService = userService;
    }

    // 处理“用户注册”请求 (模拟前端传参)
    public void handleRegisterRequest(Long id, String name, Integer age) {
        // 1. 接收前端参数, 封装成User对象 (Controller的职责)
        User user = new User(id, name, age);
        // 2. 调用Service处理业务 (只调接口方法, 不管内部逻辑)
        userService.registerUser(user);
        // 3. 返回结果给前端 (模拟)
        System.out.println("注册请求处理完成, 结果已返回");
    }

    // 处理“根据ID查用户”请求
    public void handleGetUserRequest(Long id) {
        User user = userService.getUserById(id);
        System.out.println("查询到用户, 返回给前端: " + user);
    }
}

```

- 为什么这么写？
 - Controller 只做“参数接收 / 结果返回”，不碰业务逻辑，换前端（网页→APP）只需改 Controller，不用改 Service，落地 OCP；
 - 依赖 Service 接口，而非 Impl，即使换 Service 实现（比如加“注册用户送积分”逻辑），Controller 不用改，落地 DIP。

各层之间的关系

- 依赖方向
 - 高层依赖低层的抽象，绝对不反向

● ● ●
Controller层 → 依赖 → Service层（接口）
Service层 → 依赖 → DAO层（接口）
DAO层（Impl） → 依赖 → 数据库

绝对不能让 DAO 依赖 Service，或 Service 依赖 Controller，否则分层就失去意义，耦合度会爆炸。

- 完整交互流程（以“用户注册”为例）
 - 前端发送“注册用户”请求 → Controller 接收参数（id=1，name = 张三，age=20）；
 - Controller 调用 Service 的registerUser方法（只调接口，不管内部逻辑）；
 - Service 执行业务逻辑：
 - 调用 DAO 的getById方法，查用户是否存在；
 - 如果不存在，调用 DAO 的save方法保存用户；
 - DAO 执行数据库操作（MySQL/Oracle），返回结果给 Service；
 - Service 把业务处理结果返回给 Controller；
 - Controller 把结果返回给前端。

Spring 概述

Spring 介绍



老杜

- Spring是一个开源框架，它由Rod Johnson创建。它是为了解决企业应用开发的复杂性而创建的。
- 从简单性、可测试性和松耦合的角度而言，任何Java应用都可以从Spring中受益。
- Spring是一个轻量级的控制反转(IOC)和面向切面(AOP)的容器框架。
- Spring最初的出现是为了解决EJB臃肿的设计，以及难以测试等问题。

- Spring为简化开发而生，让程序员只需关注核心业务的实现，尽可能的不再关注非业务逻辑代码（事务控制，安全日志等）。

Spring 生态

Spring 不是一个单独的框架，而是一个生态体系，Spring 是核心，周边还有：

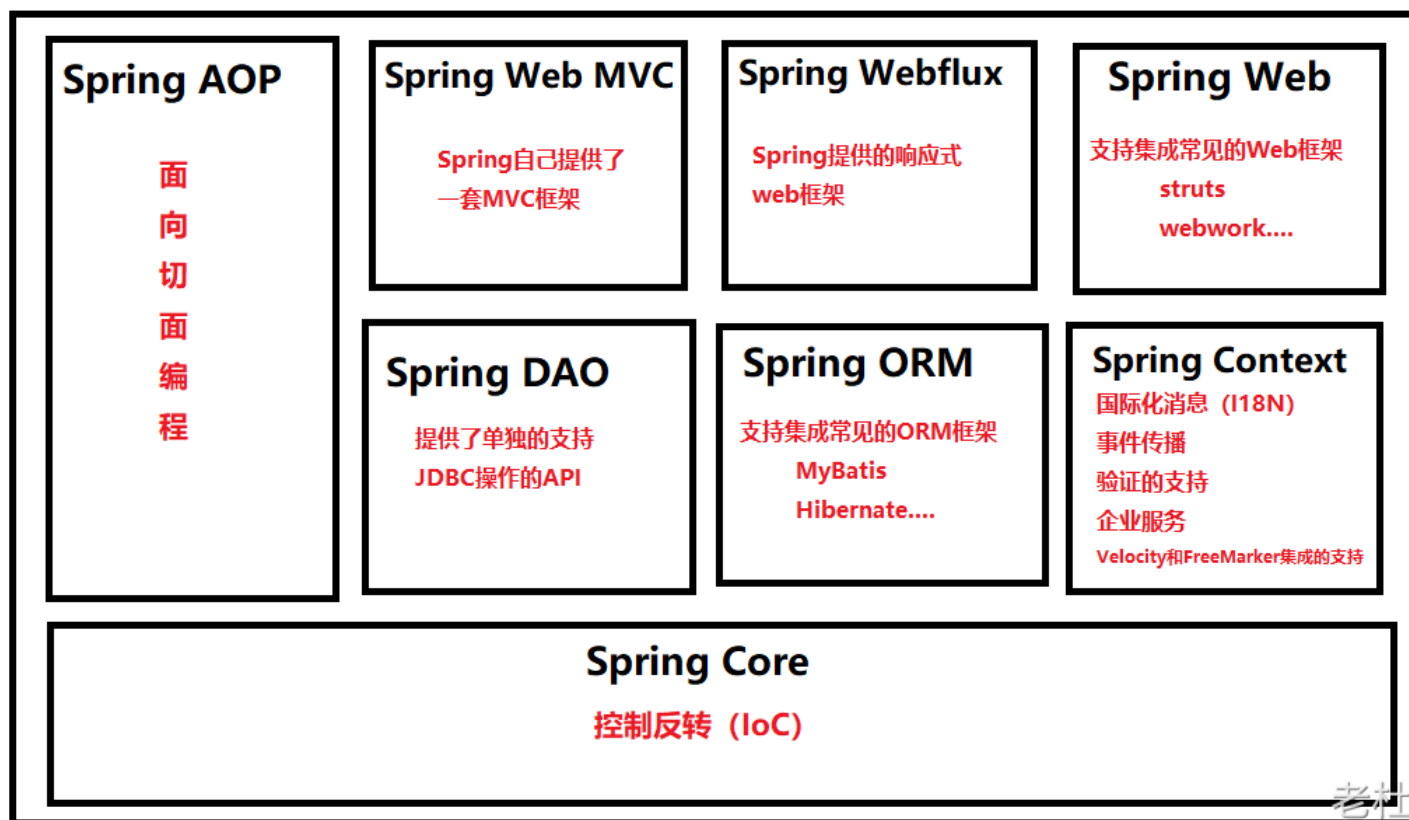
- Spring Boot：简化 Spring 配置，快速开发微服务；
- Spring Cloud：基于 Spring Boot 实现微服务治理；
- Spring MVC：处理 Web 请求；
- Spring Data：简化数据库操作；
- Spring Security：处理权限、安全。

Spring 优势

1. 轻量级：核心包体积小，无强制依赖，引入后不会增加项目负担
2. 非侵入式：业务类无需继承或者实现 Spring 的类，只需要加 setter 方法或注解，代码干净
3. 解耦：切换数据库只需改配置，不用改源码
4. 简化开发：容器自动管理对象，程序员专注业务逻辑，不用写重复的对象创建、依赖代码
5. 可扩展：除了核心 IoC/DI，还能整合 MyBatis、Redis、MQ 等框架，形成完整的生态

Spring8大模块

注意：Spring5版本之后是8个模块。在Spring5中新增了WebFlux模块。



- 底层核心层：Spring Core、Spring Context
 - 核心扩展层：Spring AOP、Spring Web、Spring Web MVC、Spring Webflux
 - 数据访问层：Spring DAO、Spring ORM
1. Spring Core模块

这是Spring框架最基础的部分，它提供了依赖注入（DependencyInjection）特征来实现容器对Bean的管理。核心容器的主要组件是BeanFactory，

BeanFactory是工厂模式的一个实现，是任何Spring应用的核心。它使用IoC将应用配置和依赖从实际的应用代码中分离出来。

2. Spring Context模块

如果说核心模块中的BeanFactory使Spring成为容器的话，那么上下文模块就是Spring成为框架的原因。

这个模块扩展了BeanFactory，增加了对国际化（I18N）消息、事件传播、验证的支持。另外提供了许多企业服务，例如电子邮件、JNDI访问、EJB集成、远程以及时序调度（scheduling）服务。也包括了对模版框架例如Velocity和FreeMarker集成的支持

3. Spring AOP模块

Spring在它的AOP模块中提供了对面向切面编程的丰富支持，Spring AOP 模块为基于 Spring 的应用程序中的对象提供了事务管理服务。通过使用 Spring AOP，不用依赖组件，就可以将声明性事务管理集成到应用程序中，可以自定义拦截器、切点、日志等操作。

4. Spring DAO模块

提供了一个JDBC的抽象层和异常层次结构，消除了烦琐的JDBC编码和数据库厂商特有的错误代码解析，用于简化JDBC。

5. Spring ORM模块

Spring提供了ORM模块。Spring并不试图实现它自己的ORM解决方案，而是为几种流行的ORM框架提供了集成方案，包括Hibernate、JDO和iBATIS SQL映射，这些都遵从 Spring 的通用事务和 DAO 异常层次结构。

6. Spring Web MVC模块

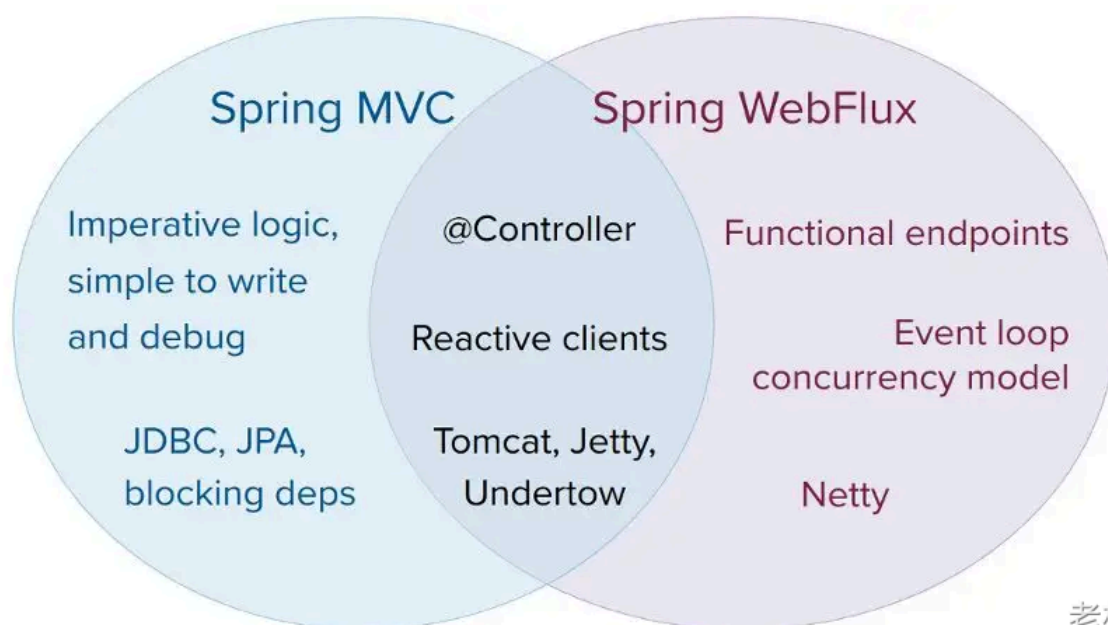
Spring为构建Web应用提供了一个功能全面的MVC框架。虽然Spring可以很容易地与其它MVC框架集成，例如Struts，但Spring的MVC框架使用IoC对控制逻辑和业务对象提供了完全的分离。

7. Spring Web

Web 上下文模块建立在应用程序上下文模块之上，为基于 Web 的应用程序提供了上下文，提供了Spring和其它Web框架的集成，比如Struts、WebWork。还提供了一些面向服务支持，例如：实现文件上传的multipart请求。

8. Spring WebFlux模块

Spring Framework 中包含的原始 Web 框架 Spring Web MVC 是专门为 Servlet API 和 Servlet 容器构建的。反应式堆栈 Web 框架 Spring WebFlux 是在 5.0 版的后期添加的。它是完全非阻塞的，支持反应式流(Reactive Stream)背压，并在Netty, Undertow和Servlet 3.1+容器等服务器上运行。



Spring特点

1. 轻量

a. 从大小与开销两方面而言Spring都是轻量的。完整的Spring框架可以在一个大小只有1MB多的JAR文件里发布。并且Spring所需的处理开销也是微不足道的。

b. Spring是非侵入式的：Spring应用中的对象不依赖于Spring的特定类。

2. 控制反转

a. Spring通过一种称作控制反转（IoC）的技术促进了松耦合。当应用了IoC，一个对象依赖的其它对象会通过被动的方式传递进来，而不是这个对象自己创建或者查找依赖对象。你可以认为IoC与JNDI相反——不是对象从容器中查找依赖，而是容器在对象初始化时不等对象请求就主动将依赖传递给它。

3. 面向切面

a. Spring提供了面向切面编程的丰富支持，允许通过分离应用的业务逻辑与系统级服务（例如审计（auditing）和事务（transaction）管理）进行内聚性的开发。应用对象只实现它们应该做的——完成业务逻辑——仅此而已。它们并不负责（甚至是意识）其它的系统级关注点，例如日志或事务支持。

4. 容器

a. Spring包含并管理应用对象的配置和生命周期，在这个意义上它是一种容器，你可以配置你的每个bean如何被创建——基于一个可配置原型（prototype），你的bean可以创建一个单独的实例或者每次需要时都生成一个新的实例——以及它们是如何相互关联的。然而，Spring不应该被混同于传统的重量级的EJB容器，它们经常是庞大与笨重的，难以使用。

5. 框架

a. Spring可以将简单的组件配置、组合成为复杂的应用。在Spring中，应用对象被声明式地组合，典型地是在一个XML文件里。Spring也提供了很多基础功能（事务管理、持久化框架集成等等），将应用逻辑的开发留给了你。

所有Spring的这些特征使你能够编写更干净、更可管理、并且更易于测试的代码。它们也为Spring中的各种模块提供了基础支持。

本教程软件版本

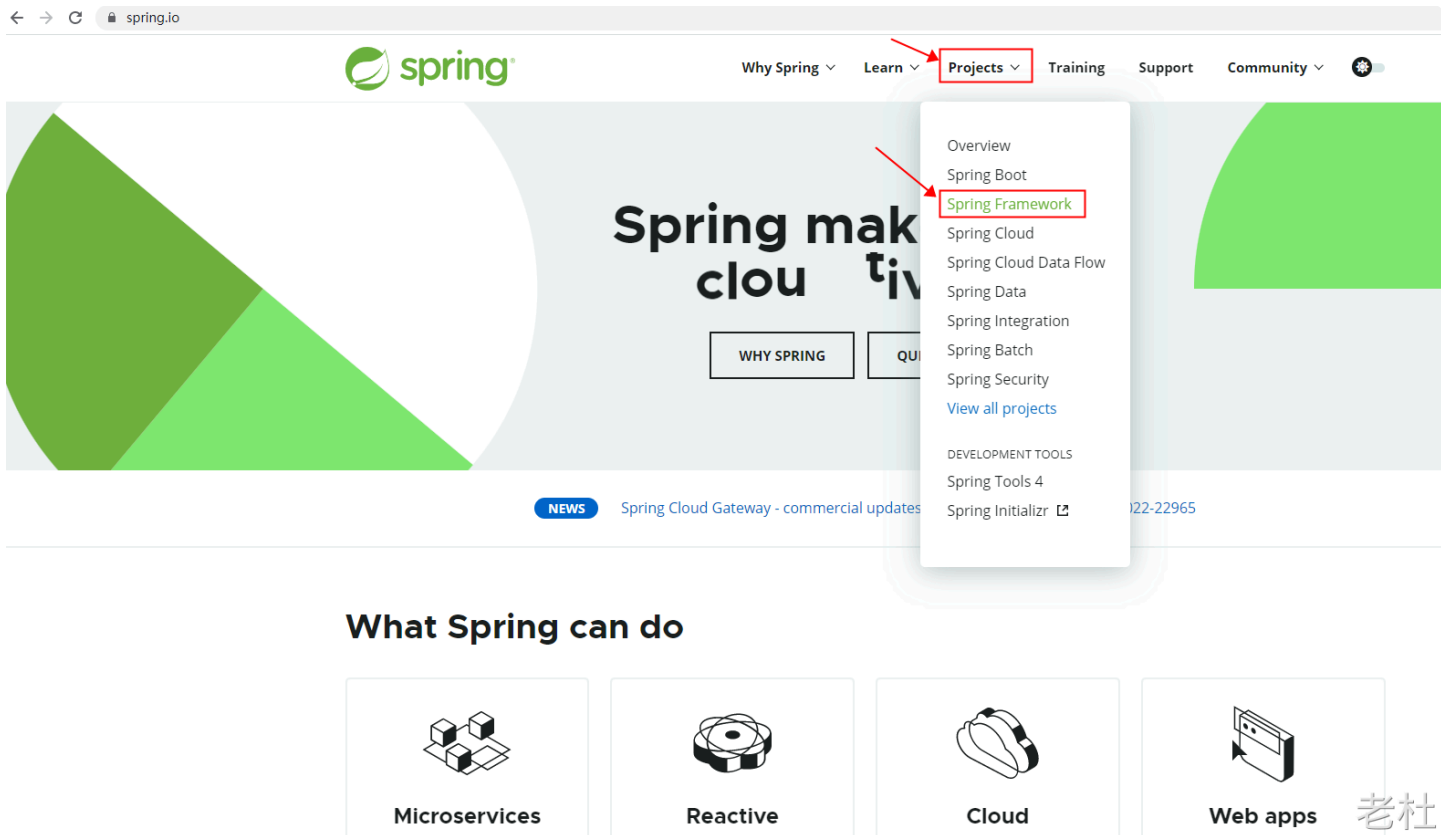
- IDEA工具：IntelliJ IDEA 2023.2 (Ultimate Edition)
- JDK：Java21 **（Spring6要求JDK最低版本是Java17）**
- Maven：3.9.12
- Spring：6.0.10
- JUnit：4.13.2

Spring 的入门程序

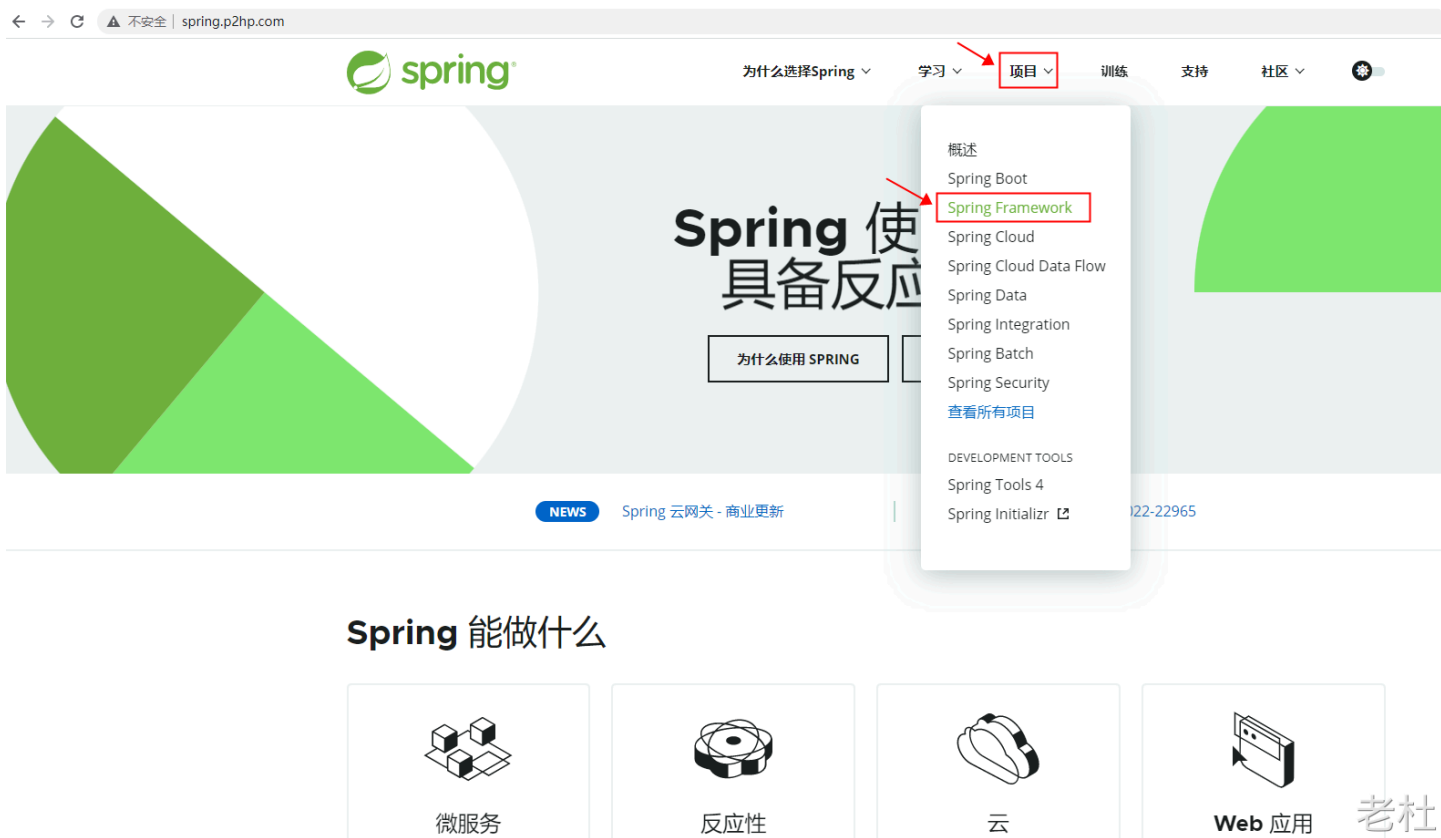
Spring 下载

以下是没有安装Maven 环境时，手动下载Spring 安装步骤

官网地址：<https://spring.io/>



官网地址（中文）：<http://spring.p2hp.com/>



打开Spring官网后，可以看到Spring Framework，以及通过Spring Framework衍生的其它框架：

All projects ▸

Spring Boot

Spring Framework

▸ Spring Data

▸ Spring Cloud

Spring Cloud Data Flow

Spring gRPC

▸ Spring Security

Spring Authorization Server

Spring for GraphQL

▸ Spring Session

Spring Integration

Spring HATEOAS

Spring Modulith

Spring REST Docs

Spring AI



Spring Framework

7.0.6



OVERVIEW

LEARN

SUPPORT

The Spring Framework provides a comprehensive programming and configuration model for modern Java-based enterprise applications - on any kind of deployment platform.

A key element of Spring is infrastructural support at the application level: Spring focuses on the "plumbing" of enterprise applications so that teams can focus on application-level business logic, without unnecessary ties to specific deployment environments.

Support Policy and Migration

For information about minimum requirements, guidance on upgrading from earlier versions and support policies, please check out [the official Spring Framework wiki page](#)

Features

- **Core technologies:** dependency injection, events, resources, i18n, validation, data binding, type conversion, SpEL, AOP.

这就是Spring 框架，其他的框架都是基于Spring 去扩展的

我们即将要学习的就是Spring Framework。

怎么下载呢？

- 第一步：进入github

All projects ▸

Spring Boot

Spring Framework

▸ Spring Data

▸ Spring Cloud

Spring Cloud Data Flow

Spring gRPC

▸ Spring Security

Spring Authorization Server

Spring for GraphQL

▸ Spring Session

Spring Integration

Spring HATEOAS

Spring Modulith

Spring REST Docs

Spring AI



Spring Framework

7.0.6



OVERVIEW

LEARN

SUPPORT

The Spring Framework provides a comprehensive programming and configuration model for modern Java-based enterprise applications - on any kind of deployment platform.

A key element of Spring is infrastructural support at the application level: Spring focuses on the "plumbing" of enterprise applications so that teams can focus on application-level business logic, without unnecessary ties to specific deployment environments.

Support Policy and Migration

For information about minimum requirements, guidance on upgrading from earlier versions and support policies, please check out [the official Spring Framework wiki page](#)

Features

- **Core technologies:** dependency injection, events, resources, i18n, validation, data binding, type conversion, SpEL, AOP.

- 第二步：找到下图位置，点击超链接

This is the home of the Spring Framework: the foundation for all [Spring projects](#). Collectively the Spring Framework and the family of Spring projects are often referred to simply as "Spring".

Spring provides everything required beyond the Java programming language for creating enterprise applications for a wide range of scenarios and architectures. Please read the [Overview](#) section of the reference documentation for a more complete introduction.

Code of Conduct

This project is governed by the [Spring Code of Conduct](#). By participating, you are expected to uphold this code of conduct. Please report unacceptable behavior to spring-code-of-conduct@spring.io.

Access to Binaries

For access to artifacts or a distribution zip, see the [Spring Framework Artifacts](#) wiki page.

Documentation

The Spring Framework maintains reference documentation ([published](#) and [source](#)), GitHub [wiki pages](#), and an [API reference](#). There are also [guides and tutorials](#) across Spring projects.

Micro-Benchmarks

See the [Micro-Benchmarks](#) wiki page.

- 第三步：找到下图位置，点击超链接

can be applied for any of the other Spring framework artifacts listed above.

Maven

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.2.11</version>
</dependency>
```



Gradle

```
implementation 'org.springframework:spring-context:6.2.11'
```



Spring Repositories

Snapshot, milestone, and release candidate versions are published to an [Artifactory](#) instance hosted by [JFrog](#). You can use the Web UI at <https://repo.spring.io> to browse the Spring Artifactory, or go directly to one of the repositories listed below.

🔗 Snapshot Repository Configuration

Add the following to enable your build tool to resolve snapshot versions.

Maven

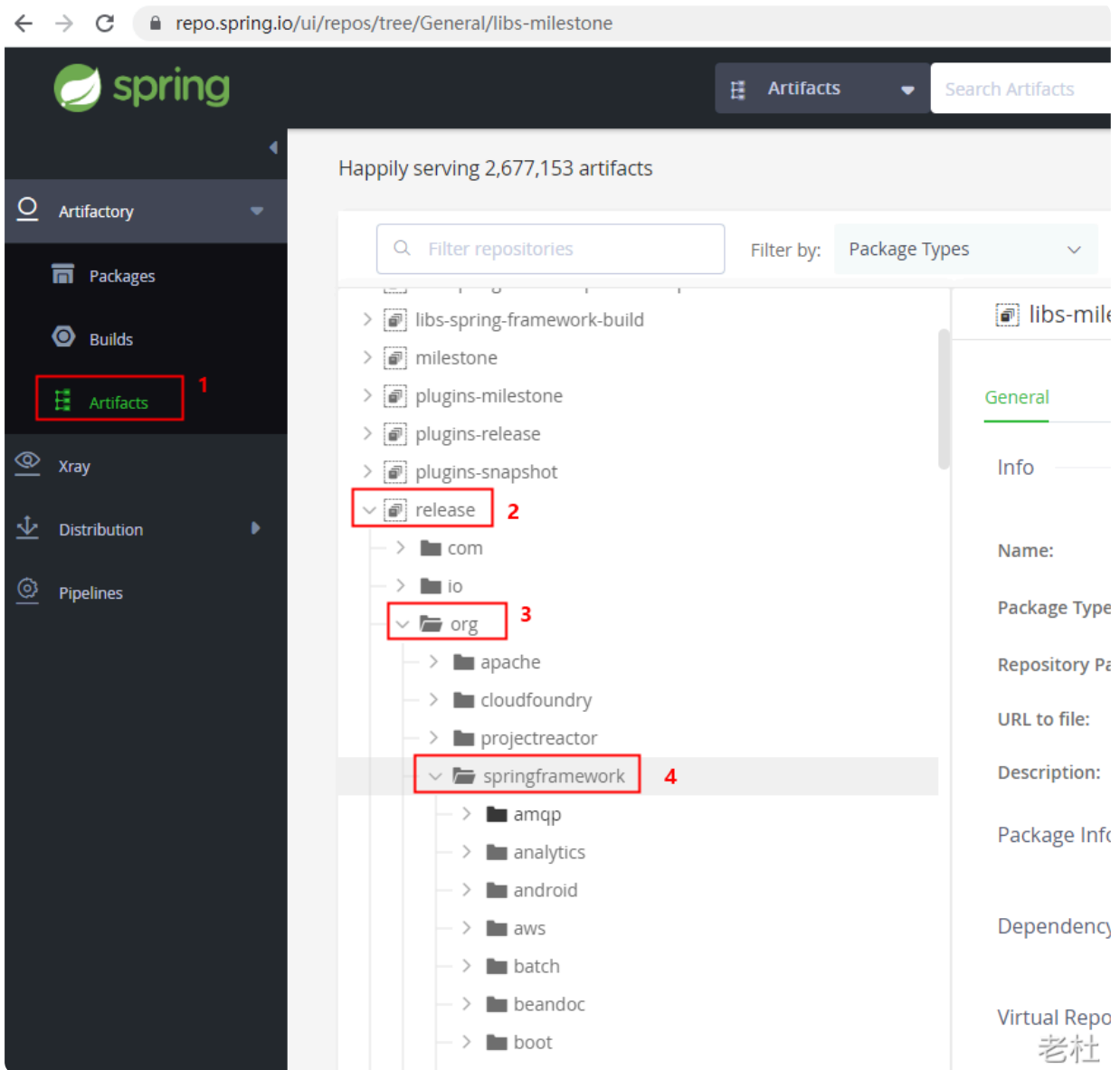
```
<repositories>
  <repository>
    <id>spring-snapshots</id>
    <name>Spring Snapshots</name>
```



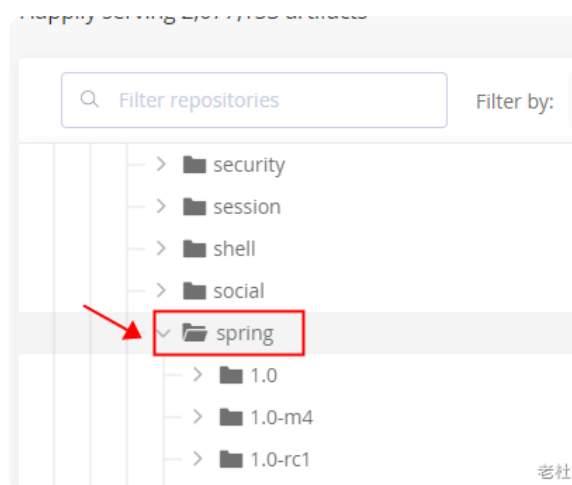
- 第四步：按照下图步骤操作

Clone this wiki

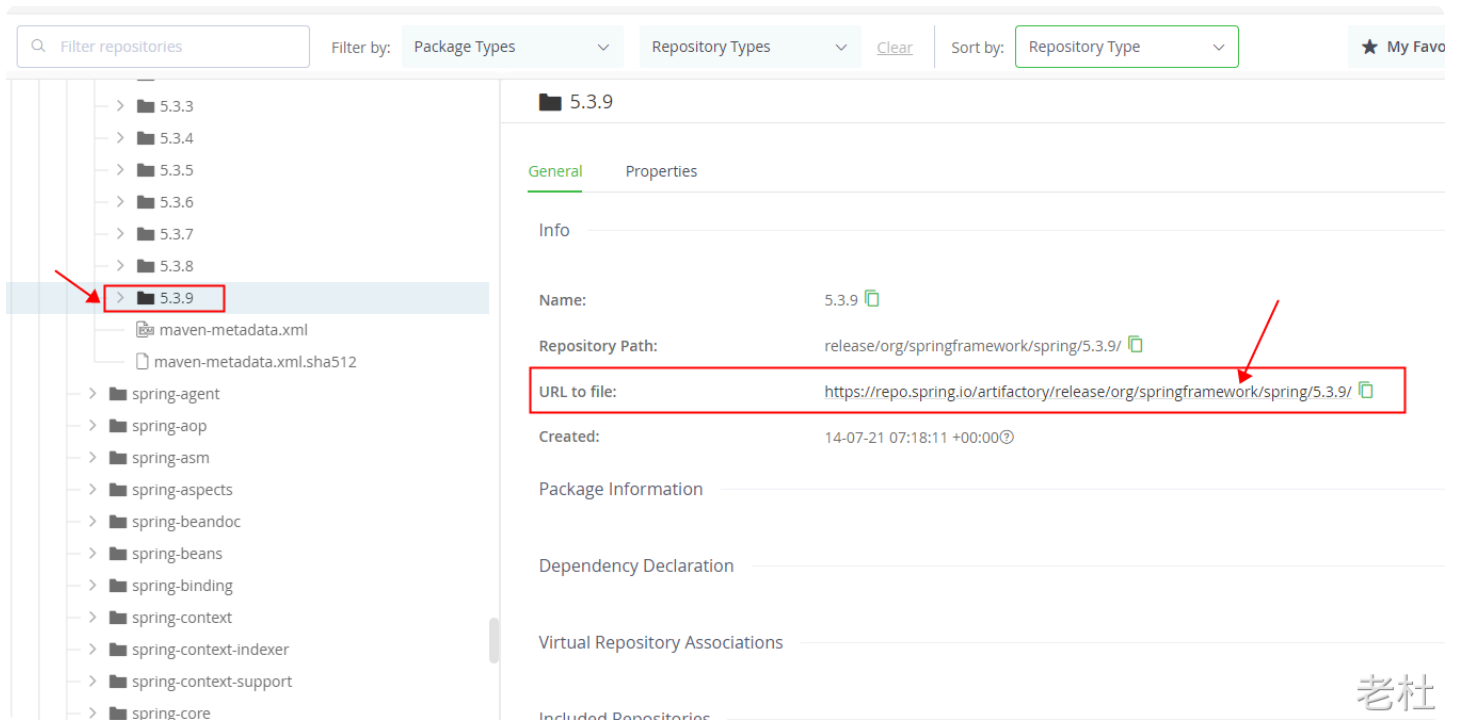
<https://github.com>



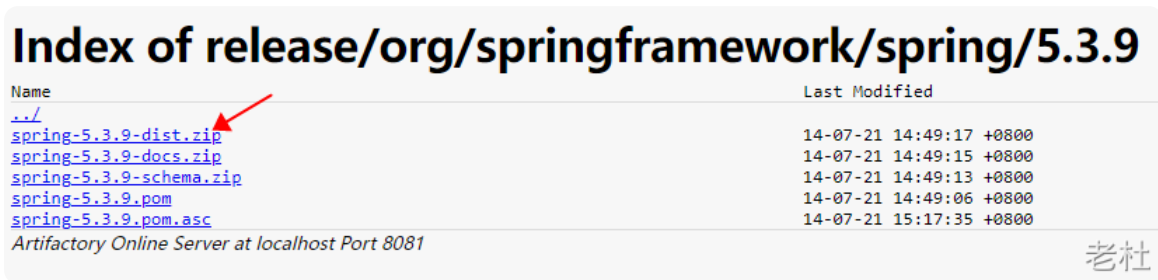
- 第五步：继续在springframework目录下找下图的spring，点开之后你会看到很多不同的版本



- 第六步：选择对应的版本

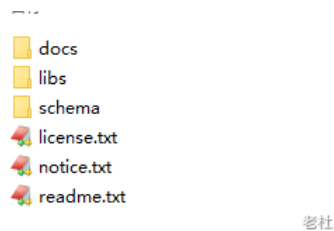


- 第七步：点击上图的url



点击spring-5.3.9-dist.zip下载spring框架。

将下载的zip包解压：



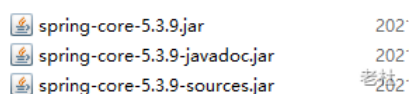
docs：spring框架的API帮助文档

libs：spring框架的jar文件（**用spring框架就是用这些jar包**）

schema：spring框架的XML配置文件相关的约束文件

Spring的jar文件

打开libs目录，会看到很多jar包：



spring-core-5.3.9.jar：字节码（**这个是支撑程序运行的jar包**）

spring-core-5.3.9-javadoc.jar：代码中的注释

spring-core-5.3.9-sources.jar：源码

我们来看一下spring框架都有哪些jar包：

-  spring-aop-5.3.9.jar
 -  spring-aspects-5.3.9.jar
 -  spring-beans-5.3.9.jar
 -  spring-context-5.3.9.jar
 -  spring-context-indexer-5.3.9.jar
 -  spring-context-support-5.3.9.jar
 -  spring-core-5.3.9.jar
 -  spring-expression-5.3.9.jar
 -  spring-instrument-5.3.9.jar
 -  spring-jcl-5.3.9.jar
 -  spring-jdbc-5.3.9.jar
 -  spring-jms-5.3.9.jar
 -  spring-messaging-5.3.9.jar
 -  spring-orm-5.3.9.jar
 -  spring-oxm-5.3.9.jar
 -  spring-r2dbc-5.3.9.jar
 -  spring-test-5.3.9.jar
 -  spring-tx-5.3.9.jar
 -  spring-web-5.3.9.jar
 -  spring-webflux-5.3.9.jar
 -  spring-webmvc-5.3.9.jar
 -  spring-websocket-5.3.9.jar
- 老杜

JAR文件	描述
spring-aop-5.3.9.jar	这个jar 文件包含在应用中使用Spring 的AOP 特性时所需的类
spring-aspects-5.3.9.jar	提供对AspectJ的支持，以便可以方便的将面向切面的功能集成进IDE中
spring-beans-5.3.9.jar	这个jar 文件是所有应用都要用到的，它包含访问配置文件、创建和管理bean 以及进行Inversion ofControl / Dependency Injection（IoC/DI）操作相关的所有类。如果应用只需基本的IoC/DI 支持，引入spring-core.jar 及 spring-beans.jar 文件就可以了。
spring-context-5.3.9.jar	这个jar 文件为Spring 核心提供了大量扩展。可以找到使用Spring ApplicationContext特性时所需的全部类，JDNI 所需的全部类，instrumentation组件以及校验Validation 方面的相关类。
spring-context-indexer-5.3.9.jar	虽然类路径扫描非常快，但是Spring内部存在大量的类，添加此依赖，可以通过在编译时创建候选对象的静态列表来提高大型应用程序的启动性能。
spring-context-support-5.3.9.jar	用来提供Spring上下文的一些扩展模块,例如实现邮件服务、视图解析、缓存、定时任务调度等
spring-core-5.3.9.jar	Spring 框架基本的核心工具类。Spring 其它组件要都要使用到这个包里的类，是其它组件的基本核心，当然你也可以在自己的应用系统中使用这些工具类。
spring-expression-5.3.9.jar	Spring表达式语言。
spring-instrument-5.3.9.jar	Spring3.0对服务器的代理接口。
spring-jcl-5.3.9.jar	Spring的日志模块。JCL，全称为"Jakarta Commons Logging"，也可称为"Apache Commons Logging"。
spring-jdbc-5.3.9.jar	Spring对JDBC的支持。

JAR文件	描述
spring-jms-5.3.9.jar	这个jar包提供了对JMS 1.0.2/1.1的支持类。JMS是Java消息服务。属于JavaEE规范之一。
spring-messaging-5.3.9.jar	为集成messaging api和消息协议提供支持
spring-orm-5.3.9.jar	Spring集成ORM框架的支持，比如集成hibernate，mybatis等。
spring-oxm-5.3.9.jar	为主流O/X Mapping组件提供了统一层抽象和封装，OXM是Object Xml Mapping。对象和XML之间的相互转换。
spring-r2dbc-5.3.9.jar	Reactive Relational Database Connectivity (关系型数据库的响应式连接) 的缩写。这个jar文件是Spring对r2dbc的支持。
spring-test-5.3.9.jar	对JUnit等测试框架的简单封装。
spring-tx-5.3.9.jar	为JDBC、Hibernate、JDO、JPA、Beans等提供的一致性的声明式和编程式事务管理支持。
spring-web-5.3.9.jar	Spring集成MVC框架的支持，比如集成Struts等。
spring-webflux-5.3.9.jar	WebFlux是 Spring5 添加的新模块，用于 web 的开发，功能和 SpringMVC 类似的，Webflux 使用当前一种比较流程响应式编程出现的框架。
spring-webmvc-5.3.9.jar	SpringMVC框架的类库
spring-websocket-5.3.9.jar	Spring集成WebSocket框架时使用

注意：

如果你只是想用Spring的IoC功能，仅需要引入：**spring-context**即可。将这个jar包添加到classpath当中。

如果采用maven只需要引入context的依赖即可。

- spring bean依赖
 - 在pom.xml 文件中

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <!-- 模块名+包名（默认打包方式jar） -->
  <groupId>org.cqipu.edu</groupId>
  <artifactId>spring6-ioc-oa-demo01</artifactId>
  <version>1.0-SNAPSHOT</version>

  <!-- JDK21+编码+Spring版本统一管理 -->
  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <spring.version>6.0.10</spring.version> <!-- 选取合适的spring 版本 -->
  </properties>

  <!-- Spring6 核心依赖（仅需spring-context，自动拉取core/beans等） -->
```

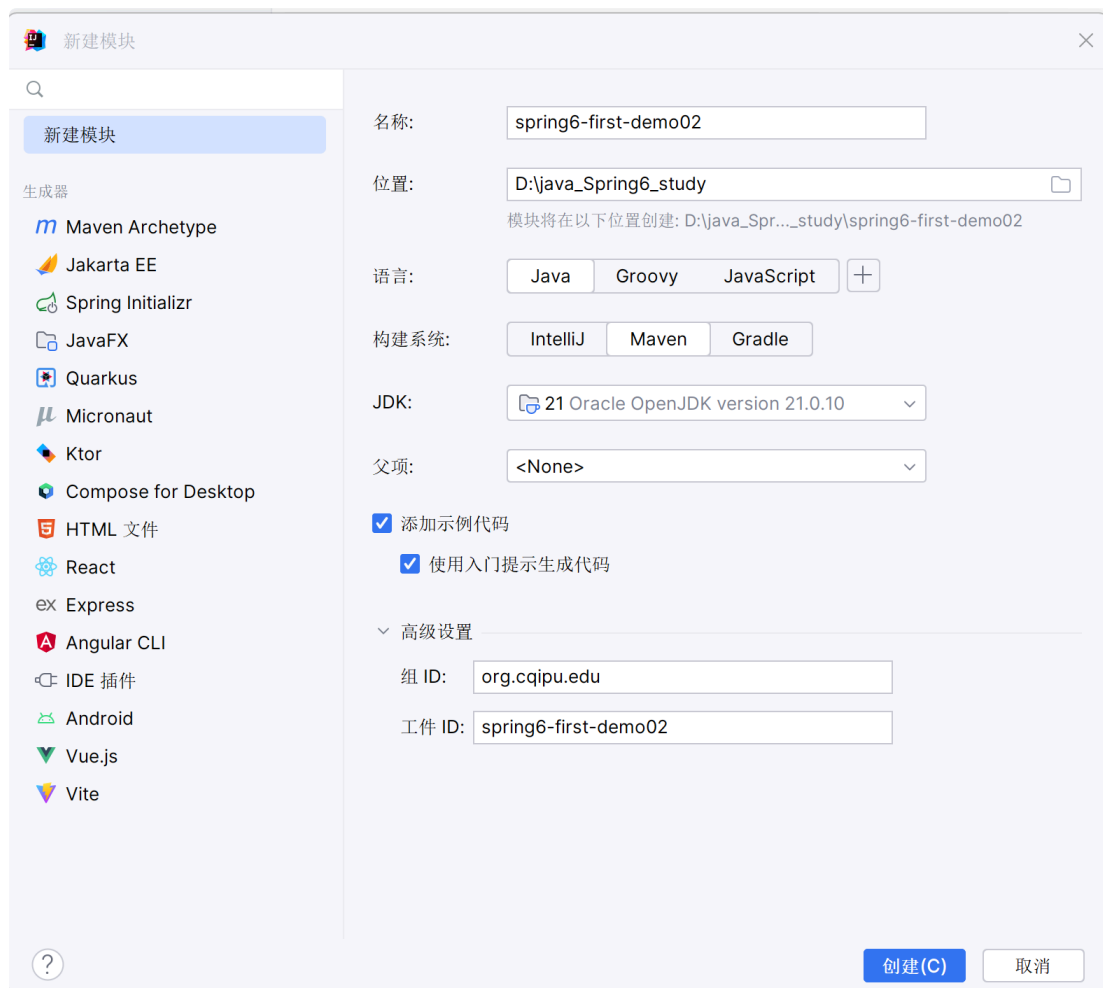
```

<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>${spring.version}</version> <!-- 用属性统一管理，易维护 -->
  </dependency>
</dependencies>

<!-- 指定编译插件版本（适配JDK21，避免版本兼容报错） -->
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version> <!-- 支持JDK21的最低版本 -->
      <configuration>
        <source>21</source>
        <target>21</target>
        <encoding>UTF-8</encoding>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

第一个Spring 程序



- 第一步：添加spring context的依赖，pom.xml配置如下

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.cqipu.edu</groupId>
    <artifactId>spring6-first-demo02</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- JDK21+编码+Spring版本统一管理 -->
    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <spring.version>6.0.10</spring.version> <!-- 选取合适的spring 版本 -->
    </properties>

    <!-- Spring6 核心依赖（仅需spring-context，自动拉取core/beans等） -->
    <dependencies>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>${spring.version}</version> <!-- 用属性统一管理，易维护 -->
        </dependency>
    </dependencies>

    <!-- 指定编译插件版本（适配JDK21，避免版本兼容报错） -->
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version> <!-- 支持JDK21的最低版本 -->
                <configuration>
                    <source>21</source>
                    <target>21</target>
                    <encoding>UTF-8</encoding>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>

```

注意：还有其他方式引入依赖

- 第二步：添加 Junit 依赖
 - pom.xml 文件

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.cqipu.edu</groupId>
    <artifactId>spring6-first-demo02</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- JDK21+编码+Spring版本统一管理 -->
    <properties>
        <maven.compiler.source>21</maven.compiler.source>

```



```

<maven.compiler.target>21</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<spring.version>6.0.10</spring.version> <!-- 选择合适的spring 版本 -->
<junit.version>4.13.2</junit.version> <!-- 选择合适的junit 版本 -->
</properties>

<dependencies>
  <!-- Spring6 核心依赖（仅需spring-context，自动拉取core/beans等） -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>${spring.version}</version> <!-- 用属性统一管理，易维护 -->
  </dependency>

  <!-- JUnit 4.13.2 测试依赖 -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope> <!-- 仅测试阶段生效，主代码不依赖 -->
  </dependency>
</dependencies>

<!-- 指定编译插件版本（适配JDK21，避免版本兼容报错） -->
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version> <!-- 支持JDK21的最低版本 -->
      <configuration>
        <source>21</source>
        <target>21</target>
        <encoding>UTF-8</encoding>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

- 第三步：定义bean：User
 - User.java

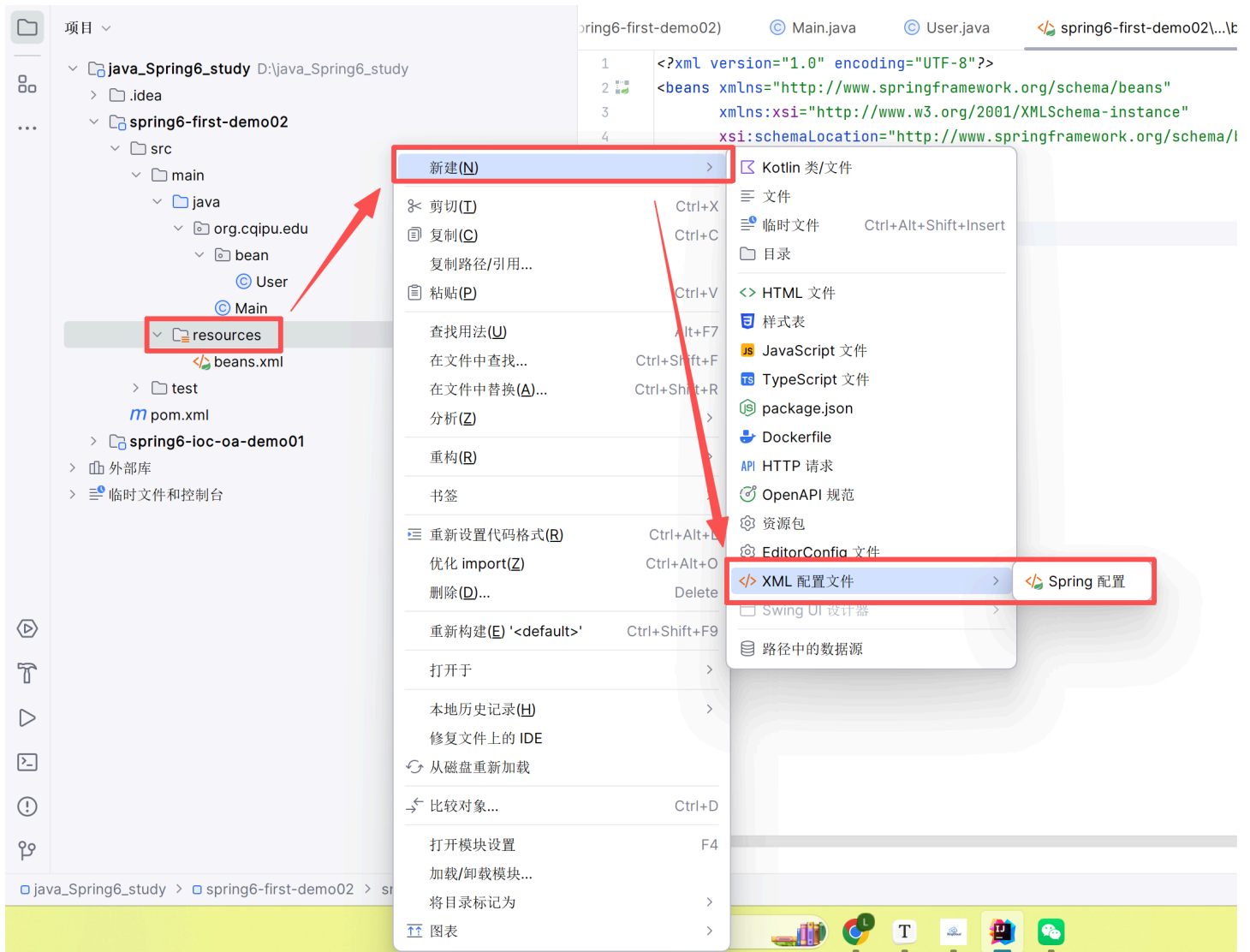
```

package org.cqipu.edu.bean;

/**
 * bean，封装用户信息。
 */
public class User {
}

```

- 第四步：编写spring的配置文件：beans.xml。该文件放在类的根路径下。



上图是使用IDEA工具自带的spring配置文件的模板进行创建。

配置文件中进行bean的配置。

- beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userBean" class="org.cqipu.edu.bean.User"/>
</beans>
```

bean的id和class属性：

- **id属性**：代表对象的唯一标识。可以看做一个人的身份证号。
- **class属性**：用来指定要创建的java对象的类名，这个类名必须是全限定类名（带包名）。

第五步：编写测试程序

- Spring6Test.java

```
package org.cqipu.edu;
```

```
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Spring6Test {

    @Test
    public void testFirst01(){
        // 初始化Spring容器上下文（解析beans.xml文件，创建所有的bean对象）
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("beans.xml");
        // 根据id获取bean对象
        Object userBean = applicationContext.getBean("userBean");
        System.out.println(userBean);
    }
}
```

第七步：运行测试程序

✓ 测试 已通过: 1共 1 个测试 - 182毫秒

D:\Java21\bin\java.exe ...
org.cqipu.edu.bean.User@42a48628

第一个Spring程序详细剖析

- beans.xml



```
<bean id="userBean" class="org.cqipu.edu.bean.User"/>
```

- Spring6Test.testFirst()



```
ApplicationContext applicationContext = new ClassPathXmlApplicationContext("beans.xml");
Object userBean = applicationContext.getBean("userBean");
```

1. bean标签的id属性可以重复吗？

- 在bean 创建 Vip.java 实体类



```
package org.cqipu.edu.bean;

public class Vip {
}
```

- beans.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userBean" class="org.cqipu.edu.bean.User"/>
    <bean id="userBean" class="org.cqipu.edu.bean.Vip"/>
</beans>
```

运行测试程序

```
D:\Java21\bin\java.exe ...
```

```
org.springframework.beans.factory.parsing.BeanDefinitionParsingException: Configuration problem: Bean name 'userBean' is already  
used in this <beans> element  
Offending resource: class path resource [beans.xml]
```

```
at org.springframework.beans.factory.parsing.FailFastProblemReporter.error(FailFastProblemReporter.java:72)  
at org.springframework.beans.factory.parsing.ReaderContext.error(ReaderContext.java:119)  
at org.springframework.beans.factory.parsing.ReaderContext.error(ReaderContext.java:111)
```

通过测试得出：在spring的配置文件中id是不能重名。

- 因此修改 beans.xml 文件

```
<bean id="vipBean" class="org.cqipu.edu.bean.Vip"/>
```

2. 底层是怎么创建对象的，是通过反射机制调用无参数构造方法吗？

- User.java

```
package org.cqipu.edu.bean;  
  
/**  
 * bean，封装用户信息。  
 */  
public class User {  
    public User() {  
        System.out.println("User的无参数构造方法执行");  
    }  
}
```

在User类中添加无参数构造方法，如上。

运行测试程序：

✓ 测试 已通过: 1共 1 个测试 – 164毫秒

```
D:\Java21\bin\java.exe ...
```

User的无参数构造方法执行

org.cqipu.edu.bean.User@293a5bf6

通过测试得知：创建对象时确实调用了无参数构造方法。

如果提供一个有参数构造方法，不提供无参数构造方法会怎样呢？

User.java

```

package org.cqipu.edu.bean;

/**
 * bean，封装用户信息。
 */
public class User {
//    public User() {
//        System.out.println("User的无参数构造方法执行");
//    }
    public User(String name){
        System.out.println("User的有参数构造方法执行");
    }
}

```

运行测试程序：

```

D:\Java21\bin\java.exe ...
3月 16, 2026 9:30:01 下午 org.springframework.context.support.AbstractApplicationContext refresh
警告: Exception encountered during context initialization - cancelling refresh attempt: org.springframework.beans.factory
.BeanCreationException: Error creating bean with name 'userBean' defined in class path resource [beans.xml]: Failed to instantiate
[org.cqipu.edu.bean.User] No default constructor found

org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'userBean' defined in class path resource
[beans.xml]: Failed to instantiate [org.cqipu.edu.bean.User]: No default constructor found

    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.instantiateBean
(AbstractAutowireCapableBeanFactory.java:1314)

```

通过测试得知：spring是通过调用类的无参数构造方法来创建对象的，所以要想让spring给你创建对象，必须保证无参数构造方法是存在的。

3. Spring是如何创建对象的呢？原理是什么？

```

Class clazz = Class.forName("org.cqipu.edu.bean.User");
Object obj = clazz.newInstance(); // 过时写法

// Class<?> clazz = Class.forName("org.cqipu.edu.bean.User");
// Object obj = clazz.getDeclaredConstructor().newInstance(); // 推荐写法

```

- o dom4j解析beans.xml文件，从中获取class的全限定类名
- o 通过反射机制调用无参数构造方法创建对象
- o 添加 dom4j 依赖到 pom.xml

```

<!-- 添加dom4j 依赖 -->
<dependency>
    <groupId>org.dom4j</groupId>
    <artifactId>dom4j</artifactId>
    <version>2.1.4</version>
</dependency>

```

- Spring6Test.java
 - o 解析 XML 的核心代码

```

package org.cqipu.edu;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

```

```
import java.lang.reflect.InvocationTargetException;
public class Spring6Test {
    @Test
    public void testFirst() throws ClassNotFoundException, InstantiationException, IllegalAccessException,
NoSuchMethodException, InvocationTargetException {
        // 1. 通过反射手动创建User对象
        Class<?> clazz = Class.forName("org.cqipu.edu.bean.User"); // 加载User类
        Object obj = clazz.getDeclaredConstructor().newInstance(); // 调用无参构造创建实例
        System.out.println("手动反射创建的User对象: " + obj);

        // 2. 通过Spring IoC容器创建User对象
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("beans.xml"); // 初始化Spring容器,
        加载配置文件
        Object userBean = applicationContext.getBean("userBean"); // 从容器获取名为userBean的Bean
        System.out.println("Spring容器创建的User对象: " + userBean);
    }
}
```

运行测试程序：

✓ 测试 已通过: 1共 1 个测试 – 162毫秒

D:\Java21\bin\java.exe ...

User的无参数构造方法执行

手动反射创建的User对象: org.cqipu.edu.bean.User@59a6e353

User的无参数构造方法执行

Spring容器创建的User对象: org.cqipu.edu.bean.User@74f0ea28

4. 把创建好的对象存储到一个什么样的数据结构当中了呢？

Map<String,Object>

key(id)	value(bean)
"userBean"	User对象
"vipBean"	Vip对象
"studentBean"	Student对象
"userServiceBean"	UserService对象

老杜

5. spring配置文件的名字必须叫做beans.xml吗？

- Spring 配置文件的名称绝对不是必须叫beans.xml

- 三种方式

● ● ●

```
@Test
public void testFirst() throws Exception {
    // 加载自定义名称的配置文件: spring-oa-config.xml
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-oa-config.xml");
    Object userBean = applicationContext.getBean("userBean");
    System.out.println(userBean);
}
```

● ● ●

```
// 加载config目录下的oa-beans.xml
ApplicationContext applicationContext = new ClassPathXmlApplicationContext("config/oa-beans.xml");
```

● ● ●

```
// 加载多个自定义名称的配置文件
String[] configFiles = {"user-config.xml", "order-config.xml"};
ApplicationContext applicationContext = new ClassPathXmlApplicationContext(configFiles);
```

通过以上的java代码可以看出，这个spring配置文件名字是我们负责提供的，显然spring配置文件的名字是随意的。

6. 像这样的beans.xml文件可以有多个吗？

再创建一个spring配置文件，起名：spring.xml

● ● ●

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="vipBean" class="org.cqipu.edu.bean.Vip"/>
</beans>
```

- Spring6Test.testFirst02()

● ● ●

```
package org.cqipu.edu;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import java.lang.reflect.InvocationTargetException;
public class Spring6Test {

    @Test
    public void testFirst02(){
        // 初始化Spring容器上下文（解析beans.xml文件，创建所有的bean对象）
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("beans.xml","spring.xml");

        // 根据id获取bean对象
        Object userBean = applicationContext.getBean("userBean");
        Object vipBean = applicationContext.getBean("vipBean");

        System.out.println(userBean);
        System.out.println(vipBean);
    }
}
```

运行测试程序：

```
D:\Java21\bin\java.exe ...
```

User的无参数构造方法执行

```
org.cqipu.edu.bean.User@49e53c76
```

```
org.cqipu.edu.bean.Vip@351d00c0
```

通过测试得知，spring的配置文件可以有多个，在ClassPathXmlApplicationContext构造方法的参数上传递文件路径即可。这是为什么呢？通过源码可以看到：

```
1 usage
public ClassPathXmlApplicationContext(String... configLocations) throws BeansException {
    this(configLocations, refresh: true, (ApplicationContext)null);
}
```

老杜

7. 在配置文件中配置的类必须是自定义的吗，可以使用JDK中的类吗，例如：java.util.Date？

- beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userBean" class="org.cqipu.edu.bean.User"/>
    <!-- <bean id="vipBean" class="org.cqipu.edu.bean.Vip"/>-->
    <bean id="dateBean" class="java.util.Date"/>
</beans>
```

- Spring6Test.testFirst02()
 - 新增 bean 对象

```
package com.powernode.spring6.test;

import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Spring6Test {

    @Test
    public void testFirst(){
        // 初始化Spring容器上下文（解析beans.xml文件，创建所有的bean对象）
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("beans.xml", "spring.xml");

        // 根据id获取bean对象
        Object userBean = applicationContext.getBean("userBean");
        Object vipBean = applicationContext.getBean("vipBean");
        Object dateBean = applicationContext.getBean("dateBean");

        System.out.println(userBean);
        System.out.println(vipBean);
        System.out.println(dateBean);
    }
}
```



```
}
```

运行测试程序：

✓ 测试 已通过: 1共 1 个测试 - 193毫秒

```
D:\Java21\bin\java.exe ...
User的无参数构造方法执行
org.cqipu.edu.bean.User@351d00c0
org.cqipu.edu.bean.Vip@2a3b5b47
Mon Mar 16 22:15:19 CST 2026
```

通过测试得知，在spring配置文件中配置的bean可以任意类，只要这个类不是抽象的，并且提供了无参数构造方法。

8. `getBean()`方法调用时，如果指定的id不存在会怎样？

```
@Test
public void testFirst02(){
    // 初始化Spring容器上下文（解析beans.xml文件，创建所有的bean对象）
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext( ...configLocations: "be

    // 根据id获取bean对象
    Object userBean = applicationContext.getBean( name: "hahaBean")
    Object vipBean = applicationContext.getBean( name: "vipBean");
    Object dateBean = applicationContext.getBean( name: "dateBean");

    System.out.println(userBean);
    System.out.println(vipBean);
    System.out.println(dateBean);
}
```

运行测试程序：

```
D:\Java21\bin\java.exe ...
```

User的无参数构造方法执行

```
org.springframework.beans.factory.NoSuchBeanDefinitionException No bean named 'hahaBean' available
```

```
at org.springframework.beans.factory.support.DefaultListableBeanFactory.getBeanDefinition(DefaultListableBeanFactory.java:893)
at org.springframework.beans.factory.support.AbstractBeanFactory.getMergedLocalBeanDefinition(AbstractBeanFactory.java:1318)
at org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBeanFactory.java:300)
at org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBeanFactory.java:200)
at org.springframework.context.support.AbstractApplicationContext.getBean(AbstractApplicationContext.java:1154)
> at org.cqipu.edu.Spring6Test.testFirst02(Spring6Test.java:36) <25 个内部行>
```

通过测试得知，当id不存在的时候，会出现异常。

9. `getBean()`方法返回的类型是Object，如果访问子类的特有属性和方法时，还需要向下转型，有其它办法可以解决这个问题吗？

- 最推荐的方式就是使用带 **类型参数** 的`getBean()`重载方法
 - 旧写法，需转型

```
● ● ●
// 返回Object，必须手动向下转型
Object userObj = applicationContext.getBean("userBean");
// 转型失败会抛ClassCastException（运行时错误）
User user = (User) userObj;
// 才能调用User的特有方法/属性
user.setName("张三");
```

- 带类型参数的`getBean()`重载方法



```
<T> T getBean(String name, Class<T> requiredType);

// User user = applicationContext.getBean("userBean", User.class);
```

- `<T>`：泛型参数，指定返回值类型；
- name: bean 的 id (如userBean)；
- requiredType: 目标类型的Class对象 (如User.class)；
- 在User 实体类加入get、set方法



```
package org.cqipu.edu.bean;
public class User {
    private String username;
    private String password;

    // 无参构造
    public User() {}

    // setter/getter 方法
    public void setUsername(String username) {
        this.username = username;
    }
    public String getUsername() {
        return username;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public String getPassword() {
        return password;
    }
}
```

- Spring6Test.testBeanType().java



```
import org.cqipu.edu.bean.User;

@Test
public void testBeanType() {
    ApplicationContext ctx = new ClassPathXmlApplicationContext("beans.xml");

    // 新写法：无需转型，类型安全
    User user = ctx.getBean("userBean", User.class);
    // 直接调用User的特有方法/属性（无编译警告、无运行时转型风险）
    user.setUsername("admin");
    user.setPassword("123456");
    System.out.println(user.getUsername()); // 直接访问子类特有属性

    // 旧写法：需手动转型，类型不安全
    Object userObj = ctx.getBean("userBean");
    User userOld = (User) userObj; // 编译期无检查，易出错
}
```

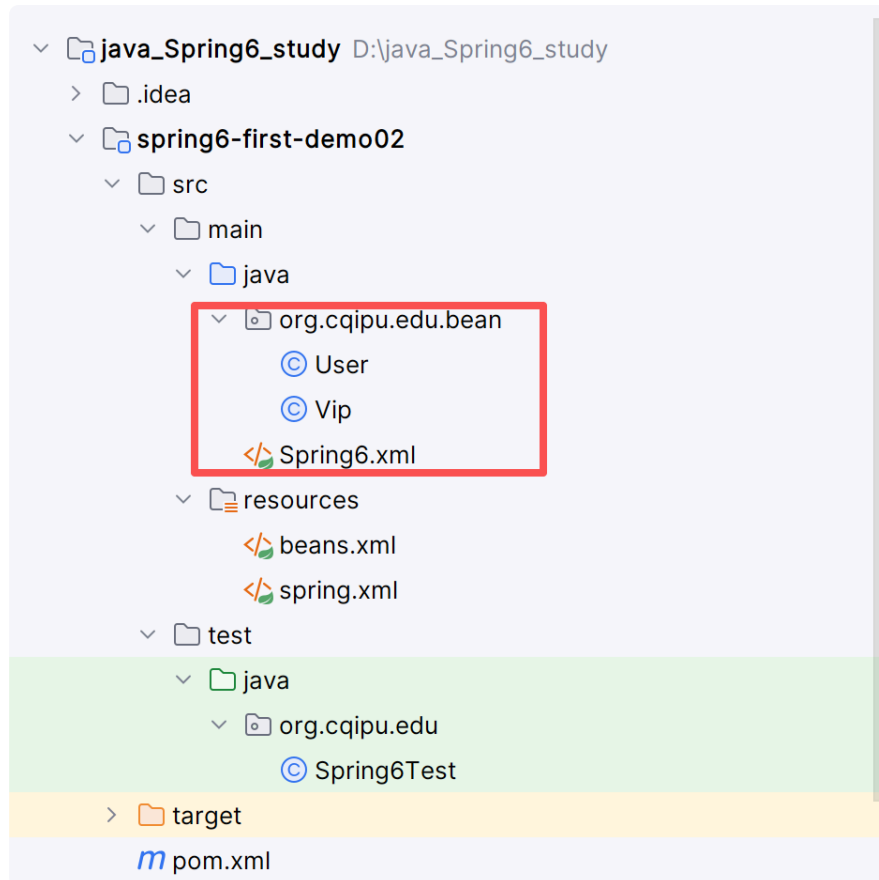
运行测试程序：

✓ 测试 已通过: 1共 1 个测试 – 162毫秒

D:\Java21\bin\java.exe ...

User的无参数构造方法执行
admin

10. ClassPathXmlApplicationContext 是从类路径中加载配置文件，如果没有在类路径当中，又应该如何加载配置文件呢？



- spring6.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="vipBean2" class="org.cqipu.edu.bean.Vip"/>
</beans>
```

- Vip.java 实体类

```
package org.cqipu.edu.bean;

public class Vip {
    public Vip() {
        System.out.println("Vip的无参数构造方法执行");
    }
}
```

- Spring6Test.testFirst03().java

```

package org.cqipu.edu;
import org.cqipu.edu.bean.User;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import java.lang.reflect.InvocationTargetException;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import org.cqipu.edu.bean.Vip;
public class Spring6Test {
    @Test
    public void testFileSystemXml() throws Exception {
        // 1. 填写 Spring6.xml 的绝对磁盘路径
        // 根据目录树, Spring6.xml 实际位置:
        // D:\java_Spring6_study\spring6-first-demo02\src\main\java\Spring6.xml
        ApplicationContext applicationContext2 = new FileSystemXmlApplicationContext(
            "D:/java_Spring6_study/spring6-first-demo02/src/main/java/Spring6.xml"
        );
        // 2. 从容器中获取 Vip Bean
        Vip vip = applicationContext2.getBean("vipBean2", Vip.class);

        // 3. 输出结果
        System.out.println("从文件系统加载 XML 成功, Vip 对象: " + vip);
    }
}

```

运行测试程序：

```
D:\Java21\bin\java.exe ...
```

Vip的无参数构造方法执行

从文件系统加载 XML 成功, Vip 对象: org.cqipu.edu.bean.Vip@6f1de4c7

没有在类路径中的话, 需要使用FileSystemXmlApplicationContext类进行加载配置文件。

这种方式较少用。一般都是将配置文件放到类路径当中, 这样可移植性更强。

11. ApplicationContext的超级父接口BeanFactory。

- 多态核心的体系

```

// 父接口 (BeanFactory) ← 子类实现 (ClassPathXmlApplicationContext)
BeanFactory beanFactory = new ClassPathXmlApplicationContext("spring.xml");

```

- BeanFactory 是 Spring IoC 容器的**顶级父接口**, 定义了 IoC 容器的核心规范;
- ApplicationContext 是 BeanFactory 的**子接口** (间接继承), 扩展了更多功能;

```

package org.cqipu.edu;

import org.cqipu.edu.bean.Vip;
import org.junit.Test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Spring6BeanFactoryTest {

    @Test

```

```

public void testBeanFactory() {
    BeanFactory beanFactory = new ClassPathXmlApplicationContext("spring.xml");

    // 2. 方式1: 基础写法 - 返回 Object 类型, 需手动转型
    Object vipBean = beanFactory.getBean("vipBean");
    System.out.println("BeanFactory 获取 Object 类型 Vip: " + vipBean);

    // 3. 方式2: 推荐写法 - 指定类型, 无需转型 (BeanFactory 也支持泛型重载方法)
    Vip vip = beanFactory.getBean("vipBean", Vip.class);
    System.out.println("BeanFactory 获取 Vip 类型对象: " + vip);
}
}

```

- 上述代码是面向接口编程的典型体现, 父接口引用子类对象, 降低耦合。

执行测试程序:

```
D:\Java21\bin\java.exe ...
```

Vip的无参数构造方法执行

BeanFactory 获取 Object 类型 Vip: org.cqipu.edu.bean.Vip@6aeb35e6

BeanFactory 获取 Vip 类型对象: org.cqipu.edu.bean.Vip@6aeb35e6

Spring6 启用Log4j2日志框架

从Spring5之后, Spring框架支持集成的日志框架是Log4j2. 如何启用日志框架:

第一步: 引入Log4j2的依赖

- pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.cqipu.edu</groupId>
    <artifactId>spring6-first-demo02</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- JDK21+编码+Spring版本统一管理 -->
    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <spring.version>6.0.10</spring.version> <!-- 选择合适的spring 版本 -->
        <junit.version>4.13.2</junit.version> <!-- 选择合适的junit 版本 -->
        <log4j2.version>2.20.0</log4j2.version> <!-- 选择合适的 Log4j2 版本 -->
    </properties>

    <dependencies>
        <!-- Spring6 核心依赖 (仅需spring-context, 自动拉取core/beans等) -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>${spring.version}</version> <!-- 用属性统一管理, 易维护 -->
        </dependency>
    </dependencies>

```

```

<!-- JUnit 4.13.2 测试依赖 -->
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope> <!-- 仅测试阶段生效，主代码不依赖 -->
</dependency>

<!-- 添加dom4j依赖 -->
<dependency>
    <groupId>org.dom4j</groupId>
    <artifactId>dom4j</artifactId>
    <version>2.1.4</version>
</dependency>

<!-- Log4j2 核心依赖 -->
<!-- Log4j2 API（日志接口） -->
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>${log4j2.version}</version>
</dependency>
<!-- Log4j2 核心实现（日志功能） -->
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>${log4j2.version}</version>
</dependency>
<!-- Log4j2 适配SLF4J -->
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-slf4j-impl</artifactId>
    <version>${log4j2.version}</version>
</dependency>
</dependencies>

<!-- 指定编译插件版本（适配JDK21，避免版本兼容报错） -->
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.11.0</version> <!-- 支持JDK21的最低版本 -->
            <configuration>
                <source>21</source>
                <target>21</target>
                <encoding>UTF-8</encoding>
            </configuration>
        </plugin>
    </plugins>
</build>
</project>

```

第二步：在类的根路径下提供log4j2.xml配置文件（文件名固定为：log4j2.xml，文件必须放到类根路径下。）

- log4j2.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>

    <loggers>
        <!--
            Level 指定日志级别，从低到高的优先级：

```

```

        ALL < TRACE < DEBUG < INFO < WARN < ERROR < FATAL < OFF
-->
<root level="DEBUG">
    <appender-ref ref="spring6Log"/>
</root>
</loggers>

<appenders>
    <!-- 输出日志信息到控制台 -->
    <console name="spring6Log" target="SYSTEM_OUT">
        <!-- 控制日志输出的格式 -->
        <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss SSS} [%t] %-3level %Logger{1024} - %msg%n"/>
    </console>
</appenders>
</configuration>

```

- 第三步：使用日志框架
 - Spring6BeanFactoryTest.java

```

package org.cqipu.edu;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.cqipu.edu.bean.Vip;
import org.junit.Test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Spring6BeanFactoryTest {

    // 1. 获取 Log4j2 日志对象
    private static final Logger logger = LogManager.getLogger(Spring6BeanFactoryTest.class);

    @Test
    public void testBeanFactory() {
        // 2. 输出不同级别日志
        logger.info("==== 开始初始化 Spring 容器 =====");
        BeanFactory beanFactory = new ClassPathXmlApplicationContext("spring.xml");

        logger.debug("容器初始化完成, 开始获取 Vip Bean"); // debug 级别
        Vip vip = beanFactory.getBean("vipBean", Vip.class);

        logger.info("成功获取 Vip Bean: {}", vip); // 占位符输出对象
        logger.warn("测试警告级别日志");
    }
}

```

执行测试程序

```

D:\Java21\bin\java.exe ...
2026-03-17 09:09:12 818 [main] INFO org.cqipu.edu.Spring6BeanFactoryTest - ===== 开始初始化 Spring 容器 =====
2026-03-17 09:09:12 855 [main] DEBUG org.springframework.context.support.ClassPathXmlApplicationContext - Refreshing org
    .springframework.context.support.ClassPathXmlApplicationContext@2dc9b0f5
2026-03-17 09:09:12 960 [main] DEBUG org.springframework.beans.factory.xml.XmlBeanDefinitionReader - Loaded 1 bean definitions from
    class path resource [spring.xml]
2026-03-17 09:09:12 989 [main] DEBUG org.springframework.beans.factory.support.DefaultListableBeanFactory - Creating shared instance
    of singleton bean 'vipBean'
Vip的无参数构造方法执行
2026-03-17 09:09:13 029 [main] DEBUG org.cqipu.edu.Spring6BeanFactoryTest - 容器初始化完成, 开始获取 Vip Bean
2026-03-17 09:09:13 066 [main] INFO org.cqipu.edu.Spring6BeanFactoryTest - 成功获取 Vip Bean: org.cqipu.edu.bean.Vip@56e8b606
2026-03-17 09:09:13 067 [main] WARN org.cqipu.edu.Spring6BeanFactoryTest - 测试警告级别日志

```

Spring 对 IoC 的实现

IoC 控制反转

概述

详细代码讲解可看第一节Spring启示录，接下来进行回顾IoC 控制反转

- 全称：IoC = Inversion of Control 控制反转
- 把对象的创建权、对象之间依赖关系的管理权，从程序员手中反转交给第三方容器（Spring）统一管理，程序员不再手动 new 对象。

控制反转是为了降低程序耦合度，提高程序扩展力，达到OCP原则，达到DIP原则。

控制反转，不是什么新技术，而是一种设计思想，用来解决对象耦合问题。

控制反转，反转的是什么？

1. 对象的创建权
 - 传统：程序员 → new UserDaoImpl() → 创建对象
 - IoC 后：Spring 容器 → 通过反射创建对象 → 程序员只管拿，不管造
2. 依赖的管理权
 - 传统：程序员 → 手动维护 Service 和 Dao 的依赖关系
 - IoC 后：Spring 容器 → 自动把 Dao 注入给 Service，程序员不用管

IoC 与 DI

IoC 是思想，DI 是实现 IoC 的具体手段！

概念	定位	说明
IoC	设计思想 / 原则	目标：解耦，反转对象控制权
DI	Dependency Injection 依赖注入	手段：Spring 通过 DI 完成 IoC 思想落地

Spring 实现 IoC 思想的**唯一方式**就是 **依赖注入（DI）**。

Spring 实现 IoC 的底层原理

底层原理共有3个，如下：

1. 配置元数据
 - 告诉 Spring 要创建哪些对象，对象之间的关系
 - 形式：XML 配置文件、注解、JavaConfig 类
2. 工厂模式（BeanFactory）
 - Spring 用工厂模式统一生产 Bean，对外提供获取 Bean 的接口，屏蔽对象创建细节。
3. Java 反射机制

Spring 不用new，而是通过类的全限定名，用反射：

- Class.forName("com.xxx.UserDaoImpl")

- newInstance() / 调用构造方法 / 调用 setter

这也是为什么：XML 配置里写全类名，Bean 必须有无参构造（默认）。

Spring IoC 完整流程

1. 加载配置元数据（XML / 注解）
2. 解析配置，得到 Bean 定义信息
3. 通过反射创建 Bean 实例
4. 通过依赖注入（DI）维护 Bean 之间的关系
5. 把 Bean 放入 Spring 容器（Bean 池）
6. 程序员从容器获取 Bean 使用

Spring 中 IoC 的容器载体

IoC 思想的落地，就是 Spring 的 IoC 容器，有两个核心接口：

1. BeanFactory（顶层接口）
 - Spring IoC 容器的顶层接口，基础容器
 - 懒加载：用到 Bean 时才创建
 - 定位：底层原理用，开发极少直接用
2. ApplicationContext（常用容器）
 - BeanFactory 的子接口，功能增强容器
 - 饿汉式：容器启动就创建所有单例 Bean
 - 上课、企业开发100% 用这个
 - 常用实现类：ClassPathXmlApplicationContext（读取 XML）

IoC 核心价值与总结

核心价值：

1. 彻底解耦：对象之间不再硬编码依赖，修改实现类不用改源码
2. 符合设计原则：满足 OCP 开闭原则、DIP 依赖倒置原则
3. 易于测试：可轻松替换 mock 对象做单元测试
4. 统一管理对象：Spring 容器控制对象的创建、生命周期、依赖

总结：

1. IoC 是控制反转，是解决对象耦合的设计思想
2. 反转的是：对象创建权 + 依赖管理权，从程序员反转给 Spring 容器
3. IoC 是思想，DI 是实现 IoC 的具体手段
4. Spring IoC 底层靠：配置 + 工厂模式 + Java 反射
5. 开发用ApplicationContext作为 IoC 容器

DI 依赖注入

IoC 是“要解耦”的目标，DI 是“怎么解耦”的具体操作

概述

DI (Dependency Injection)：依赖注入，指容器在创建 Bean 的过程中，自动将该 Bean 所依赖的其他 Bean（或数据）赋值给它的属性，无需程序员手动赋值。

DI 的核心目标：给容器中的 Bean “填依赖”

```
● ● ●
// 1. 定义的类（需要被容器管理，且有依赖）
public class UserService {
    // 依赖: UserDao是UserService的“依赖”
    private UserDao userDao;

    // （此处先留空，后续会加set方法/构造方法）
    public void addUser() {
        userDao.insert(); // 要让这行不报错，必须给userDao赋值
    }
}

// 2. 告诉Spring: 要创建UserService和UserDao的对象
// （后续会写XML配置/注解，这里先理解逻辑）
```

DI 要解决的核心问题：

Spring 容器创建了 UserService 对象后，如何把 UserDao 对象“填”到 UserService 的 userDao 属性里？—— 这就是依赖注入要干的事。

DI 的 2 个基本前提

不管是 set 注入还是构造注入，先满足这两个前提：

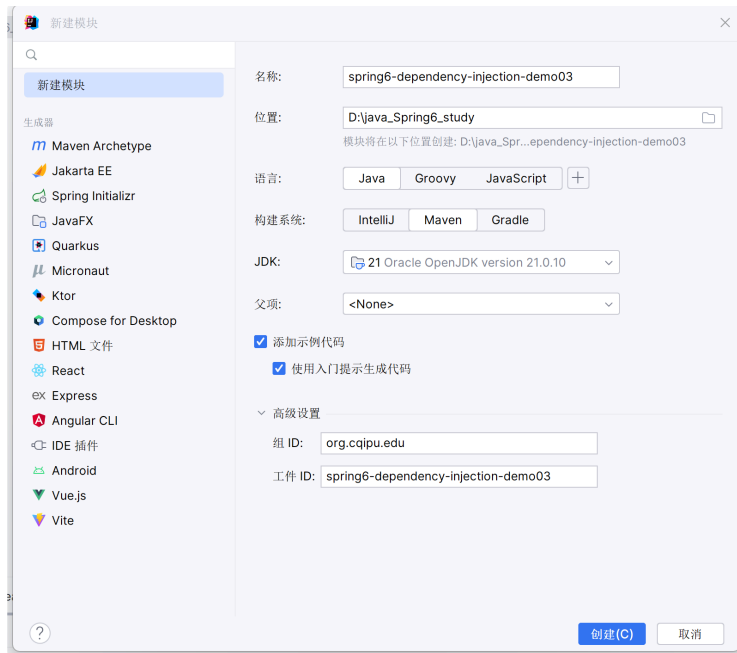
1. 前提 1：需要被注入的“依赖对象”(比如 UserDao)，必须也由 Spring 容器管理→ 通俗说：容器只能“拿自己的东西”给你，不能给你它没管理的对象；
2. 前提 2：被注入的类（比如 UserService），必须提供“接收依赖”的入口→ 通俗说：你得给容器留个“口子”，让它能把依赖塞进来（这个“口子”就是后续要学的setter 方法或构造方法）。

DI 的两大核心实现方向

Spring 给 Bean “填依赖” 只有两种核心思路：

1. 方向 1：“事后赋值”—— 基于Setter方法的注入
 - 容器先创建空的 Bean 对象（比如 UserService），再通过某个“普通方法”给属性赋值（这个方法就是 setter 方法）；
2. 方向 2：“天生赋值”—— 基于构造的注入
 - 容器在创建 Bean 对象的那一刻，就通过构造方法把依赖传进去，对象创建出来就带着完整的依赖。

新建模块：spring6-dependency-injection-demo03



Setter 注入

容器先创建 Bean 的空对象（通过无参构造），再通过反射调用该 Bean 的 `setXxx()` 方法，将容器中管理的其他 Bean（或简单值）赋值给当前 Bean

的属性 —— 这就是 Set 方法注入。

先造对象，再赋值（事后赋值），是开发中最常用的 DI 方式。

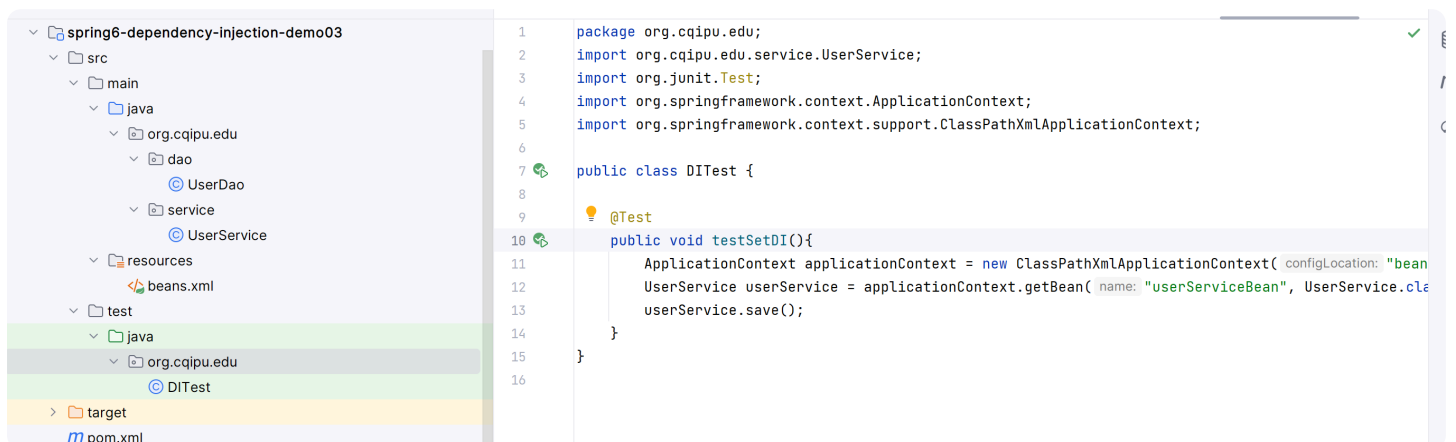
核心前提

1. 被注入的依赖对象（比如 UserDao）必须由 Spring 容器管理（要在 XML 里配 `<bean>`）；
2. 被注入的目标类（比如 UserService）必须为依赖属性提供标准的 setter 方法（遵循 JavaBean 规范：set+属性名首字母大写，比如 `setUserDao()`）；
3. 目标类必须有无参构造方法（默认有，除非你写了带参构造但没显式写无参）——Spring 创建 Bean 的默认方式是无参构造。

环境准备

- JDK: 21
- Spring 依赖: spring-context--6.0.10
- 新建模块: spring6-dependency-injection-demo03
- junit 依赖: 4.13.2

项目结构



set注入，基于set方法实现的，底层会通过反射机制调用属性对应的set方法然后给属性赋值。这种方式要求属性必须对外提供set方法。

- pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.cqipu.edu</groupId>
    <artifactId>spring6-dependency-injection-demo03</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- JDK21+编码+Spring版本统一管理 -->
    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <spring.version>6.0.10</spring.version> <!-- 选取合适的spring 版本 -->
        <junit.version>4.13.2</junit.version> <!-- 选取合适的junit 版本 -->
        <log4j2.version>2.20.0</log4j2.version> <!-- 选取合适的 Log4j2 版本 -->
    </properties>

    <dependencies>
        <!-- Spring6 核心依赖（仅需spring-context，自动拉取core/beans等） -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>${spring.version}</version> <!-- 用属性统一管理，易维护 -->
        </dependency>

        <!-- JUnit 4.13.2 测试依赖 -->
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope> <!-- 仅测试阶段生效，主代码不依赖 -->
        </dependency>

        <!-- 添加dom4j 依赖 -->
        <dependency>
            <groupId>org.dom4j</groupId>
            <artifactId>dom4j</artifactId>
            <version>2.1.4</version>
        </dependency>

        <!-- Log4j2 核心依赖 -->
        <!-- Log4j2 API（日志接口） -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-api</artifactId>
            <version>${log4j2.version}</version>
        </dependency>

        <!-- Log4j2 核心实现（日志功能） -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-core</artifactId>
            <version>${log4j2.version}</version>
        </dependency>

        <!-- Log4j2 适配SLF4J -->
        <dependency>
```

```

        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-slf4j-impl</artifactId>
        <version>${log4j2.version}</version>
    </dependency>
</dependencies>

<!-- 指定编译插件版本（适配JDK21，避免版本兼容报错） -->
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.11.0</version> <!-- 支持JDK21的最低版本 -->
            <configuration>
                <source>21</source>
                <target>21</target>
                <encoding>UTF-8</encoding>
            </configuration>
        </plugin>
    </plugins>
</build>
</project>

```

- UserDao

```

package org.cqipu.edu.dao;

public class UserDao {
    public void insert(){
        System.out.println("执行UserDaoImpl的insert方法：插入用户数据");
    }
}

```

- UserService

```

package org.cqipu.edu.service;
import org.cqipu.edu.dao.UserDao;

public class UserService {
    private UserDao userDao;

    // 使用set方式注入，必须提供set方法。
    // 反射机制要调用这个方法给属性赋值的。
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        userDao.insert();
    }
}

```

- beans.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="userDaoBean" class="org.cqipu.edu.dao.UserDao"/>

    <bean id="userServiceBean" class="org.cqipu.edu.service.UserService">
        <property name="userDao" ref="userDaoBean"/>
    </bean>
</beans>
```

- 测试程序

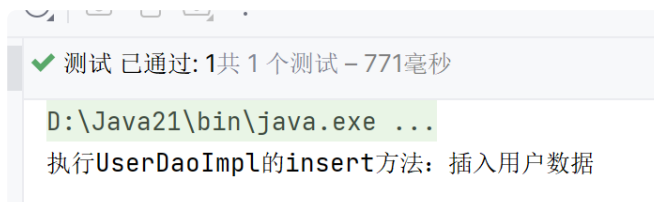


```
package org.cqipu.edu;
import org.cqipu.edu.service.UserService;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class DITest {

    @Test
    public void testSetDI(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("beans.xml");
        UserService userService = applicationContext.getBean("userServiceBean", UserService.class);
        userService.save();
    }
}
```

运行结果：



重点内容是，什么原理：

- beans.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="userDaoBean" class="org.cqipu.edu.dao.UserDao"/>

    <bean id="userServiceBean" class="org.cqipu.edu.service.UserService">
        <property name="userDao" ref="userDaoBean"/>
    </bean>
</beans>
```

实现原理：

通过property标签获取到属性名：userDao

通过属性名推断出set方法名：setUserDao

通过反射机制调用setUserDao()方法给属性赋值

property标签的name是属性名。

property标签的ref是要注入的bean对象的id。(通过ref属性来完成bean的装配，这是bean最简单的一种装配方式。装配指的是：创建系统组件之间关联的动作)

可以把set方法注释掉，再测试一下：

```
D:\Java21\bin\java.exe ...  
  
org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'userServiceBean' defined in class path  
resource [beans.xml]: Invalid property 'userDao' of bean class [org.cqipu.edu.service.UserService]: Bean property 'userDao' is not  
writable or has an invalid setter method. Does the parameter type of the setter match the return type of the getter?  
  
at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.applyPropertyValues  
(AbstractAutowireCapableBeanFactory.java:1718)
```

通过测试得知，底层实际上调用了setUserDao()方法。所以需要确保这个方法的存在。

我们现在把属性名修改一下，但方法名还是setUserDao()，我们来测试一下：

- UserService.java

```
package org.cqipu.edu.service;  
  
import org.cqipu.edu.dao.UserDao;  
  
public class UserService {  
    private UserDao aaa;  
  
    // 使用set方式注入，必须提供set方法。  
    // 反射机制要调用这个方法给属性赋值的。  
    public void setUserDao(UserDao userDao) {  
        this.aaa = userDao;  
    }  
  
    public void save(){  
        aaa.insert();  
    }  
}
```

运行结果：

✓ 测试 已通过: 1共 1 个测试 - 758毫秒

```
D:\Java21\bin\java.exe ...
```

执行UserDaoImpl的insert方法：插入用户数据

通过测试看到程序仍然可以正常执行，说明property标签的name是：setUserDao()方法名演变得到的。演变的规律是：

- setUsername() 演变为 username
- setPassword() 演变为 password
- setUserDao() 演变为 userDao
- setUserService() 演变为 userService

另外，对于property标签来说，ref属性也可以采用标签的方式，但使用ref属性是多数：



```
<bean id="userServiceBean" class="org.cqipu.edu.service.UserService">
  <property name="userDao">
    <ref bean="userDaoBean"/>
  </property>
</bean>
```

总结：set注入的核心实现原理：通过反射机制调用set方法来给属性赋值，让两个对象之间产生关系。

构造注入

核心原理：通过调用构造方法来给属性赋值。

构造注入是“天生赋值”(创建对象时就塞依赖)，适合必选依赖；Set注入是“事后赋值”(创建对象后再塞)，适合可选依赖，二者是Spring DI的两大核心。

- 官方定义
 - 容器在创建 Bean 对象的那一刻，直接调用该 Bean 的带参构造方法，将依赖的 Bean（或简单值）作为构造参数传入，完成依赖注入。
 - 核心是“先赋值，再创建完整对象”，对象创建出来就有完整的依赖，不会出现“空依赖”。

与 Set 注入的核心对比

特性	Set 方法注入	构造方法注入
赋值时机	对象创建后（事后赋值）	对象创建时（天生赋值）
依赖类型	可选依赖（可有可无）	必选依赖（必须有）
核心标签	<property>	<constructor-arg>
核心前提	有 setter 方法 + 无参构造	有对应带参构造方法

- 核心前提
 - 被注入的依赖对象（比如 UserDao）必须由 Spring 容器管理（XML 中配 <bean>）；
 - 被注入的目标类（比如 UserService）必须提供对应参数的带参构造方法（参数是要注入的依赖）；
 - 无参构造可选：如果目标类只有带参构造，Spring 会用带参构造创建对象；如果既有无参又有带参，Spring 默认用无参（但构造注入会指定用带参）。
- 环境准备

和 Set 注入完全一致：JDK21、spring-context 依赖、相同的项目结构。

- OrderDao.java



```
package org.cqipu.edu.dao;

public class OrderDao {
    public void deleteById(){
        System.out.println("正在删除订单。。。");
    }
}
```

- OrderService.java



```
package org.cqipu.edu.service;
```



```
import org.cqipu.edu.dao.OrderDao;

public class OrderService {
    private OrderDao orderDao;

    // 通过反射机制调用构造方法给属性赋值
    public OrderService(OrderDao orderDao) {
        this.orderDao = orderDao;
    }

    public void delete(){
        orderDao.deleteById();
    }
}
```

- beans.xml

```
<bean id="orderDaoBean" class="com.powernode.spring6.dao.OrderDao"/>
<bean id="orderServiceBean" class="com.powernode.spring6.service.OrderService">
    <!--index="0"表示构造方法的第一个参数，将orderDaoBean对象传递给构造方法的第一个参数。-->
    <constructor-arg index="0" ref="orderDaoBean"/>
</bean>
```

- 测试程序

```
package org.cqipu.edu;
import org.cqipu.edu.service.OrderService;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class DITest {
    @Test
    public void testConstructorDI(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("beans.xml");
        OrderService orderServiceBean = applicationContext.getBean("orderServiceBean", OrderService.class);
        orderServiceBean.delete();
    }
}
```

运行结果如下：

✓ 测试 已通过: 1共 1 个测试 - 460毫秒

D:\Java21\bin\java.exe ...

正在删除订单。。。

进程已结束，退出代码为 0

如果构造方法有两个参数：

- OrderService.java

```
package org.cqipu.edu.service;
import org.cqipu.edu.dao.OrderDao;
import org.cqipu.edu.dao.UserDao;
public class OrderService {
    private OrderDao orderDao;
    private UserDao userDao;
```

```
// 通过反射机制调用构造方法给属性赋值
public OrderService(OrderDao orderDao, UserDao userDao) {
    this.orderDao = orderDao;
    this.userDao = userDao;
}

public void delete(){
    orderDao.deleteById();
    userDao.insert();
}
}
```

- beans.xml



```
<bean id="orderDaoBean" class="org.cqipu.edu.dao.OrderDao"/>
<bean id="orderServiceBean" class="org.cqipu.edu.service.OrderService">
    <!-- index="0"表示构造方法的第一个参数，将orderDaoBean对象传递给构造方法的第一个参数。-->
    <!-- 第一个参数下标是0-->
    <constructor-arg index="0" ref="orderDaoBean"/>
    <!-- 第二个参数下标是1-->
    <constructor-arg index="1" ref="userDaoBean"/>
</bean>
<bean id="userDaoBean" class="org.cqipu.edu.dao.UserDao"/>
```

运行测试：

```
测试已通过，共1个测试 = 431毫秒
D:\Java21\bin\java.exe ...
正在删除订单。。。
执行UserDaoImpl的insert方法：插入用户数据
```

不使用参数下标，使用参数的名字可以吗？

- beans.xml



```
<bean id="orderDaoBean" class="org.cqipu.edu.dao.OrderDao"/>
<bean id="orderServiceBean" class="org.cqipu.edu.service.OrderService">
    <!-- 这里使用了构造方法上参数的名字-->
    <constructor-arg name="orderDao" ref="orderDaoBean"/>
    <constructor-arg name="userDao" ref="userDaoBean"/>
</bean>
<bean id="userDaoBean" class="org.cqipu.edu.dao.UserDao"/>
```

测试运行：

```
D:\Java21\bin\java.exe ...
正在删除订单。。。
执行UserDaoImpl的insert方法：插入用户数据
```

配置文件中构造方法参数的类型顺序和构造方法参数的类型顺序不一致呢？

- beans.xml

```
<bean id="orderDaoBean" class="org.cqipu.edu.dao.OrderDao"/>
<bean id="orderServiceBean" class="org.cqipu.edu.service.OrderService">
  <!-- 顺序已经和构造方法的参数顺序不同了-->
  <constructor-arg ref="userDaoBean"/>
  <constructor-arg ref="orderDaoBean"/>
</bean>
<bean id="userDaoBean" class="org.cqipu.edu.dao.UserDao"/>
```

测试运行：

```
D:\Java21\bin\java.exe ...
```

正在删除订单。。。

执行UserDaoImpl的insert方法：插入用户数据

通过测试得知，通过构造方法注入的时候：

- 可以通过下标
- 可以通过参数名
- 也可以不指定下标和参数名，可以类型自动推断。

Spring在装配方面做的还是比较健壮的

- 无参构造的影响
 - 如果目标类只有带参构造：Spring 会用这个带参构造创建对象（构造注入）；
 - 如果目标类既有无参又有带参：Spring 默认用无参构造，此时必须通过 `<constructor-arg>` 指定用带参构造（否则依赖注入失败）；
 - 如果手动写了带参构造，但没写无参，又没配置 `<constructor-arg>`：运行会报NoSuchMethodException（找不到无参构造）。

优先用构造注入处理必选依赖，Set 注入处理可选依赖

Setter 注入专题

注入外部 Bean

在这一章之前，使用的案例就是注入外部Bean的方式。

- 外部 Bean：指在 Spring 配置文件中独立配置、有自己唯一 id 的 Bean
- 注入外部 Bean：通过 `<property ref="外部Bean的id"/>`，将这个独立的 Bean 注入到另一个 Bean 中

外部 Bean 是容器中“公共可用”的 Bean，可被多个目标 Bean 复用。

- 前提
 - 外部 Bean 必须在 XML 中独立配置
 - 目标 Bean 必须为依赖属性提供setter 方法；
 - 注入时用ref属性（而非value），ref的值必须和外部 Bean 的 id 完全一致。

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="userDaoBean" class="org.cqipu.edu.dao.UserDao"/>

    <bean id="userServiceBean" class="org.cqipu.edu.service.UserService">
        <property name="userDao" ref="userDaoBean"/>
    </bean>
</beans>
```

外部Bean的特点：bean定义到外面，在property标签中使用ref属性进行注入。通常这种方式是常用。

注入内部Bean

内部Bean的方式：在bean标签中嵌套bean标签。

- 内部 Bean：指不独立配置、嵌套在目标 Bean 的 `<property>` 标签内的 Bean（没有独立的 `<bean>` 标签，即使写了 id 也无效）；
- 注入内部 Bean：直接在目标 Bean 的 `<property>` 里嵌套 `<bean>` 标签，创建的对象仅给当前目标 Bean 使用，容器中无法通过 id 获取这个内部 Bean；

内部 Bean 是“一次性”的专属对象，不可被其他 Bean 复用，无需配置独立 id。

- 前提
 - 内部 Bean 无需独立配置，直接嵌套在目标 Bean 的 `<property>` 标签内；
 - 目标 Bean 必须为依赖属性提供 setter 方法；
 - 内部 Bean 即使配置了 `id`，也无法被容器或其他 Bean 引用（id 仅为“摆设”）；
 - 内部 Bean 的类必须有无参构造。
- spring-inner-bean.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="userServiceBean" class="org.cqipu.edu.service.UserService">
        <property name="userDao">
            <bean class="org.cqipu.edu.dao.UserDao"/>
        </property>
    </bean>
</beans>
```

- DITest.testInnerBean()

```
@Test
public void testInnerBean(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-inner-bean.xml");
    UserService userService = applicationContext.getBean("userServiceBean", UserService.class);
    userService.save();
}
```

运行结果：

✓ 测试 已通过: 1共 1 个测试 – 559毫秒

D:\Java21\bin\java.exe ...

执行 UserDaoImpl 的 insert 方法：插入用户数据

特性	内部 Bean	外部 Bean
配置位置	嵌套在 <code><property></code> 内，非独立	独立配置 <code><bean></code> 标签，有自己的 id
复用性	不可复用（仅当前 Bean 使用）	可复用（多个 Bean 可引用）
id 属性	写了也无效（容器不识别）	必须唯一，用于被引用
适用场景	依赖对象仅当前 Bean 使用	依赖对象需要被多个 Bean 共享
配置冗余度	低（无需独立配置）	稍高（需独立配置）

这种方式作为了解

注入简单类型

简单类型注入用value属性，对象类型用ref属性

- 简单类型：指 Java 的八大基本数据类型（byte/short/int/long/float/double/char/boolean）+ String 类型（Spring 官方将 String 归为简单类型）；
- Set 注入简单类型：通过 `<property name="属性名" value="简单类型值"/>`，将字符串形式的简单值注入到 Bean 的属性中，Spring 会自动完成“字符串→对应简单类型”的类型转换；
- 前提
 - 目标 Bean 的简单类型属性（比如 `name` / `age`）必须提供标准的 setter 方法；
 - 目标 Bean 必须有无参构造方法；
 - `value` 属性值是字符串形式，Spring 会自动转换为对应类型；
 - 特殊值（如空字符串、null）有专属写法（后续专题会讲，这里先讲常规值）。

我们之前在进行注入的时候，对象的属性是另一个对象。

- UserService.java
 - 对象的属性是另一个对象

```
public class UserService{

    private UserDao userDao;

    public void setUserDao(UserDao userDao){
        this.userDao = userDao;
    }

}
```

那如果对象的属性是int类型呢？

- User.java
 - 对象的属性是int类型

```

package org.cqipu.edu.bean;

public class User{
    private int age;
    public void setAge(int age){
        this.age = age;
    }
}

```

可以通过set注入的方式给该属性赋值吗？

- 当然可以。因为只要能够调用set方法就可以给属性赋值。

编写程序给一个User对象的age属性赋值20：

- 第一步：定义User类，提供age属性，提供age属性的setter方法。
 - User.java

```

package org.cqipu.edu.bean;
public class User {
    private int age;

    public void setAge(int age) {
        this.age = age;
    }
    @Override
    public String toString() {
        return "User{" +
            "age=" + age +
            '}';
    }
}

```

- 第二步：编写spring配置文件：spring-simple-type.xml
 - spring-simple-type.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="userBean" class="org.cqipu.edu.bean.User">
        <!-- 如果像这种int类型的属性，我们称为简单类型，这种简单类型在注入的时候要使用value属性，不能使用ref-->
        <!--<property name="age" value="20"/>-->
        <property name="age">
            <value>20</value>
        </property>
    </bean>
</beans>

```

- 第三步：编写测试程序
 - DITest.testSimpleType



```
import org.cqipu.edu.bean.User;

@Test
public void testSimpleType(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-simple-type.xml");
    User user = applicationContext.getBean("userBean", User.class);
    System.out.println(user);
}
```

第四步：运行测试：

✓ 测试 已通过: 1共 1 个测试 – 559毫秒

D:\Java21\bin\java.exe ...

执行 UserDaoImpl 的 insert 方法：插入用户数据

需要特别注意：如果给简单类型赋值，使用value属性或value标签。而不是ref。

- 简单类型包括哪些呢？可以通过Spring的源码来分析一下：BeanUtils类
 - BeanUtils.java



```
public class BeanUtils{

    //.....

    /**
     * Check if the given type represents a "simple" property: a simple value
     * type or an array of simple value types.
     * <p>See {@link #isSimpleValueType(Class)} for the definition of <em>simple
     * value type</em>.
     * <p>Used to determine properties to check for a "simple" dependency-check.
     * @param type the type to check
     * @return whether the given type represents a "simple" property
     * @see org.springframework.beans.factory.support.RootBeanDefinition#DEPENDENCY_CHECK_SIMPLE
     * @see org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory#checkDependencies
     * @see #isSimpleValueType(Class)
     */
    public static boolean isSimpleProperty(Class<?> type) {
        Assert.notNull(type, "'type' must not be null");
        return isSimpleValueType(type) || (type.isArray() && isSimpleValueType(type.getComponentType()));
    }

    /**
     * Check if the given type represents a "simple" value type: a primitive or
     * primitive wrapper, an enum, a String or other CharSequence, a Number, a
     * Date, a Temporal, a URI, a URL, a Locale, or a Class.
     * <p>{@code Void} and {@code void} are not considered simple value types.
     * @param type the type to check
     * @return whether the given type represents a "simple" value type
     * @see #isSimpleProperty(Class)
     */
    public static boolean isSimpleValueType(Class<?> type) {
        return (Void.class != type && void.class != type &&
            (ClassUtils.isPrimitiveOrWrapper(type) ||
            Enum.class.isAssignableFrom(type) ||
            CharSequence.class.isAssignableFrom(type) ||
            Number.class.isAssignableFrom(type) ||
            Date.class.isAssignableFrom(type) ||
```

```

        Temporal.class.isAssignableFrom(type) ||
        URI.class == type ||
        URL.class == type ||
        Locale.class == type ||
        Class.class == type));
    }

    //.....
}

```

通过源码分析得知，简单类型包括：

- 基本数据类型
- 基本数据类型对应的包装类
- String或其他的CharSequence子类
- Number子类
- Date子类
- Enum子类
- URIURITemporal子类
- Locale
- Class
- 另外还包括以上简单值类型对应的数组类型。

经典案例：给数据源的属性注入值：

假设我们现在要自己手写一个数据源，我们都知道所有的数据源都要实现javax.sql.DataSource接口，并且数据源中应该有连接数据库的信息，例如：driver、url、username、password等。

- MyDataSource.java

```

package org.cqipu.edu.bean;

import javax.sql.DataSource;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.SQLFeatureNotSupportedException;
import java.util.logging.Logger;

public class MyDataSource implements DataSource {
    private String driver;
    private String url;
    private String username;
    private String password;

    public void setDriver(String driver) {
        this.driver = driver;
    }

    public void setUrl(String url) {
        this.url = url;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}

```



```

}

@Override
public String toString() {
    return "MyDataSource{" +
        "driver='" + driver + '\'' +
        ", url='" + url + '\'' +
        ", username='" + username + '\'' +
        ", password='" + password + '\'' +
        '}';
}

@Override
public Connection getConnection() throws SQLException {
    return null;
}

@Override
public Connection getConnection(String username, String password) throws SQLException {
    return null;
}

@Override
public PrintWriter getLogWriter() throws SQLException {
    return null;
}

@Override
public void setLogWriter(PrintWriter out) throws SQLException {
}

@Override
public void setLoginTimeout(int seconds) throws SQLException {
}

@Override
public int getLoginTimeout() throws SQLException {
    return 0;
}

@Override
public Logger getParentLogger() throws SQLFeatureNotSupportedException {
    return null;
}

@Override
public <T> T unwrap(Class<T> iface) throws SQLException {
    return null;
}

@Override
public boolean isWrapperFor(Class<?> iface) throws SQLException {
    return false;
}
}

```

我们给driver、url、username、password四个属性分别提供了setter方法，我们可以使用spring的依赖注入完成数据源对象的创建和属性的赋值吗？看配置文件

- spring-datasource.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dataSource" class="org.cqipu.edu.bean.MyDataSource">
        <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/spring"/>
        <property name="username" value="root"/>
        <property name="password" value="123456"/>
    </bean>
</beans>
```

- 测试程序：
 - DITest.testDataSource



```
import org.cqipu.edu.bean.MyDataSource;

@Test
public void testDataSource(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-datasource.xml");
    MyDataSource dataSource = applicationContext.getBean("dataSource", MyDataSource.class);
    System.out.println(dataSource);
}
```

运行测试：

✓ 测试 已通过: 1共 1 个测试 - 530毫秒

```
D:\Java21\bin\java.exe ...
MyDataSource{driver='com.mysql.cj.jdbc.Driver', url='jdbc:mysql://localhost:3306/spring', username='root', password='123456'}
```

接下来，我们编写一个程序，把所有的简单类型全部测试一遍：

- 编写一个类A：
 - A.java



```
package org.cqipu.edu.bean;

import java.net.URI;
import java.net.URL;
import java.time.LocalDate;
import java.util.Date;
import java.util.Locale;

public class A {
    // 基本数据类型
    private byte b;
    private short s;
    private int i;
    private long l;
    private float f;
    private double d;
    private boolean flag;
    private char c;

    // 包装类型
    private Byte b1;
    private Short s1;
```

```
private Integer i1;
private Long l1;
private Float f1;
private Double d1;
private Boolean flag1;
private Character c1;

// 其他类型
private String str;
private Date date;
private Season season;
private URI uri;
private URL url;
private LocalDate localDate;
private Locale locale;
private Class clazz;

// ===== 无参构造函数 =====
public A() {
}

// ===== Setter 方法 =====
public void setB(byte b) {
    this.b = b;
}

public void setS(short s) {
    this.s = s;
}

public void setI(int i) {
    this.i = i;
}

public void setL(Long l) {
    this.l = l;
}

public void setF(float f) {
    this.f = f;
}

public void setD(double d) {
    this.d = d;
}

public void setFlag(boolean flag) {
    this.flag = flag;
}

public void setC(char c) {
    this.c = c;
}

public void setB1(Byte b1) {
    this.b1 = b1;
}

public void setS1(Short s1) {
    this.s1 = s1;
}

public void setI1(Integer i1) {
    this.i1 = i1;
}
```

```

public void setL1(Long l1) {
    this.l1 = l1;
}

public void setF1(Float f1) {
    this.f1 = f1;
}

public void setD1(Double d1) {
    this.d1 = d1;
}

public void setFlag1(Boolean flag1) {
    this.flag1 = flag1;
}

public void setC1(Character c1) {
    this.c1 = c1;
}

public void setStr(String str) {
    this.str = str;
}

public void setDate(Date date) {
    this.date = date;
}

public void setSeason(Season season) {
    this.season = season;
}

public void setUri(URI uri) {
    this.uri = uri;
}

public void setUrl(URL url) {
    this.url = url;
}

public void setLocalDate(LocalDate localDate) {
    this.localDate = localDate;
}

public void setLocale(Locale locale) {
    this.locale = locale;
}

public void setClazz(Class clazz) {
    this.clazz = clazz;
}

// ===== toString 方法 =====
@Override
public String toString() {
    return "A{" +
        "b=" + b +
        ", s=" + s +
        ", i=" + i +
        ", l=" + l +
        ", f=" + f +
        ", d=" + d +
        ", flag=" + flag +
        ", c=" + c +

```

```

        ", b1=" + b1 +
        ", s1=" + s1 +
        ", i1=" + i1 +
        ", l1=" + l1 +
        ", f1=" + f1 +
        ", d1=" + d1 +
        ", flag1=" + flag1 +
        ", c1=" + c1 +
        ", str='" + str + '\'' +
        ", date=" + date +
        ", season=" + season +
        ", uri=" + uri +
        ", url=" + url +
        ", localDate=" + localDate +
        ", locale=" + locale +
        ", clazz=" + clazz +
        '}'';
    }
}

// ===== 枚举类 =====
enum Season {
    SPRING, SUMMER, AUTUMN, WINTER
}

```

- spring-all-simple-type.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="a" class="org.cqipu.edu.bean.A">
        <property name="b" value="1"/>
        <property name="s" value="1"/>
        <property name="i" value="1"/>
        <property name="l" value="1"/>
        <property name="f" value="1"/>
        <property name="d" value="1"/>
        <property name="flag" value="false"/>

        <property name="c" value="a"/>
        <property name="b1" value="2"/>
        <property name="s1" value="2"/>
        <property name="i1" value="2"/>
        <property name="l1" value="2"/>
        <property name="f1" value="2"/>
        <property name="d1" value="2"/>
        <property name="flag1" value="true"/>
        <property name="c1" value="a"/>

        <property name="str" value="zhangsan"/>
        <!-- 注意: value后面的日期字符串格式不能随便写, 必须是Date对象toString()方法执行的结果。-->
        <!-- 如果想使用其他格式的日期字符串, 就需要进行特殊处理了。具体怎么处理, 可以看后面的课程!!! -->
        <property name="date" value="Fri Sep 30 15:26:38 CST 2022"/>
        <property name="season" value="WINTER"/>
        <property name="uri" value="/save.do"/>
        <!-- spring6之后, 会自动检查url是否有效, 如果无效会报错。-->
        <property name="url" value="http://www.baidu.com"/>
        <property name="localDate" value="EPOCH"/>
        <!-- java.util.Locale 主要在软件的本地化时使用。它本身没有什么功能, 更多的是作为一个参数辅助其他方法完成输出的本地化。-
-->
        <property name="locale" value="CHINESE"/>

```

```
<property name="clazz" value="java.lang.String"/>
</bean>
</beans>
```

- 编写测试程序：
 - `DITest.testAllSimpleType()`

```
import org.cqipu.edu.bean.A;

@Test
public void testAllSimpleType(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-all-simple-type.xml");
    A a = applicationContext.getBean("a", A.class);
    System.out.println(a);
}
```

测试结果：

✓ 测试已通过: 1共 1 个测试 - 467毫秒

```
D:\Java21\bin\java.exe ...
A{b=1, s=1, i=1, l=1, f=1.0, d=1.0, flag=false, c=a, b1=2, s1=2, i1=2, l1=2, f1=2.0, d1=2.0, flag1=true, c1=a, str='zhangsan',
date=Sat Oct 01 05:26:38 CST 2022, season=WINTER, uri=/save.do, url=http://www.baidu.com, localDate=1970-01-01, locale=chinese,
clazz=class java.lang.String}
```

需要注意的是：

- 如果把Date当做简单类型的话，日期字符串格式不能随便写。格式必须符合Date的toString()方法格式。显然这就比较鸡肋了。如果我们提供一个这样的日期字符串：2010-10-11，在这里是无法赋值给Date类型的属性的。
- spring6之后，当注入的是URL，那么这个url字符串是会进行有效性检测的。如果是一个存在的url，那就没问题。如果不存在则报错。

级联属性赋值（了解）

这是 Set 注入的“进阶写法”，用于给 Bean 的“属性的属性”赋值，但实际开发中用得少，仅作了解，核心是掌握“为什么需要 getter”这个关键前提。

- 级联属性：指 Bean 的“属性嵌套属性”
 - 比如User有Address类型的address属性，Address又有province/city属性，那么address.province、address.city就是级联属性；
- 级联属性赋值：通过 `<property name="级联属性名" value="值"/>`，直接给 Bean 的嵌套属性赋值，无需在内部 Bean 里逐个配置属性；

先给外层属性（address）注入对象，再通过“外层属性.内层属性”的格式给内层属性赋值，本质是“先获取嵌套对象，再调用其 setter 方法”。

- 前提
 - 目标 Bean 的外层嵌套属性必须先完成注入。不管是注入外部 Bean、内部 Bean，必须先有这个对象，否则级联赋值会报空指针；
 - 外层嵌套属性的对象必须为该属性提供getter方法。Spring 需要通过 getter 获取嵌套对象，才能给其内层属性赋值；
 - 内层属性必须有setter方法，用于最终赋值；
 - 目标 Bean 仍需无参构造。
- Clazz.java

```
package org.cqipu.edu.bean;

public class Clazz {
```

```

private String name;

publicClazz() {
}

publicClazz(String name) {
    this.name = name;
}

publicString getName() {
    return name;
}

publicvoid setName(String name) {
    this.name = name;
}

@Override
publicString toString() {
    return "Clazz{" +
        "name='" + name + '\'' +
        '}';
}
}

```

- Student.java

```

package org.cqipu.edu.bean;
public class Student {
    privateString name;
    privateClazz clazz;

    publicStudent() {
    }

    publicStudent(String name, Clazz clazz) {
        this.name = name;
        this.clazz = clazz;
    }

    publicvoid setName(String name) {
        this.name = name;
    }

    publicvoid setClazz(Clazz clazz) {
        this.clazz = clazz;
    }

    publicClazz getClazz() {
        return clazz;
    }

    @Override
    publicString toString() {
        return "Student{" +
            "name='" + name + '\'' +
            ", clazz=" + clazz +
            '}';
    }
}

```

- spring-cascade.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="clazzBean" class="org.cqipu.edu.bean.Clazz"/>

    <bean id="student" class="org.cqipu.edu.bean.Student">
        <property name="name" value="张三"/>

        <!-- 要点1: 以下两行配置的顺序不能颠倒-->
        <property name="clazz" ref="clazzBean"/>
        <!-- 要点2: clazz属性必须有getter方法-->
        <property name="clazz.name" value="高三一班"/>
    </bean>
</beans>
```

- 测试程序



```
import org.cqipu.edu.bean.Student;
@Test
public void testCascade(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-cascade.xml");
    Student student = applicationContext.getBean("student", Student.class);
    System.out.println(student);
}
```

测试结果：

✓ 测试 已通过: 1共 1 个测试 – 522毫秒

D:\Java21\bin\java.exe ...

Student{name='张三', clazz=Clazz{name='高三一班'}}

- 要点：
 - 在spring配置文件中，如上，注意顺序。
 - 在spring配置文件中，clazz属性必须提供getter方法。

注入数组

数组注入用 `<array>` 标签，简单类型数组配 `<value>`，对象类型数组配 `<ref>`，核心是“标签嵌套 + 元素匹配”，且数组是“同类型元素的有序集合”，Spring 会严格保证元素类型和顺序。

- 数组类型：指 Java 的一维数组，Spring 不支持直接注入多维数组，需拆分为一维数组嵌套，分为两类：
 - 简单类型数组（如 `String[]` / `int[]`）；
 - 对象类型数组（如 `User[]` / `UserDao[]`）；
- Set 注入数组类型：通过 `<property name="数组属性名"><array>...</array></property>` 标签嵌套，给数组属性赋值，简单类型数组用 `<value>` 填元素，对象类型数组用 `<ref>` 引用外部 Bean；

`<array>` 是数组注入的专属标签，元素必须和数组类型一致，顺序和配置顺序一致。

- 前提
 - 目标 Bean 的数组属性必须提供标准的 setter 方法）；

- 数组为对象类型时，数组元素对应的 Bean 必须由 Spring 容器管理配置 `<bean>` ；
- 目标 Bean 必须有无参构造方法；
- 数组元素类型必须和数组属性类型一致。

当数组中的元素是简单类型：

- Person.java

```
package org.cqipu.edu.bean;
import java.util.Arrays;
public class Person {
    private String[] favariteFoods;

    public void setFavariteFoods(String[] favariteFoods) {
        this.favariteFoods = favariteFoods;
    }

    @Override
    public String toString() {
        return "Person{" +
            "favariteFoods=" + Arrays.toString(favariteFoods) +
            '}';
    }
}
```

- spring-array-simple.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="person" class="org.cqipu.edu.bean.Person">
        <property name="favariteFoods">
            <array>
                <value>鸡排</value>
                <value>汉堡</value>
                <value>鹅肝</value>
            </array>
        </property>
    </bean>
</beans>
```

- DITest.testArraySimple()

```
@Test
public void testArraySimple(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-array-simple.xml");
    Person person = applicationContext.getBean("person", Person.class);
    System.out.println(person);
}
```

测试结果：

```
D:\Java21\bin\java.exe ...
Person{favariteFoods=[鸡排, 汉堡, 鹅肝]}
```

当数组中的元素是非简单类型：一个订单中包含多个商品。

- Goods.java

```
package org.cqipu.edu.bean;
public class Goods {
    private String name;

    public Goods() {
    }

    public Goods(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Goods{" +
            "name='" + name + '\'' +
            '}';
    }
}
```

- Order.java

```
package org.cqipu.edu.bean;
import java.util.Arrays;

public class Order {
    // 一个订单中有多个商品
    private Goods[] goods;

    public Order() {
    }

    public Order(Goods[] goods) {
        this.goods = goods;
    }

    public void setGoods(Goods[] goods) {
        this.goods = goods;
    }

    @Override
    public String toString() {
        return "Order{" +
            "goods=" + Arrays.toString(goods) +
            '}';
    }
}
```

- spring-array.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="goods1" class="org.cqipu.edu.bean.Goods">
        <property name="name" value="西瓜"/>
    </bean>

    <bean id="goods2" class="org.cqipu.edu.bean.Goods">
        <property name="name" value="苹果"/>
    </bean>

    <bean id="order" class="org.cqipu.edu.bean.Order">
        <property name="goods">
            <array>
                <!-- 这里使用ref标签即可-->
                <ref bean="goods1"/>
                <ref bean="goods2"/>
            </array>
        </property>
    </bean>
</beans>
```

- 测试程序

```
import org.cqipu.edu.bean.Order;
@Test
public void testArray(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-array.xml");
    Order order = applicationContext.getBean("order", Order.class);
    System.out.println(order);
}
```

测试结果：

✓ 测试 已通过: 1共 1 个测试 – 456毫秒

D:\Java21\bin\java.exe ...

Order{goods=[Goods{name='西瓜'}, Goods{name='苹果'}]}

- 要点：
 - 如果数组中是简单类型，使用value标签。
 - 如果数组中是非简单类型，使用ref标签。

注入List集合

List 和数组都是“有序集合”，但 List 支持动态扩容，无需固定长度，注入标签从 `<array>` 换成 `<list>`，核心逻辑和数组一致，只是标签不同，这是最关键的區別。

- 定义
 - List 集合：Java 中最常用的有序集合，可重复元素、动态扩容，分为两类：
 - 简单类型 List: `List<String>/List<Integer>`；
 - 对象类型 List: `List<User>/List<UserDao>`；

- Set 注入 List 集合：通过 `<property name="List属性名"><list>...</list></property>` 标签嵌套赋值，简单类型 List 用 `<value>` 填元素，对象类型 List 用 `<ref>` 引用外部 Bean；

`<list>` 是 List 注入的专属标签，List 无需指定长度，这是和数组 `<array>` 的核心区别。

特性	数组	List 集合
长度	固定长度	动态扩容
核心标签	<code><array></code>	<code><list></code>
适用场景	元素数量固定的场景	元素数量动态变化的场景
类型限制	只能是一维数组	支持泛型

- 前提
 - 目标 Bean 的 List 属性必须提供标准的 setter 方法；
 - 对象类型 List 的元素 Bean 必须由 Spring 容器管理；
 - 目标 Bean 必须有无参构造方法；
 - List 泛型类型需和 `<value>` / `<ref>` 元素类型一致。

List集合：有序可重复

- People.java

```
package org.cqipu.edu.bean;

import java.util.List;
public class People {
    // 一个人有多个名字
    private List<String> names;

    public void setNames(List<String> names) {
        this.names = names;
    }

    @Override
    public String toString() {
        return "People{" +
            "names=" + names +
            '}';
    }
}
```

- spring-collection.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="peopleBean" class="org.cqipu.edu.bean.People">
        <property name="names">
            <list>
                <value>铁锤</value>
                <value>张三</value>
                <value>张三</value>
                <value>张三</value>
            </list>
        </property>
    </bean>
</beans>
```

```
        <value>狼</value>
    </list>
</property>
</bean>
</beans>
```

- 测试程序

```
@Test
public void testCollection(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-collection.xml");
    People peopleBean = applicationContext.getBean("peopleBean", People.class);
    System.out.println(peopleBean);
}
```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 531毫秒

D:\Java21\bin\java.exe ...

People{names=[铁锤, 张三, 张三, 张三, 狼]}

注意：注入List集合的时候使用list标签，如果List集合中是简单类型使用value标签，反之使用ref标签。

注入Set集合

Set 和 List 最大的区别是“无序、不可重复”，注入标签从 `<list>` 换成 `<set>`，核心配置逻辑一致，但 Spring 会自动去重并打乱顺序（底层默认用 HashSet），这是 Set 注入最核心的特性。

- 定义
 - Set 集合：Java 中无序、不可重复的集合，底层常用 HashSet，基于哈希表实现，分为两类：
 - 简单类型 Set（如 `Set<String>` / `Set<Integer>`）；
 - 对象类型 Set（如 `Set<User>` / `Set<UserDao>`）；
 - Set 注入 Set 集合：通过 `<property name="Set属性名"><set>...</set></property>` 标签嵌套赋值，简单类型 Set 用 `<value>` 填元素，对象类型 Set 用 `<ref>` 引用外部 Bean；

`<set>` 是 Set 注入的专属标签，Spring 会自动对元素去重，重复元素只保留一个，且最终顺序和配置顺序无关。

特性	Set 集合	List 集合
有序性	无序（配置顺序≠最终顺序）	有序（配置顺序 = 最终顺序）
重复性	不可重复	可重复
核心标签	<code><set></code>	<code><list></code>
底层实现	默认 HashSet	默认 ArrayList
适用场景	需去重、无需顺序的场景	需保留顺序、允许重复的场景

- 前提
 - 目标 Bean 的 Set 属性必须提供标准的 setter 方法；
 - 对象类型 Set 的元素 Bean 必须由 Spring 容器管理；
 - 目标 Bean 必须有无参构造方法；

- o Set 泛型类型需和 `<value>` / `<ref>` 元素类型一致;
- o 对象类型 Set 去重规则: 基于对象的 `equals()` 和 `hashCode()` 方法, 需重写这两个方法, 否则无法去重。

Set集合: 无序不可重复

- People.java

```
package org.cqipu.edu.bean;
import java.util.Set;
public class People {
    // 一个人有多个电话
    private Set<String> phones;

    // 一个人的多个名字(别名)
    private Set<String> names;

    // ===== 无参构造函数 =====
    public People() {
    }

    // ===== Setter 方法 =====
    public void setPhones(Set<String> phones) {
        this.phones = phones;
    }

    public void setNames(Set<String> names) {
        this.names = names;
    }

    // ===== Getter 方法 =====
    public Set<String> getPhones() {
        return phones;
    }

    public Set<String> getNames() {
        return names;
    }

    // ===== toString 方法 =====
    @Override
    public String toString() {
        return "People{" +
            "phones=" + phones +
            ", names=" + names +
            '}';
    }
}
```

- spring-collection.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="peopleBean" class="org.cqipu.edu.bean.People">
        <!-- 注入 List 集合 - names (允许重复) -->
        <property name="names">
            <list>
                <value>铁锤</value>
                <value>张三</value>
            </list>
        </property>
    </bean>
</beans>
```

```

        <value>张三</value>
        <value>张三</value>
        <value>狼</value>
    </list>
</property>
<!-- 注入 Set 集合 - phones (自动去重) -->
<property name="phones">
    <set>
        <value>110</value>
        <value>110</value>
        <value>120</value>
        <value>120</value>
        <value>119</value>
        <value>119</value>
    </set>
</property>
</bean>
</beans>

```

- 测试程序

```

● ● ●
@Test
public void testCollection(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-collection.xml");
    People peopleBean = applicationContext.getBean("peopleBean", People.class);
    System.out.println(peopleBean);
}

```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 426毫秒

D:\Java21\bin\java.exe ...

People{phones=[110, 120, 119], names=[铁锤, 张三, 狼]}

- 要点:
 - 使用 `<set>` 标签
 - `set` 集合中元素是简单类型的使用 `value` 标签，反之使用 `ref` 标签。

注入Map集合

Map 是“键值对 (Key-Value)”集合，键唯一、无序，值可重复，注入核心标签是 `<map>` + `<entry>`，简单类型键值用 `key` / `value`，对象类型用

`key-ref` / `value-ref`，这是 Map 和 List/Set 最核心的区别。

- 定义
 - Map 集合：Java 中以“键值对”存储数据的集合（底层常用 HashMap），核心特性：
 - 键 (Key) 唯一，重复键会覆盖原有值；
 - 无序，键值对顺序和配置无关；
 - 值 (Value) 可重复；
 - Set 注入 Map 集合：通过 `<property name="Map属性名"><map><entry>...</entry></map></property>` 赋值
 - 简单类型键值：`<entry key="键" value="值"/>`；
 - 对象类型键 / 值：`<entry key-ref="键Bean的id" value-ref="值Bean的id"/>`；

`<map>` 是 Map 专属标签，`<entry>` 包裹单个键值对，键唯一是 Map 的核心约束。

特性	Map 集合	List 集合	Set 集合
存储形式	键值对 (Key-Value)	单一元素	单一元素
核心约束	键唯一、无序，值可重复	有序、元素可重复	无序、元素不可重复
核心标签	<code><map></code> + <code><entry></code>	<code><list></code>	<code><set></code>
适用场景	键值映射数据（如昵称 - 用户、类型 - 内容）	有序列表数据（如订单）	去重无序列表（如标签）

- 前提
 - 目标 Bean 的 Map 属性必须提供标准的 setter 方法；
 - Map 为对象类型时（键 / 值是对象），对应的 Bean 必须由 Spring 容器管理；
 - 目标 Bean 必须有无参构造方法；
 - Map 的键必须唯一；
 - 键 / 值类型需和 Map 泛型一致。
- People.java

```
package org.cqipu.edu.bean;
import java.util.Set;
import java.util.Map;
public class People {
    // 一个人有多个电话
    private Set<String> phones;

    // 一个人的多个名字（别名）
    private Set<String> names;

    // 一个人有多个住址
    private Map<Integer, String> addrs;

    // ===== 无参构造函数 =====
    public People() {
    }

    // ===== Setter 方法 =====
    public void setPhones(Set<String> phones) {
        this.phones = phones;
    }

    public void setNames(Set<String> names) {
        this.names = names;
    }

    public void setAddrs(Map<Integer, String> addrs) {
        this.addrs = addrs;
    }

    // ===== Getter 方法 =====
    public Set<String> getPhones() {
        return phones;
    }

    public Set<String> getNames() {
        return names;
    }
}
```



```
// ===== toString 方法 =====
@Override
public String toString() {
    return "People{" +
        "addr=" + addr +
        ", phones=" + phones +
        ", names=" + names +
        '}';
}
}
```

- spring-collection.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="peopleBean" class="org.cqipu.edu.bean.People">
        <!-- 注入 List 集合 - names (允许重复) -->
        <property name="names">
            <list>
                <value>铁锤</value>
                <value>张三</value>
                <value>张三</value>
                <value>张三</value>
                <value>狼</value>
            </list>
        </property>
        <!-- 注入 Set 集合 - phones (自动去重) -->
        <property name="phones">
            <set>
                <value>110</value>
                <value>110</value>
                <value>120</value>
                <value>120</value>
                <value>119</value>
                <value>119</value>
            </set>
        </property>

        <property name="addr">
            <map>
                <!-- 如果key不是简单类型, 使用 key-ref 属性-->
                <!-- 如果value不是简单类型, 使用 value-ref 属性-->
                <entry key="1" value="北京大兴区"/>
                <entry key="2" value="上海浦东区"/>
                <entry key="3" value="深圳宝安区"/>
            </map>
        </property>
    </bean>
</beans>
```

- 测试程序

```
@Test
public void testCollection(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-collection.xml");
    People peopleBean = applicationContext.getBean("peopleBean", People.class);
    System.out.println(peopleBean);
}
```

✓ 测试 已通过: 1 共 1 个测试 - 494毫秒

```
D:\Java21\bin\java.exe ...
People{addrs={1=北京大兴区, 2=上海浦东区, 3=深圳宝安区}, phones=[110, 120, 119], names=[铁锤, 张三, 狼]}
```

- 要点：
 - 使用 `<map>` 标签
 - 如果 key 是简单类型，使用 key 属性，反之使用 key-ref 属性。
 - 如果 value 是简单类型，使用 value 属性，反之使用 value-ref 属性。

注入 Properties

Properties 是“特殊的 Map”；键值全为 String 类型，底层继承 Hashtable，专门用于存储配置类数据，如数据库连接信息、系统参数，注入核心标签是 `<props>` + `<prop key="键">值</prop>`，而非 Map 的 `<map>` + `<entry>`，这是 Properties 和普通 Map 最核心的区别。

- 定义
 - Properties：Java 中专门用于存储配置项的特殊集合（`java.util.Properties`），核心特性：
 - 键（Key）和值（Value）必须都是 String 类型；
 - 键唯一；
 - 底层基于 Hashtable，无序；
 - Set 注入 Properties：通过 `<property name="Properties属性名"><props><prop key="键">值</prop></props></property>` 赋值；
 - 核心标签：`<props>` 是 Properties 专属标签，`<prop key="键">值</prop>` 包裹单个配置项；
 - 核心约束：键值只能是 String，无需手动转换；
 - 核心场景：数据库连接信息（url、username、password）、系统配置（编码、超时时间）等。

特性	Properties	Map<String, String>
类型约束	键值强制为 String	键值声明为 String
核心标签	<code><props></code> + <code><prop></code>	<code><map></code> + <code><entry></code>
特有方法	支持加载配置文件（load ()）	无配置文件加载方法
适用场景	纯配置项存储（如数据库连接）	通用键值对存储
底层实现	继承 Hashtable（线程安全）	常用 HashMap（非线程安全）

- 前提
 - 目标 Bean 的 Properties 属性必须提供标准的 setter 方法；
 - 目标 Bean 必须有无参构造方法；
 - `<prop>` 的键值必须是 String 类型；
 - 键唯一。

java.util.Properties继承java.util.Hashtable，所以Properties也是一个Map集合。

- People.java

```
package org.cqipu.edu.bean;
import java.util.Set;
import java.util.Properties;
```

```

import java.util.Map;
public class People {
    // 一个人有多个电话
    private Set<String> phones;
    // 一个人的多个名字（别名）
    private Set<String> names;
    // 一个人有多个住址
    private Map<Integer, String> addr;
    private Properties properties;

    // ===== 无参构造函数 =====
    public People() {
    }

    // ===== Setter 方法 =====
    public void setPhones(Set<String> phones) {
        this.phones = phones;
    }

    public void setNames(Set<String> names) {
        this.names = names;
    }

    public void setAddr(Map<Integer, String> addr) {
        this.addr = addr;
    }

    public void setProperties(Properties properties) {
        this.properties = properties;
    }

    // ===== Getter 方法 =====
    public Set<String> getPhones() {
        return phones;
    }

    public Set<String> getNames() {
        return names;
    }

    // ===== toString 方法 =====
    @Override
    public String toString() {
        return "People{" +
            "properties=" + properties +
            ", addr=" + addr +
            ", phones=" + phones +
            ", names=" + names +
            '}';
    }
}

```

- spring-collection01.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="peopleBean" class="org.cqipu.edu.bean.People">
        <property name="properties">
            <props>
                <prop key="driver">com.mysql.cj.jdbc.Driver</prop>
            </props>
        </property>
    </bean>

```

```
<prop key="url">jdbc:mysql://localhost:3306/spring</prop>
<prop key="username">root</prop>
<prop key="password">123456</prop>
</props>
</property>
</bean>
</beans>
```

运行测试：

```
@Test
public void testCollection02(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-collection01.xml");
    People peopleBean = applicationContext.getBean("peopleBean", People.class);
    System.out.println(peopleBean);
}
```

```
✔ 测试 已通过: 1共 1 个测试 - 436毫秒
D:\Java21\bin\java.exe ...
People{properties={password=123456, driver=com.mysql.cj.jdbc.Driver, url=jdbc:mysql://localhost:3306/spring, username=root},
  addr=null, phones=null, names=null}
```

要点：

- 使用 `<props>` 标签嵌套 `<prop>` 标签完成。

注入null和空字符串

- null 和空字符串 ("") 是完全不同的概念，Spring 中注入写法也完全不同：
 - 空字符串用 `value=""` ，
 - null 值必须用 `<null/>` 标签，
- 绝对不能把 `value="null"` 当成 null 注入

概念	本质含义	Spring 注入写法	业务场景举例
空字符串 ("")	字符串对象，但长度为 0	<code><property name="属性名" value=""/></code>	用户名为空、备注为空文本
null 值	没有任何对象（属性未赋值）	<code><property name="属性名"><null/></property></code>	未填写的邮箱、未设置的昵称
易错写法	字符串 "null"（非空非 null）	<code><property name="属性名" value="null"/></code>	无合理业务场景，纯新手错误

- 前提
 - 目标 Bean 的字符串属性必须提供标准的 setter 方法；
 - 目标 Bean 必须有无参构造方法；
 - 注入 null 时， `<null/>` 必须作为 `<property>` 的子标签；
 - 注入空字符串时， `value=""` 直接写空即可。
- 实操
 - 注入空字符串使用： `<value/>` 或者 `value=""`
 - 注入null使用： `<null/>` 或者 不为该属性赋值
 - 我们先来看一下，怎么注入空字符串。
 - Vip.java



```
package org.cqipu.edu.bean;

public class Vip {
    private String email;

    public void setEmail(String email) {
        this.email = email;
    }

    @Override
    public String toString() {
        return "Vip{" +
            "email='" + email + '\'' +
            '}';
    }
}
```

- spring-null.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="vipBean" class="org.cqipu.edu.bean.Vip">
        <!-- 空串的第一种方式-->
        <!--<property name="email" value=""/>-->
        <!-- 空串的第二种方式-->
        <property name="email">
            <value/>
        </property>
    </bean>
</beans>
```

- 测试程序



```
@Test
public void testNull(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-null.xml");
    Vip vipBean = applicationContext.getBean("vipBean", Vip.class);
    System.out.println(vipBean);
}
```

测试结果：

✓ 测试 已通过: 1共 1 个测试 - 427毫秒

```
D:\Java21\bin\java.exe ...
Vip{email=''}

```

- 怎么注入null呢？

第一种方式：不给属性赋值

- spring-null.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="vipBean" class="org.cqipu.edu.bean.Vip" />
</beans>

```

测试结果：

```

@Test
public void testNull(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-null.xml");
    Vip vipBean = applicationContext.getBean("vipBean", Vip.class);
    System.out.println(vipBean);
}

```

✓ 测试 已通过: 1共 1 个测试 - 419毫秒

D:\Java21\bin\java.exe ...

Vip{email='null'}

第二种方式：使用 `<null/>`

- spring-null.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="vipBean" class="org.cqipu.edu.bean.Vip">
        <property name="email">
            <null/>
        </property>
    </bean>
</beans>

```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 430毫秒

D:\Java21\bin\java.exe ...

Vip{email='null'}

注入的值中含有特殊符号

- XML 有 5 个保留特殊符号 (`<` `>` `&` `'` `"`)，直接写在 `value` 里会导致 XML 文件解析失败，Spring 提供两种解决方案：
 - CDATA 包裹（推荐）
 - XML 实体引用

- 其中 CDATA 是最简洁、最常用的方式。

XML 解析器会把以下 5 个字符当成“语法符号”，而非普通文本，直接写在value中会报错：

特殊符号	含义	XML 实体引用
<	标签开始	<
>	标签结束	>
&	实体引用开始	&
'	单引号	'
"	双引号	"

处理方式	写法格式	优点	缺点
CDATA 包裹	<![CDATA[包含特殊符号的值]]>	简单，无需记忆实体符号，支持所有特殊符号	写法稍长，不能嵌套 CDATA
XML 实体引用	用 < 代替 < 等	写法紧凑	需记忆实体符号，易记错

- 前提
 - 目标 Bean 的字符串属性必须提供标准的 setter 方法；
 - 目标 Bean 必须有无参构造方法；
 - 仅字符串类型的值需要处理特殊符号；
 - CDATA 仅包裹“值内容”，不能包裹 key、标签名等 XML 语法部分。
- XML中有5个特殊字符，分别是：<、>、'、"、&
- 以上5个特殊符号在XML中会被特殊对待，会被当做XML语法的一部分进行解析，如果这些特殊符号直接出现在注入的字符串当中，会报错。

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans http://www
       <bean id="mathBean" class="com.powernode.spring6.beans.Math">
         <property name="result" value="2 < 3"/>
       </bean>
</beans>
```

The value of attribute "value" associated with an element type "property" must not contain the '<' character.

Tag name: **property**
Description : Bean definitions can have zero or more properties. Property elements correspond to JavaBean setter methods exposed by the bean classes. Spring supports primitives, references to other beans in the same or related factories, lists, maps and properties.
Maven: org.springframework:spring-beans:6.0.0-M2 (spring-beans-6.0.0-M2.jar)

老杜

- 先使用转义字符来代替：
 - Math.java

```
package org.cqipu.edu.bean;

public class Math {
    private String result;
```

```

    public void setResult(String result) {
        this.result = result;
    }

    @Override
    public String toString() {
        return "Math{" +
            "result='" + result + '\'' +
            '}';
    }
}

```

- spring-special.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="mathBean" class="org.cqipu.edu.bean.Math">
        <property name="result" value="2 < 3"/>
    </bean>
</beans>

```

- 测试程序

```

@Test
public void testSpecial(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-special.xml");
    Math mathBean = applicationContext.getBean("mathBean", Math.class);
    System.out.println(mathBean);
}

```

✓ 测试 已通过: 1共 1 个测试 - 487毫秒

```

D:\Java21\bin\java.exe ...
Math{result='2 < 3'}

```

- 我们再来使用CDATA方式：
 - spring-special.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="mathBean" class="org.cqipu.edu.bean.Math">
        <property name="result">
            <!-- 只能使用value标签-->
            <value><![CDATA[2 < 3]]></value>
        </property>
    </bean>
</beans>

```

注意：使用CDATA时，不能使用value属性，只能使用value标签。

执行结果：

✓ 测试 已通过: 1共 1 个测试 - 487毫秒

```
D:\Java21\bin\java.exe ...
Math{result='2 < 3'}
```

p命名空间注入

p 命名空间本质还是 Set 注入，底层仍调用 setter 方法，只是把嵌套的 `<property>` 标签简化为 XML 属性，让配置更简洁；但它仅支持简单类型和外部 Bean 注入，复杂类型（数组 / 集合 / null / 特殊符号）仍需用传统 Set 注入，这是核心适用边界。

对比维度	传统 Set 注入写法	p 命名空间注入写法	核心区别
语法形式	嵌套 <code><property></code> 标签	XML 属性直接赋值	简化标签层级，配置更紧凑
简单类型注入	<code><property name="name" value="张三"/></code>	<code>p:name="张三"</code>	去掉 <code><property></code> ，直接写属性
对象类型注入	<code><property name="addr" ref="address"/></code>	<code>p:addr-ref="address"</code>	对象类型加 <code>-ref</code> 后缀
底层逻辑	调用 setter 方法	完全相同	语法简化，逻辑不变
适用范围	所有 Set 注入场景	仅简单类型 + 外部 Bean（不支持复杂类型）	有明确适用边界

- 前提
 - 必须声明 p 命名空间：在 XML 的 `<beans>` 根标签中添加 `xmlns:p="http://www.springframework.org/schema/p"`，否则 XML 解析报错；
 - 目标 Bean 的属性必须有标准 setter 方法；
 - 目标 Bean 必须有无参构造方法；
 - p 命名空间不支持：数组、List/Set/Map/Properties、null 值、CDATA 特殊符号、级联属性 —— 这些场景仍需用传统 Set 注入。
- Customer.java

```
package org.cqipu.edu.bean;
public class Customer {
    private String name;
    private int age;

    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "Customer{" +
            "name='" + name + '\'' +
            ", age=" + age +
            '}';
    }
}
```

```
}
```

- spring-p.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:p="http://www.springframework.org/schema/p"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="customerBean" class="org.cqipu.edu.bean.Customer" p:name="zhangsan" p:age="20"/>
</beans>
```

- 测试程序

```
@Test
public void testP(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-p.xml");
    Customer customerBean = applicationContext.getBean("customerBean", Customer.class);
    System.out.println(customerBean);
}
```

✓ 测试 已通过: 1共 1 个测试 - 444毫秒

```
D:\Java21\bin\java.exe ...
Customer{name='zhangsan', age=20}
```

把setter 方法注释掉

```
D:\Java21\bin\java.exe ...

org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'customerBean' defined in class path resource
[spring-p.xml]: Invalid property 'age' of bean class [org.cqipu.edu.bean.Customer]: Bean property 'age' is not writable or has an
invalid setter method. Does the parameter type of the setter match the return type of the getter?
```

使用p 命名空间实际上是对set注入的简化

c 命名空间注入

c 命名空间和 p 命名空间是“姐妹语法”，但核心区别是：p 命名空间依赖 setter 方法，c 命名空间依赖构造方法；c 命名空间把嵌套的 `<constructor-arg>` 标签简化为 XML 属性，仅支持简单类型和外部 Bean 注入，复杂类型仍需用传统构造器注入。

对比维度	传统构造器注入写法	c 命名空间注入写法	核心区别
底层逻辑	调用构造方法	完全相同	语法简化，逻辑不变
语法形式	嵌套 <code><constructor-arg></code> 标签	XML 属性直接赋值	去掉 <code><constructor-arg></code> ，更紧凑
简单类型（按参数名）	<code><constructor-arg name="name" value="张三"/></code>	<code>c:name="张三"</code>	按参数名匹配构造方法

对比维度	传统构造器注入写法	c 命名空间注入写法	核心区别
简单类型（按索引）	<code><constructor-arg index="0" value="张三"/></code>	<code>c:_0="张三"</code>	按参数索引（从 0 开始）匹配
对象类型（按参数名）	<code><constructor-arg name="addr" ref="address"/></code>	<code>c:addr-ref="address"</code>	对象类型加 <code>-ref</code> 后缀
适用范围	所有构造器注入场景	仅简单类型 + 外部 Bean（不支持复杂类型）	有明确适用边界

- 前提
 - 必须声明 c 命名空间：在 XML 的 `<beans>` 根标签中添加 `xmlns:c="http://www.springframework.org/schema/c"`，否则 XML 解析报错；
 - 目标 Bean 必须有对应的构造方法；
 - 无需 setter 方法；
 - c 命名空间不支持：数组、List/Set/Map/Properties、null 值、CDATA 特殊符号 —— 这些场景仍需用传统构造器注入。
- MyTime.java

```
package org.cqipu.edu.bean;

public class MyTime {
    private int year;
    private int month;
    private int day;

    public MyTime(int year, int month, int day) {
        this.year = year;
        this.month = month;
        this.day = day;
    }

    @Override
    public String toString() {
        return "MyTime{" +
            "year=" + year +
            ", month=" + month +
            ", day=" + day +
            '}';
    }
}
```

- spring-c.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:c="http://www.springframework.org/schema/c"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!--<bean id="myTimeBean" class="com.powernode.spring6.beans.MyTime" c:year="1970" c:month="1" c:day="1"/>-->

    <bean id="myTimeBean" class="org.cqipu.edu.bean.MyTime" c:_0="2008" c:_1="8" c:_2="8"/>

</beans>
```

测试程序

```
● ● ●
@Test
public void testC(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-c.xml");
    MyTime myTimeBean = applicationContext.getBean("myTimeBean", MyTime.class);
    System.out.println(myTimeBean);
}
```

执行结果：

✓ 测试 已通过: 1共 1 个测试 - 438毫秒

D:\Java21\bin\java.exe ...

MyTime{year=2008, month=8, day=8}

把构造方法注释去掉：

❗ 测试 失败: 1共 1 个测试 - 418毫秒

D:\Java21\bin\java.exe ...

org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'myTimeBean' defined in class path resource [spring-c.xml]: Could not resolve matching constructor on bean class [org.cqipu.edu.bean.MyTime] (hint: specify index/type/name arguments for simple parameters to avoid type ambiguities. You should also check the consistency of arguments when mixing indexed and named arguments, especially in case of bean definition inheritance)

所以，c命名空间是依靠构造方法的。

注意： 不管是p命名空间还是c命名空间，注入的时候都可以注入简单类型以及非简单类型。

util 命名空间

util 命名空间是 Spring 针对集合注入的“进阶优化”，之前的集合注入都是“内联式”，写在某个 bean 内部，只能该 bean 使用，而 util 命名空间能把集合抽离成“独立的、可复用的 bean”，多个 bean 可共享同一个集合；同时 util 命名空间还简化了集合的配置语法，是企业开发中复用集合的首选方案。

对比维度	传统内联集合注入	util 命名空间集合注入	核心区别
集合定义位置	某个 bean 的 <property> 内部	独立的util:xxx标签（全局 bean）	内联不可复用 vs 全局可复用
复用性	仅当前 bean 可用	多个 bean 通过 ref 引用	解决集合重复定义问题
配置语法	嵌套层级多（bean→property→list）	扁平化（直接定义 util:list）	简化配置结构
核心用途	单个 bean 的专属集合	多个 bean 共享的通用集合	适配“全局通用配置”场景

- 前提
 - 必须声明 util 命名空间 + 引入 xsd schema：util 是 Spring 的扩展命名空间，需在 <beans> 中同时声明命名空间和对应的 xsd 地址（缺一不可），否则 XML 解析报错；
 - 目标 Bean 的集合属性必须有标准 setter 方法；
 - util 命名空间核心聚焦集合复用：仅对 List/Set/Map/Properties 生效，简单类型 / 对象类型无需用 util；
 - 抽离的集合 bean 需指定唯一 id：其他 bean 通过 ref 引用该 id，实现复用。
- util 命名空间完整声明

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:util="http://www.springframework.org/schema/util" <!-- 声明util命名空间 -->
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/util <!-- util的xsd地址 -->
                           https://www.springframework.org/schema/util/spring-util.xsd"> <!-- util的xsd文件 -->

```

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:util="http://www.springframework.org/schema/util"
       xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/util http://www.springframework.org/schema/util/spring-util.xsd"

```

老杜

- spring-util.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:util="http://www.springframework.org/schema/util"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/util
                           http://www.springframework.org/schema/util/spring-util.xsd">

    <bean id="dataSource1" class="org.cqipu.edu.bean.MyDataSource1">
        <property name="properties">
            <props>
                <prop key="driver">com.mysql.cj.jdbc.Driver</prop>
                <prop key="url">jdbc:mysql://localhost:3306/test</prop>
            </props>
        </property>
    </bean>

    <bean id="dataSource2" class="org.cqipu.edu.bean.MyDataSource2">
        <property name="properties">
            <props>
                <prop key="driver">com.mysql.cj.jdbc.Driver</prop>
                <prop key="url">jdbc:mysql://localhost:3306/test</prop>
            </props>
        </property>
    </bean>

</beans>

```

- MyDataSource1.java

```

package org.cqipu.edu.bean;
import java.util.Properties;

public class MyDataSource1 {
    private Properties properties;

    public void setProperties(Properties properties) {
        this.properties = properties;
    }

    @Override
    public String toString() {
        return "MyDataSource1{" +
            "properties=" + properties +
            '}';
    }
}

```

```
}  
}
```

- MyDataSource2

```
package org.cqipu.edu.bean;  
import java.util.Properties;  
  
public class MyDataSource2 {  
    private Properties properties;  
  
    public void setProperties(Properties properties) {  
        this.properties = properties;  
    }  
  
    @Override  
    public String toString() {  
        return "MyDataSource2{" +  
            "properties=" + properties +  
            '}';  
    }  
}
```

测试程序

```
@Test  
public void testUtil(){  
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-util.xml");  
  
    MyDataSource1 dataSource1 = applicationContext.getBean("dataSource1", MyDataSource1.class);  
    System.out.println(dataSource1);  
  
    MyDataSource2 dataSource2 = applicationContext.getBean("dataSource2", MyDataSource2.class);  
    System.out.println(dataSource2);  
}
```

✓ 测试 已通过: 1共 1 个测试 - 417毫秒

D:\Java21\bin\java.exe ...

MyDataSource1{properties={driver=com.mysql.cj.jdbc.Driver, url=jdbc:mysql://localhost:3306/test}}

MyDataSource2{properties={driver=com.mysql.cj.jdbc.Driver, url=jdbc:mysql://localhost:3306/test}}

基于XML的自动装配

自动装配的本质是“Spring 根据预设规则，自动为 Bean 的对象类型属性匹配并注入容器中的对应 Bean”，无需手动写 `<property ref="xxx"/>` 或

`<constructor-arg ref="xxx"/>`；但自动装配仅适用于“对象类型属性”，简单类型仍需手动注入，且有明确的匹配规则

(byName/byType/constructor)。

- 定义
 - 手动装配（之前学的）：必须显式通过 `<property ref="beanId"/>` 指定要注入的 Bean；
 - 自动装配：Spring 根据你指定的“匹配规则”（比如按属性名、按类型），从容器中自动找对应的 Bean，注入到当前 Bean 的对象属性中，只需要给 `<bean>` 加 `autowire="规则"`，无需写 ref。

维度	手动装配	自动装配
配置写法	<property name="addr" ref="address"/>	<bean id="user" class="..." autowire="byName"/>
匹配逻辑	人工指定 Bean id	Spring 按规则自动匹配
适用范围	所有类型（对象 + 简单类型）	仅对象类型属性（简单类型需手动）
透明性	清晰（一眼看到注入的 Bean）	不透明（需看规则 + Bean 命名 / 类型）
简化程度	繁琐（每个对象属性都要写 ref）	简洁（一行 autowire 搞定）

- 前提
 - 仅针对对象类型属性：String/int/boolean 等简单类型无法自动装配，仍需手动写 value="" ；
 - 依赖 setter / 构造方法：
 - byName/byType：目标 Bean 的对象属性必须有标准 setter 方法；
 - constructor：目标 Bean 必须有对应的构造方法；
 - 被装配的 Bean 必须在容器中：要注入的 Bean 必须配置 <bean> ，且在 Spring 容器中；
 - 唯一性约束（byType/constructor）：按类型匹配时，容器中该类型的 Bean 必须唯一，否则报 NoUniqueBeanDefinitionException。

Spring XML 支持 4 种自动装配规则，如下：

规则	核心匹配逻辑	关键要求	优点	缺点
byName	按“对象属性名”匹配容器中“Bean id/name”	属性名 = Bean id/name	匹配精准，不易冲突	依赖 Bean 命名规范（属性名和 id 必须一致）
byType	按“对象属性类型”匹配容器中的 Bean	容器中该类型的 Bean 唯一	无需关注命名，更灵活	同类型 Bean 多个时直接报错
constructor	按构造方法参数的“类型 / 名称”匹配 Bean	有对应参数类型的构造方法，且类型唯一	适配构造器注入场景	构造方法参数类型多易冲突
autodetect	先试 constructor，再试 byType	无特殊要求	-	废弃，不推荐使用

根据名称自动装配

- UserDao.java

```
package org.cqipu.edu.dao;

public class UserDao {
    public void insert(){
        System.out.println("执行UserDaoImpl的insert方法：插入用户数据");
    }
}
```

- UserService.java

```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
public class UserService {
    private UserDao aaa;
```

```
// 这个set方法非常关键
public void setAaa(UserDao aaa) {
    this.aaa = aaa;
}
public void save(){
    aaa.insert();
}
}
```

- spring-utowire.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="userService" class="org.cqipu.edu.service.UserService" autowire="byName"/>

    <bean id="aaa" class="org.cqipu.edu.dao.UserDao"/>
</beans>
```

- 测试程序

```
@Test
public void testAutowireByName(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-utowire.xml");
    UserService userService = applicationContext.getBean("userService", UserService.class);
    userService.save();
}
```

运行结果：

✓ 测试 已通过: 1共 1 个测试 – 554毫秒

D:\Java21\bin\java.exe ...

执行UserDaoImpl的insert方法：插入用户数据

我们来测试一下，byName装配是和属性名有关还是和set方法名有关系：

```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
public class UserService {
    private UserDao aaa; // 这个不变

    // 这个set方法非常关键
    // public void setAaa(UserDao aaa) {
    //     this.aaa = aaa;
    // }

    // set方法名变化了
    public void setDao(UserDao aaa){
        this.aaa = aaa;
    }
    public void save(){
        aaa.insert();
    }
}
```


测试结果：

❗ 测试 失败: 1共 1 个测试 - 457毫秒

D:\Java21\bin\java.exe ...

java.lang.NullPointerException: Cannot invoke "org.cqipu.edu.dao.UserDao.insert()" because "this.aaa" is null

at org.cqipu.edu.service.UserService.save(UserService.java:37)

> at org.cqipu.edu.DITest.testAutowireByName(DITest.java:132) <25 个内部行>

通过测试得知，aaa属性并没有赋值成功。也就是并没有装配成功。

我们将spring配置文件修改以下：

- spring-autowire.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="userService" class="org.cqipu.edu.service.UserService" autowire="byName"/>

<!--    <bean id="aaa" class="org.cqipu.edu.dao.UserDao"/>-->

    <!-- 这个id修改了-->
    <bean id="dao" class="org.cqipu.edu.dao.UserDao"/>
</beans>
```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 448毫秒

D:\Java21\bin\java.exe ...

执行UserDaoImpl的insert方法：插入用户数据

根据类型自动装配

- AccountDao.java

```
package org.cqipu.edu.dao;

public class AccountDao {
    public void insert(){
        System.out.println("正在保存账户信息");
    }
}
```

- AccountService.java

```

package org.cqipu.edu.service;
import org.cqipu.edu.dao.AccountDao;
public class AccountService {
    private AccountDao accountDao;

    public void setAccountDao(AccountDao accountDao) {
        this.accountDao = accountDao;
    }

    public void save(){
        accountDao.insert();
    }
}

```

- spring-autowire01.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <!--byType 表示根据类型自动装配-->
    <bean id="accountService" class="org.cqipu.edu.service.AccountService" autowire="byType"/>

    <bean class="org.cqipu.edu.dao.AccountDao"/>
</beans>

```

测试程序

```

@Test
public void testAutowireByType(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-autowire01.xml");
    AccountService accountService = applicationContext.getBean("accountService", AccountService.class);
    accountService.save();
}

```

✓ 测试 已通过: 1共 1 个测试 – 463毫秒

D:\Java21\bin\java.exe ...

正在保存账户信息

我们把UserService中的set方法注释掉，再执行：

Tests failed: 1 of 1 test – 325 ms

C:\dev\Java\jdk-17.0.4\bin\java.exe ...

java.lang.NullPointerException: Cannot invoke "com.powernode.spring6.dao.AccountDao.insert()" because "this.accountDao" is null

at com.powernode.spring6.service.AccountService.save(AccountService.java:19)

at com.powernode.spring6.test.DITest.testAutowireByType(DITest.java:24) <27 internal lines>

老杜

可以看到无论是byName还是byType，在装配的时候都是基于set方法的。所以set方法是必须要提供的。提供构造方法是不行的，大家可以测试一下。这里就不再赘述。

如果byType，根据类型装配时，如果配置文件中有两个类型一样的bean会出现什么问题呢？

- spring-autoload01.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <!--byType表示根据类型自动装配-->
    <bean id="accountService" class="org.cqipu.edu.service.AccountService" autowire="byType"/>

    <bean id="x" class="org.cqipu.edu.dao.AccountDao"/>
    <bean id="y" class="org.cqipu.edu.dao.AccountDao"/>
</beans>
```

 spring-autoload01.xml D:\java_Spring6_study\spring6-dependency-injection-demo03\src\main\resources 6 个问题

❗ 无法自动装配。存在多个 'AccountDao' 类型的 Bean。Bean: y,x。属性: 'accountDao':6

测试结果

D:\Java21\bin\java.exe ...

```
org.springframework.beans.factory.UnsatisfiedDependencyException: Error creating bean with name 'accountService' defined in class
path resource [spring-autoload01.xml]: Unsatisfied dependency expressed through bean property 'accountDao': No qualifying bean of
type 'org.cqipu.edu.dao.AccountDao' available: expected single matching bean but found 2: x,y
```

测试结果说明了，当byType进行自动装配的时候，配置文件中某种类型的Bean必须是唯一的，不能出现多个。

Spring引入外部属性配置文件

我们都知道编写数据源的时候是需要连接数据库的信息的，例如：driver url username password等信息。这些信息可以单独写到一个属性配置文件中吗，这样用户修改起来会更加的方便。当然可以。

- 第一步：写一个数据源类，提供相关属性。
 - MyDataSource.java

```
package org.cqipu.edu.bean;

import javax.sql.DataSource;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.SQLFeatureNotSupportedException;
import java.util.logging.Logger;

public class MyDataSource implements DataSource {
    private String driver;
    private String url;
    private String username;
    private String password;

    public void setDriver(String driver) {
        this.driver = driver;
    }

    public void setUrl(String url) {
        this.url = url;
    }
}
```

```

public void setUsername(String username) {
    this.username = username;
}

public void setPassword(String password) {
    this.password = password;
}

@Override
public String toString() {
    return "MyDataSource{" +
        "driver='" + driver + '\'' +
        ", url='" + url + '\'' +
        ", username='" + username + '\'' +
        ", password='" + password + '\'' +
        '}';
}

@Override
public Connection getConnection() throws SQLException {
    return null;
}

@Override
public Connection getConnection(String username, String password) throws SQLException {
    return null;
}

@Override
public PrintWriter getLogWriter() throws SQLException {
    return null;
}

@Override
public void setLogWriter(PrintWriter out) throws SQLException {
}

@Override
public void setLoginTimeout(int seconds) throws SQLException {
}

@Override
public int getLoginTimeout() throws SQLException {
    return 0;
}

@Override
public Logger getParentLogger() throws SQLFeatureNotSupportedException {
    return null;
}

@Override
public <T> T unwrap(Class<T> iface) throws SQLException {
    return null;
}

@Override
public boolean isWrapperFor(Class<?> iface) throws SQLException {
    return false;
}
}

```

- 第二步：在类路径下新建jdbc.properties文件，并配置信息。

- o jdbc.properties

```
driver=com.mysql.cj.jdbc.Driver
url=jdbc:mysql://localhost:3306/spring
username=root
password=root123
```

- 第三步：在spring配置文件中引入context命名空间。
 - o spring-properties.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">

</beans>
```

- 第四步：在spring中配置使用jdbc.properties文件。
 - o spring-properties.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">

    <context:property-placeholder location="jdbc.properties"/>

    <bean id="dataSource" class="org.cqipu.edu.bean.MyDataSource">
        <property name="driver" value="${driver}"/>
        <property name="url" value="${url}"/>
        <property name="username" value="${username}"/>
        <property name="password" value="${password}"/>
    </bean>
</beans>
```

测试程序

```
@Test
public void testProperties(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-properties.xml");
    MyDataSource dataSource = applicationContext.getBean("dataSource", MyDataSource.class);
    System.out.println(dataSource);
}
```

测试结果：

✓ 测试 已通过: 1共 1 个测试 - 496毫秒

```
D:\Java21\bin\java.exe ...
```

```
MyDataSource{driver='com.mysql.cj.jdbc.Driver', url='jdbc:mysql://localhost:3306/spring', username='jack', password='root123'}
```

Bean 的作用域

Bean 的作用域，本质是Spring IoC 容器对 Bean 实例的创建规则、存在范围、生命周期的约束，简单说就是Spring 容器会根据 Bean 的作用域，决定创建多少个 Bean 实例、什么时候创建、在哪个范围内可以使用这个实例。

Spring 中 Bean 的作用域分核心基础作用域（所有环境通用）和Web 环境专属作用域（仅在 Web 项目中生效）。

singleton

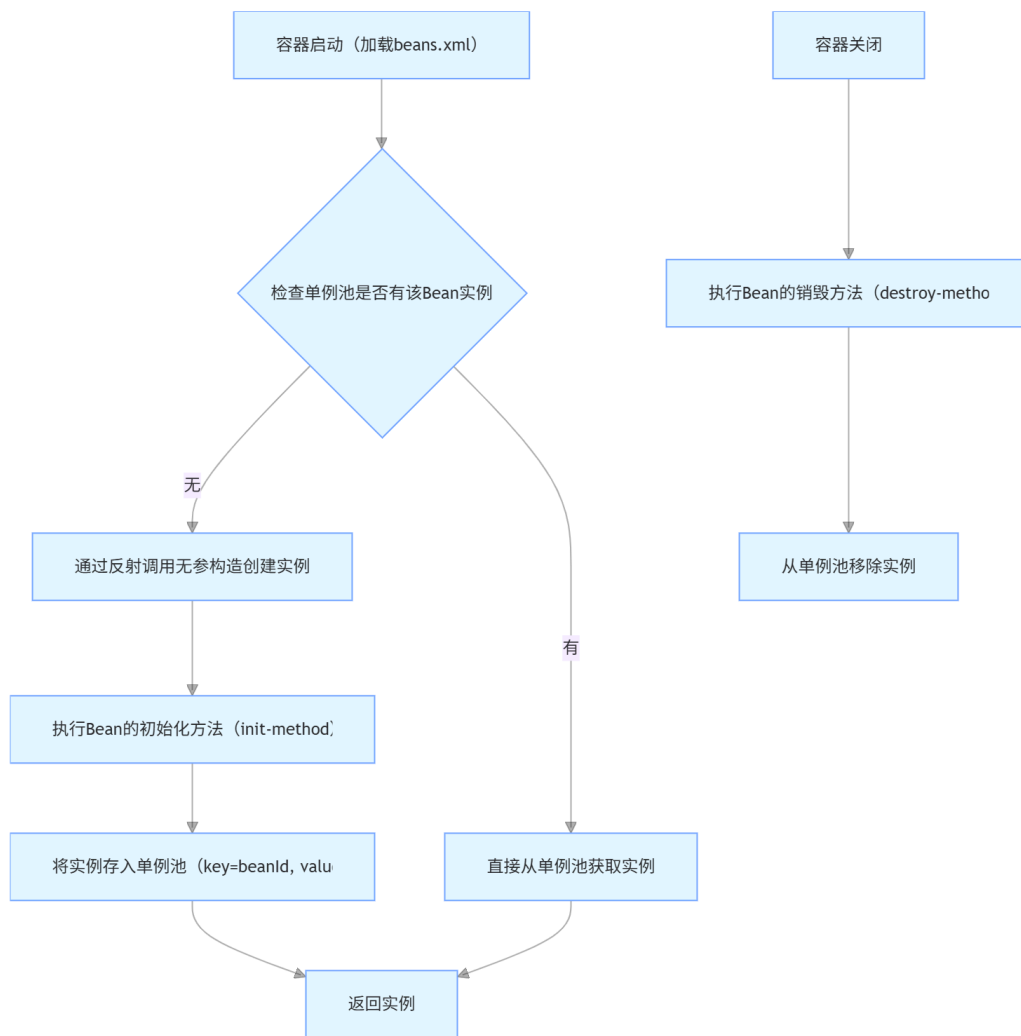
singleton是 Spring IoC 容器中 Bean 的默认作用域，其核心特征是：

在同一个 Spring IoC 容器中，针对同一个 Bean 定义，即 `<bean>` 标签，只会创建一个唯一的实例，该实例被存入 Spring 的单例池（DefaultSingletonBeanRegistry）中缓存，后续所有getBean()调用都直接从单例池获取该实例，不会新建；该实例的生命周期完全由 Spring 容器管理（创建→初始化→使用→销毁）。

- 设计模式的单例：JVM 级别唯一，整个 JVM 中只有一个实例；
- Spring 的 singleton：容器级别唯一，一个 Spring 容器对应一个实例，多个容器可以有多个实例。

singleton 底层实现原理

Spring 容器对 singleton Bean 的管理核心依赖「单例池」（本质是ConcurrentHashMap），核心流程如下：



- 关键点

- 单例池：Spring 底层用 `Map<String, Object> singletonObjects` 存储单例 Bean，线程安全；
- 创建时机：默认是立即加载，容器启动时就创建，而非第一次`getBean()`时；
- 获取逻辑：每次`getBean()`先查单例池，有则直接返回，无则创建，但 singleton Bean 只会创建一次；
- 销毁逻辑：只有容器完全关闭时，才会调用单例 Bean 的销毁方法，释放资源。

默认情况下，Spring的IoC容器创建的Bean对象是单例的。来测试一下：

- SpringBean

```
package org.cqipu.edu.bean;

public class SpringBean {
}
```

- spring-scope.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="sb" class="org.cqipu.edu.bean.SpringBean" />
</beans>
```

- 测试程序



```
@Test
public void testScope(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-scope.xml");

    SpringBean sb1 = applicationContext.getBean("sb", SpringBean.class);
    System.out.println(sb1);

    SpringBean sb2 = applicationContext.getBean("sb", SpringBean.class);
    System.out.println(sb2);
}
```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 526毫秒

```
D:\Java21\bin\java.exe ...
org.cqipu.edu.bean.SpringBean@6995bf68
org.cqipu.edu.bean.SpringBean@6995bf68
```

通过测试得知：Spring的IoC容器中，默认情况下，Bean对象是单例的。

这个对象在什么时候创建的呢？可以为SpringBean提供一个无参数构造方法，测试一下，如下：

- SpringBean



```
package org.cqipu.edu.bean;

public class SpringBean {
    public SpringBean() {
        System.out.println("SpringBean的无参数构造方法执行。");
    }
}
```

将测试程序中getBean()所在行代码注释掉：

- 测试程序



```
@Test
public void testScope(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-scope.xml");
}
```


✓ 测试 已通过: 1共 1 个测试 – 431毫秒

D:\Java21\bin\java.exe ...

SpringBean的无参数构造方法执行。

通过测试得知，默认情况下，Bean对象的创建是在初始化Spring上下文的时候就完成的。

prototype

prototype是 Spring 中与singleton完全对立的基础作用域，核心特征：

在同一个 Spring IoC 容器中，针对同一个prototype Bean 定义，每次调用getBean()都会创建一个全新的实例；Spring 容器不缓存该类型 Bean 实例，也不管理其完整生命周期（仅负责创建，不负责初始化后的任何管理，包括销毁）。

维度	singleton	prototype
实例数量	容器内唯一	每次 getBean 创建新实例
创建时机	默认立即加载（容器启动）	延迟加载（getBean 时）
缓存策略	存入单例池（ConcurrentHashMap）	不缓存，创建后直接返回
生命周期管理	容器全程管理（创建→初始化→销毁）	仅管创建 + 初始化，不管销毁

1. 无缓存：prototype Bean 不会进入 Spring 的单例池，每次getBean()都是“全新创建→初始化→返回”的完整流程；
2. 懒加载天然生效：无论是否配置lazy-init，prototype Bean 都只会在getBean()时创建（lazy-init对prototype无意义）；
3. 销毁无管理：Spring 容器不会记录prototype Bean 的引用，因此容器关闭时不会调用其destroy-method，实例销毁完全由 JVM 的垃圾回收（GC）负责；
4. 初始化仍执行：虽然不管理销毁，但prototype Bean 的初始化方法（init-method）仍会在创建实例后执行（和singleton一致）。

如果想让Spring的Bean对象以多例的形式存在，可以在bean标签中指定scope属性的值为：prototype，这样Spring会在每一次执行getBean()方法的时候创建Bean对象，调用几次则创建几次。

- spring-scope.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="sb" class="org.cqipu.edu.bean.SpringBean" scope="prototype" />
</beans>
```

测试程序



@Test

```
public void testScope(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-scope.xml");

    SpringBean sb1 = applicationContext.getBean("sb", SpringBean.class);
    System.out.println(sb1);

    SpringBean sb2 = applicationContext.getBean("sb", SpringBean.class);
    System.out.println(sb2);
}
```

测试结果

```
D:\Java21\bin\java.exe ...
SpringBean的无参数构造方法执行。
org.cqipu.edu.bean.SpringBean@5a18cd76
SpringBean的无参数构造方法执行。
org.cqipu.edu.bean.SpringBean@3da30852
```

我们可以把测试代码中的getBean()方法所在行代码注释掉：

测试代码



@Test

```
public void testScope(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-scope.xml");
}
```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 410毫秒

```
D:\Java21\bin\java.exe ...
```

可以看到这一次在初始化Spring上下文的时候，并没有创建Bean对象。

那你可能会问：scope如果没有配置，它的默认值是什么呢？默认值是singleton，单例的。

- spring-scope.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="sb" class="org.cqipu.edu.bean.SpringBean" scope="singleton" />
</beans>
```

- 测试程序



@Test

```
public void testScope(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-scope.xml");

    SpringBean sb1 = applicationContext.getBean("sb", SpringBean.class);
    System.out.println(sb1);

    SpringBean sb2 = applicationContext.getBean("sb", SpringBean.class);
    System.out.println(sb2);
}
```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 413毫秒

D:\Java21\bin\java.exe ...

SpringBean的无参数构造方法执行。

org.cqipu.edu.bean.SpringBean@3568f9d2

org.cqipu.edu.bean.SpringBean@3568f9d2

通过测试得知，没有指定scope属性时，默认是singleton单例的。

其它scope

scope属性的值不止两个，它一共包括8个选项：

- singleton：默认的，单例。
- prototype：原型。每调用一次getBean()方法则获取一个新的Bean对象。或每次注入的时候都是新对象。
- request：一个请求对应一个Bean。仅限于在WEB应用中使用。
- session：一个会话对应一个Bean。仅限于在WEB应用中使用。
- global session：portlet应用中专用的。如果在Servlet的WEB应用中使用global session的话，和session一个效果。（portlet和servlet都是规范。servlet运行在servlet容器中，例如Tomcat。portlet运行在portlet容器中。）
- application：一个应用对应一个Bean。仅限于在WEB应用中使用。
- websocket：一个websocket生命周期对应一个Bean。仅限于在WEB应用中使用。
- 自定义scope：很少使用。

接下来咱们自定义一个Scope，线程级别的Scope，在同一个线程中，获取的Bean都是同一个。跨线程则是不同的对象：（以下内容作为了解）

- 第一步：自定义Scope。（实现Scope接口）
 - spring内置了线程范围的类：org.springframework.context.support.SimpleThreadScope，可以直接用。
- 第二步：将自定义的Scope注册到Spring容器中。
 - spring-scope.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- 注册自定义Scope -->
    <bean class="org.springframework.beans.factory.config.CustomScopeConfigurer">
        <property name="scopes">
```

```

        <map>
            <entry key="myThread">
                <bean class="org.springframework.context.support.SimpleThreadScope"/>
            </entry>
        </map>
    </property>
</bean>

<!-- 普通Bean定义 -->
<bean id="sb" class="org.cqipu.edu.bean.SpringBean" scope="myThread" />
</beans>

```

- 第三步：使用Scope。
 - spring-scope.xml

```

<bean id="sb" class="org.cqipu.edu.bean.SpringBean" scope="myThread" />

```

编写测试程序：

```

@Test
public void testCustomScope(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-scope.xml");
    SpringBean sb1 = applicationContext.getBean("sb", SpringBean.class);
    SpringBean sb2 = applicationContext.getBean("sb", SpringBean.class);
    System.out.println(sb1);
    System.out.println(sb2);
    // 启动线程
    new Thread(new Runnable() {
        @Override
        public void run() {
            SpringBean a = applicationContext.getBean("sb", SpringBean.class);
            SpringBean b = applicationContext.getBean("sb", SpringBean.class);
            System.out.println(a);
            System.out.println(b);
        }
    }).start();
}

```

✓ 测试 已通过: 1共 1 个测试 - 469毫秒

D:\Java21\bin\java.exe ...

SpringBean的无参数构造方法执行。

org.cqipu.edu.bean.SpringBean@74c79fa2

org.cqipu.edu.bean.SpringBean@74c79fa2

SpringBean的无参数构造方法执行。

org.cqipu.edu.bean.SpringBean@36a47be3

org.cqipu.edu.bean.SpringBean@36a47be3

GoF之工厂模式

- 设计模式：一种可以被重复利用的解决方案。
- GoF（Gang of Four），中文名——四人组。
- 《Design Patterns: Elements of Reusable Object-Oriented Software》（即《设计模式》一书），1995年由 Erich Gamma、Richard Helm、Ralph Johnson 和 John Vlissides 合著。这几位作者常被称为“四人组”（Gang of Four）。
- 该书中描述了23种设计模式。我们平常所说的设计模式就是指这23种设计模式。

- 不过除了GoF23种设计模式之外，还有其它的设计模式，比如：JavaEE的设计模式（DAO模式、MVC模式等）。
- GoF23种设计模式可分为三大类：
 - **创建型**（5个）：解决对象创建问题。
 - 单例模式
 - 工厂方法模式
 - 抽象工厂模式
 - 建造者模式
 - 原型模式
 - **结构型**（7个）：一些类或对象组合在一起的经典结构。
 - 代理模式
 - 装饰模式
 - 适配器模式
 - 组合模式
 - 享元模式
 - 外观模式
 - 桥接模式
 - **行为型**（11个）：解决类或对象之间的交互问题。
 - 策略模式
 - 模板方法模式
 - 责任链模式
 - 观察者模式
 - 迭代子模式
 - 命令模式
 - 备忘录模式
 - 状态模式
 - 访问者模式
 - 中介者模式
 - 解释器模式
- 工厂模式是解决对象创建问题的，所以工厂模式属于创建型设计模式。这里为什么学习工厂模式呢？这是因为Spring框架底层使用了大量的工厂模式。

工厂模式的三种形态

工厂模式通常有三种形态：

- 第一种：简单工厂模式（Simple Factory）：不属于23种设计模式之一。简单工厂模式又叫做：静态 工厂方法模式。简单工厂模式是工厂方法模式的一种特殊实现。
- 第二种：工厂方法模式（Factory Method）：是23种设计模式之一。
- 第三种：抽象工厂模式（Abstract Factory）：是23种设计模式之一。

简单工厂模式

简单工厂模式的角色包括三个：

- 抽象产品 角色
- 具体产品 角色

- 工厂类 角色

新建模块：spring6-factory-pattern-dao-demo04

简单工厂模式的代码如下：

抽象产品角色：

- Weapon

```
● ● ●
package org.cqipu.edu.factory;

public abstract class Weapon {
    /**
     * 所有的武器都有攻击行为
     */
    public abstract void attack();
}
```

- Tank

```
● ● ●
package org.cqipu.edu.factory;

/**
 * 坦克（具体产品角色）
 */
public class Tank extends Weapon{
    @Override
    public void attack() {
        System.out.println("坦克开炮！");
    }
}
```

- Fighter

```
● ● ●
package org.cqipu.edu.factory;

/**
 * 战斗机（具体产品角色）
 */
public class Fighter extends Weapon{
    @Override
    public void attack() {
        System.out.println("战斗机投下原子弹！");
    }
}
```

- Dagger

```

package org.cqipu.edu.factory;

/**
 * 匕首（具体产品角色）
 */
public class Dagger extends Weapon{
    @Override
    public void attack() {
        System.out.println("砍他丫的！");
    }
}

```

工厂类角色：

- WeaponFactory

```

package org.cqipu.edu.factory;

/**
 * 工厂类角色
 */
public class WeaponFactory {
    /**
     * 根据不同的武器类型生产武器
     * @param weaponType 武器类型
     * @return 武器对象
     */
    public static Weapon get(String weaponType){
        if (weaponType == null || weaponType.trim().length() == 0) {
            return null;
        }
        Weapon weapon = null;
        if ("TANK".equals(weaponType)) {
            weapon = new Tank();
        } else if ("FIGHTER".equals(weaponType)) {
            weapon = new Fighter();
        } else if ("DAGGER".equals(weaponType)) {
            weapon = new Dagger();
        } else {
            throw new RuntimeException("不支持该武器！");
        }
        return weapon;
    }
}

```

测试程序（客户端程序）：

- Client

```

package org.cqipu.edu;

import org.cqipu.edu.factory.WeaponFactory;
import org.cqipu.edu.factory.Weapon;

/**
 */
public class Client {
    public static void main(String[] args) {
        Weapon weapon1 = WeaponFactory.get("TANK");
        weapon1.attack();

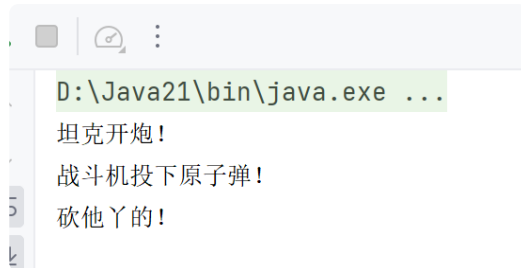
        Weapon weapon2 = WeaponFactory.get("FIGHTER");
    }
}

```

```
        weapon2.attack();

        Weapon weapon3 = WeaponFactory.get("DAGGER");
        weapon3.attack();
    }
}
```

测试结果



简单工厂模式的优点：

- 客户端程序不需要关心对象的创建细节，需要哪个对象时，只需要向工厂索要即可，初步实现了责任的分离。客户端只负责“消费”，工厂负责“生产”。生产和消费分离。

简单工厂模式的缺点：

- 缺点1：工厂类集中了所有产品的创造逻辑，形成一个无所不知的全能类，有人把它叫做上帝类。显然工厂类非常关键，不能出问题，一旦出问题，整个系统瘫痪。
- 缺点2：不符合OCP开闭原则，在进行系统扩展时，需要修改工厂类。

Spring中的BeanFactory就使用了简单工厂模式。

工厂方法模式

工厂方法模式既保留了简单工厂模式的优点，同时又解决了简单工厂模式的缺点。

工厂方法模式的角色包括：

- 抽象工厂角色
- 具体工厂角色
- 抽象产品角色
- 具体产品角色

代码如下：

- 抽象产品角色
 - Weapon

```
package org.cqipu.edu.factory;

public abstract class Weapon {
    /**
     * 所有的武器都有攻击行为
     */
    public abstract void attack();
}
```

- 具体产品角色
 - Gun



```
package org.cqipu.edu.factory;

/**
 * 具体产品角色
 */
public class Gun extends Weapon{
    @Override
    public void attack() {
        System.out.println("开枪射击!");
    }
}
```

- 具体产品角色
 - Fighter



```
package org.cqipu.edu.factory;

/**
 * 战斗机（具体产品角色）
 */
public class Fighter extends Weapon{
    @Override
    public void attack() {
        System.out.println("战斗机投下原子弹!");
    }
}
```

- 抽象工厂角色
 - WeaponFactory02



```
package org.cqipu.edu.factory;

/**
 * 武器工厂接口(抽象工厂角色)
 */
public interface WeaponFactory02 {
    Weapon get();
}
```

- 具体工厂角色
 - FighterFactory



```
package org.cqipu.edu.factory;

/**
 * 具体工厂角色
 */
public class FighterFactory implements WeaponFactory02{
    @Override
    public Weapon get() {
        return new Fighter();
    }
}
```

- GunFactory



```
package org.cqipu.edu.factory;

/**
 * 具体工厂角色
 */
public class GunFactory implements WeaponFactory02{
    @Override
    public Weapon get() {
        return new Gun();
    }
}
```

测试代码



```
package org.cqipu.edu;
import org.cqipu.edu.factory.WeaponFactory02;
import org.cqipu.edu.factory.Weapon;
import org.cqipu.edu.factory.GunFactory;
import org.cqipu.edu.factory.FighterFactory;

public class Client02 {
    public static void main(String[] args) {
        WeaponFactory02 factory = new GunFactory();
        Weapon weapon = factory.get();
        weapon.attack();

        WeaponFactory02 factory1 = new FighterFactory();
        Weapon weapon1 = factory1.get();
        weapon1.attack();
    }
}
```

测试结果



如果想扩展一个新的产品，只要新增一个产品类，再新增一个该产品对应的工厂即可，例如新增：匕首

- 增加：具体产品角色
 - Dagger



```
package org.cqipu.edu.factory;

/**
 * 匕首（具体产品角色）
 */
public class Dagger extends Weapon{
    @Override
    public void attack() {
        System.out.println("砍他丫的！");
    }
}
```

- 增加：具体工厂角色

- DaggerFactory

```
package org.cqipu.edu.factory;

public class DaggerFactory implements WeaponFactory02{
    @Override
    public Weapon get() {
        return new Dagger();
    }
}
```

测试程序

```
package org.cqipu.edu;

import org.cqipu.edu.factory.WeaponFactory02;
import org.cqipu.edu.factory.Weapon;
import org.cqipu.edu.factory.GunFactory;
import org.cqipu.edu.factory.FighterFactory;
import org.cqipu.edu.factory.DaggerFactory;

public class Client03 {
    public static void main(String[] args) {
        WeaponFactory02 factory = new GunFactory();
        Weapon weapon = factory.get();
        weapon.attack();

        WeaponFactory02 factory1 = new FighterFactory();
        Weapon weapon1 = factory1.get();
        weapon1.attack();

        WeaponFactory02 factory2 = new DaggerFactory();
        Weapon weapon2 = factory2.get();
        weapon2.attack();
    }
}
```

测试结果

```
D:\Java21\bin\java.exe ...
```

```
开枪射击！
战斗机投下原子弹！
砍他丫的！
```

我们可以看到在进行功能扩展的时候，不需要修改之前的源代码，显然工厂方法模式符合OCP原则。

工厂方法模式的优点：

- 一个调用者想创建一个对象，只要知道其名称就可以了。
- 扩展性高，如果想增加一个产品，只要扩展一个工厂类就可以。
- 屏蔽产品的具体实现，调用者只关心产品的接口。

工厂方法模式的缺点：

- 每次增加一个产品时，都需要增加一个具体类和对象实现工厂，使得系统中类的个数成倍增加，在一定程度上增加了系统的复杂度，同时也增加了系统具体类的依赖。这并不是什么好事。

抽象工厂模式（了解）

抽象工厂模式相对于工厂方法模式来说，就是工厂方法模式是针对一个产品系列的，而抽象工厂模式是针对多个产品系列的，即工厂方法模式是一个产品系列一个工厂类，而抽象工厂模式是多个产品系列一个工厂类。

抽象工厂模式特点：抽象工厂模式是所有形态的工厂模式中最为抽象和最具一般性的一种形态。抽象工厂模式是指当有多个抽象角色时，使用的一种工厂模式。抽象工厂模式可以向客户端提供一个接口，使客户端在不必指定产品的具体的情况下，创建多个产品族中的产品对象。它有多个抽象产品类，每个抽象产品类可以派生出多个具体产品类，一个抽象工厂类，可以派生出多个具体工厂类，每个具体工厂类可以创建多个具体产品类的实例。每一个模式都是针对一定问题的解决方案，工厂方法模式针对的是一个产品等级结构；而抽象工厂模式针对的是多个产品等级结果。

抽象工厂中包含4个角色：

- 抽象工厂角色
- 具体工厂角色
- 抽象产品角色
- 具体产品角色

抽象工厂模式的类图如下：

抽象工厂模式代码如下：

- 第一部分：武器产品族
 - Weapon.java

```
● ● ●
package org.cqipu.edu.product;

/**
 * 武器产品族
 */
public abstract class Weapon {
    public abstract void attack();
}
```

- Gun.java

```
● ● ●
package org.cqipu.edu.product;

/**
 * 武器产品族中的产品等级1
 */
public class Gun extends Weapon{
    @Override
    public void attack() {
        System.out.println("开枪射击！");
    }
}
```

- Dagger.java



```
package org.cqipu.edu.product;

/**
 * 武器产品族中的产品等级2
 */
public class Dagger extends Weapon{
    @Override
    public void attack() {
        System.out.println("砍丫的!");
    }
}
```

- 第二部分：水果产品族
 - Fruit.java



```
package org.cqipu.edu.product;

/**
 * 水果产品族
 */
public abstract class Fruit {
    /**
     * 所有果实都有一个成熟周期。
     */
    public abstract void ripeCycle();
}
```

- Orange.java



```
package org.cqipu.edu.product;

/**
 * 水果产品族中的产品等级1
 */
public class Orange extends Fruit{
    @Override
    public void ripeCycle() {
        System.out.println("橘子的成熟周期是10个月");
    }
}
```

- Apple.java



```
package org.cqipu.edu.product;

/**
 * 水果产品族中的产品等级2
 */
public class Apple extends Fruit{
    @Override
    public void ripeCycle() {
        System.out.println("苹果的成熟周期是8个月");
    }
}
```

- 第三部分：抽象工厂类
 - AbstractFactory.java



```
package org.cqipu.edu.factory;

import org.cqipu.edu.product.Fruit;
import org.cqipu.edu.product.Weapon;
/**
 * 抽象工厂
 */
public abstract class AbstractFactory {
    public abstract Weapon getWeapon(String type);
    public abstract Fruit getFruit(String type);
}
```

- 第四部分：具体工厂类
 - WeaponFactory03.java



```
package org.cqipu.edu.factory;

import org.cqipu.edu.product.Dagger;
import org.cqipu.edu.product.Fruit;
import org.cqipu.edu.product.Gun;
import org.cqipu.edu.product.Weapon;
/**
 * 武器族工厂
 */
public class WeaponFactory03 extends AbstractFactory{

    public Weapon getWeapon(String type){
        if (type == null || type.trim().length() == 0) {
            return null;
        }
        if ("Gun".equals(type)) {
            return new Gun();
        } else if ("Dagger".equals(type)) {
            return new Dagger();
        } else {
            throw new RuntimeException("无法生产该武器");
        }
    }

    @Override
    public Fruit getFruit(String type) {
        return null;
    }
}
```

- FruitFactory



```
package org.cqipu.edu.factory;

import org.cqipu.edu.product.*;
import org.cqipu.edu.product.Weapon;
/**
 * 水果族工厂
 */
public class FruitFactory extends AbstractFactory{
    @Override
    public Weapon getWeapon(String type) {
        return null;
    }
}
```

```

    }

    public Fruit getFruit(String type){
        if (type == null || type.trim().length() == 0) {
            return null;
        }
        if ("Orange".equals(type)) {
            return new Orange();
        } else if ("Apple".equals(type)) {
            return new Apple();
        } else {
            throw new RuntimeException("我家果园不产这种水果");
        }
    }
}

```

第五部分：测试程序

```

package org.cqipu.edu;

import org.cqipu.edu.factory.AbstractFactory;
import org.cqipu.edu.factory.FruitFactory;
import org.cqipu.edu.factory.WeaponFactory03;
import org.cqipu.edu.product.Fruit;
import org.cqipu.edu.product.Weapon;

public class Client04 {
    public static void main(String[] args) {
        // 客户端调用方法时只面向AbstractFactory调用方法。
        AbstractFactory factory = new WeaponFactory03(); // 注意：这里的new WeaponFactory()可以采用 简单工厂模式 进行隐藏。

        Weapon gun = factory.getWeapon("Gun");
        Weapon dagger = factory.getWeapon("Dagger");

        gun.attack();
        dagger.attack();

        AbstractFactory factory1 = new FruitFactory(); // 注意：这里的new FruitFactory()可以采用 简单工厂模式 进行隐藏。
        Fruit orange = factory1.getFruit("Orange");
        Fruit apple = factory1.getFruit("Apple");

        orange.ripeCycle();
        apple.ripeCycle();
    }
}

```

```

D:\Java21\bin\java.exe ...
开枪射击！
砍丫的！
橘子的成熟周期是10个月
苹果的成熟周期是8个月

```

抽象工厂模式的优缺点：

- 优点：当一个产品族中的多个对象被设计成一起工作时，它能保证客户端始终只使用同一个产品族中的对象。
- 缺点：产品族扩展非常困难，要增加一个系列的某一产品，既要在AbstractFactory里加代码，又要在具体的里面加代码。